

Document Number:	P0267R7
Date:	2018-02-10
Revises:	P0267R6
Reply to:	Michael B. McLaughlin mikebmcl@gmail.com Herb Sutter Microsoft Inc. hsutter@microsoft.com Jason Zink jzink_1@yahoo.com Guy Davidson guy@creative-assembly.com
Audience:	LEWG

A Proposal to Add 2D Graphics Rendering and Display to C++

Note: this is an early draft. It's known to be incomplet and incorrekt, and it has lots of bad fomattting.

Contents

Contents	ii
List of Tables	iv
List of Figures	v
1 Revision history	1
1.1 Revision 7	1
1.2 Revision 6	1
2 Scope	2
3 Normative references	3
4 Terms and definitions	4
5 Error reporting	10
6 Header <experimental/io2d> synopsis	11
7 Colors	12
7.1 Introduction to color	12
7.2 Color usage requirements	12
7.3 Class <code>rgba_color</code>	12
8 Linear algebra	24
8.1 Class <code>basic_point_2d</code>	24
8.2 Class <code>basic_matrix_2d</code>	27
9 Geometry	33
9.1 Class <code>basic_bounding_box</code>	33
9.2 Class <code>basic_circle</code>	35
10 Text rendering and display	37
11 Paths	38
11.1 Overview of paths	38
11.2 Path examples (Informative)	38
11.3 Figure items	42
11.4 Class <code>basic_interpreted_path</code>	65
11.5 Class <code>basic_path_builder</code>	66
12 Brushes	74
12.1 Overview of brushes	74
12.2 Gradient brushes	74
12.3 Enum class <code>wrap_mode</code>	78
12.4 Enum class <code>filter</code>	79

12.5	Enum class <code>brush_type</code>	80
12.6	Class <code>gradient_stop</code>	80
12.7	Class <code>basic_brush</code>	82
13	Surfaces	85
13.1	Enum class <code>antialias</code>	85
13.2	Enum class <code>fill_rule</code>	86
13.3	Enum class <code>line_cap</code>	87
13.4	Enum class <code>line_join</code>	87
13.5	Enum class <code>compositing_op</code>	88
13.6	Enum class <code>format</code>	94
13.7	Enum class <code>scaling</code>	95
13.8	Enum class <code>refresh_rate</code>	98
13.9	Enum class <code>image_file_format</code>	100
13.10	Class <code>basic_render_props</code>	100
13.11	Class <code>basic_brush_props</code>	101
13.12	Class <code>basic_clip_props</code>	103
13.13	Class <code>basic_stroke_props</code>	104
13.14	Class <code>basic_mask_props</code>	106
13.15	Overview of surface classes	107
13.16	Class <code>basic_image_surface</code>	116
13.17	Class <code>basic_output_surface</code>	122
13.18	Class <code>basic_unmanaged_output_surface</code>	129
14	Input	135
15	Standalone functions	136
15.1	Standalone functions synopsis	136
15.2	<code>format_stride_for_width</code>	136
15.3	<code>angle_for_point</code>	136
15.4	<code>point_for_angle</code>	136
15.5	<code>arc_start</code>	136
15.6	<code>arc_center</code>	137
15.7	<code>arc_end</code>	137
A	Bibliography	138

List of Tables

1	Class identifiers modified since R6	1
2	rgba_color static members values	19
3	Path interpretation state data	64
4	Figure item interpretation effects	64
5	wrap_mode enumerator meanings	78
6	filter enumerator meanings	79
7	brush_type enumerator meanings	80
8	antialias enumerator meanings	85
9	fill_rule enumerator meanings	86
10	line_cap enumerator meanings	87
11	line_join enumerator meanings	87
12	compositing_op basic enumerator meanings	90
13	compositing_op blend enumerator meanings	91
14	compositing_op hsl enumerator meanings	94
15	format enumerator meanings	94
16	scaling enumerator meanings	96
17	refresh_rate value meanings	99
18	imagefileformat enumerator meanings	100
19	surface rendering and composing operations	108
20	surface rendering and composing common state data	108
21	surface rendering and composing specific state data	108
22	Point transformations	109
23	Output surface observable state	112

List of Figures

1	Example 1 result	39
2	Example 2 result	40
3	Path example 3	41
4	Path example 4	42

1 Revision history [io2d.revisionhistory]

1.1 Revision 7 [io2d.revisionhistory.r7]

- ¹ The significant difference between R7 and R6 is the abstraction of the implementation into separate classes. These classes provide math and rendering support. The linear algebra and geometry classes are templated over any appropriate math support class, while the path, brush and surface classes are templated over any appropriate rendering support class.
- ² The reference implementation of this paper provides a software implementation of the math and rendering support classes. This is based on the Cairo library; indeed, so far the reference implementation has been based on Cairo. However, it is now possible to provide an implementation more appropriate to the target platform.
- ³ For example, a Windows implementation could provide support classes based on DirectX, while a Linux implementation could provide support classes based on OpenGL. In fact, any hardware vendor could provide a support library, targeting a specific implementation and their particular silicon if they wanted to exploit particular features of their hardware.
- ⁴ Additionally, the surface classes have been modified: now there are simply managed and unmanaged output surfaces, the latter of which offers developers the opportunity to take finer control of the drawing surface
- ⁵ The modified classes are as follows

Table 1 — Class identifiers modified since R6

R6 identifier	R7 identifier
vector_2d	basic_point_2d
matrix_2d	basic_matrix_2d
rectangle	basic_bounding_box
circle	basic_circle
path_group	basic_interpreted_path
path_builder	basic_path_builder
color_stop	gradient_stop
brush	basic_brush
render_props	basic_render_props
brush_props	basic_brush_props
clip_props	basic_clip_props
stroke_props	basic_stroke_props
mask_props	basic_mask_props
image_surface	basic_image_surface
display_surface	basic_output_surface

- ⁶ The `surface` class and the `mapped_surface` class have been withdrawn, while the `basic_unmanaged_output_surface` class has been introduced.
- ⁷ The reference implementation, including a software-only implementation of math and rendering support classes, is available at https://github.com/mikebmcl/P0267_RefImpl

1.2 Revision 6 [io2d.revisionhistory.r6]

- ¹ Presented to LEWG in Toronto, July 2017

2 Scope

[io2d.scope]

- ¹ This Technical Specification specifies requirements for implementations of an interface that computer programs written in the C++ programming language may use to render and display 2D computer graphics.

3 Normative references

[io2d.refs]

¹ The following documents are referred to in the text in such a way that some or all of their content constitutes requirements of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

- (1.1) — ISO/IEC 14882, *Programming languages — C++*
- (1.2) — ISO/IEC 2382 (all parts), *Information technology — Vocabulary*
- (1.3) — ISO/IEC 10646-1:1993, *Information technology — Universal Multiple-Octet Coded Character Set (UCS) — Part 1: Architecture and Basic Multilingual Plane*
- (1.4) — ISO/IEC 10918-1, *Information technology – Digital compression and coding of continuous-tone still images: Requirements and guidelines*
- (1.5) — ISO 12639, *Graphic technology – Prepress digital data exchange – Tag image file format for image technology (TIFF/IT)*
- (1.6) — ISO/IEC 15948 *Information technology – Computer graphics and image processing – Portable Network Graphics (PNG) Functional specification*
- (1.7) — ISO/IEC TR 19769:2004, *Information technology — Programming languages, their environments and system software interfaces — Extensions for the programming language C to support new character data types*
- (1.8) — ISO 15076-1, *Image technology colour management — Architecture, profile format and data structure — Part 1: Based on ICC.1:2004-10*
- (1.9) — IEC 61966-2-1, *Colour Measurement and Management in Multimedia Systems and Equipment - Part 2-1: Default RGB Colour Space - sRGB*
- (1.10) — ISO 32000-1:2008, *Document management — Portable document format — Part 1: PDF 1.7*
- (1.11) — ISO 80000-2:2009, *Quantities and units — Part 2: Mathematical signs and symbols to be used in the natural sciences and technology*
- (1.12) — Tantek Çelik et al., *CSS Color Module Level 3 — W3C Recommendation 07 June 2011*, Copyright © 2011 W3C® (MIT, ERCIM, Keio)

² The compressed image data format described in ISO/IEC 10918-1 is hereinafter called the *JPEG format*.

³ The tag image file format described in ISO 12639 is hereinafter called the *TIFF format*. The datastream and associated file format described in ISO/IEC 15948 is hereinafter called the *PNG format*.

⁵ The library described in ISO/IEC TR 19769:2004 is hereinafter called the *C Unicode TR*.

⁶ The document CSS Color Module Level 3 — W3C Recommendation 07 June 2011 is hereinafter called the *CSS Colors Specification*.

4 Terms and definitions [io2d.defns]

For the purposes of this document, the following terms and definitions apply. ISO and IEC maintain terminological databases for use in standardization at the following addresses:

- IEC Electropedia: available at <http://www.electropedia.org/>
- ISO Online browsing platform: available at <http://www.iso.org/obp>

¹ Terms that are used only in a small portion of this document are defined where they are used and italicized where they are defined.

4.1 [io2d.defns.stndcrdspace] **standard coordinate space**

Euclidean plane described by a Cartesian coordinate system where the first coordinate is measured along a horizontal axis, called the x axis, oriented from left to right, the second coordinate is measured along a vertical axis, called the y axis, oriented from top to bottom, and rotation of a point around the origin by a positive value expressed in radians is counterclockwise

4.2 [io2d.defns.point] **point**

⟨point⟩ coordinate designated by a floating point x axis value and a floating point y axis value within the *standard coordinate space* (4.1)

4.3 [io2d.defns.point.integral] **point**

⟨integral point⟩ coordinate designated by an integral x axis value and an integral y axis value within the *standard coordinate space* (4.1)

4.4 [io2d.defns.normalize] **normalize**

map a closed set of evenly spaced values in the range $[0, x]$ to an evenly spaced sequence of floating point values in the range $[0, 1]$ [*Note*: The definition of *normalize* given is the definition for normalizing unsigned input. Signed normalization, i.e. the mapping of a closed set of evenly spaced values in the range $[-x, x]$ to an evenly spaced sequence of floating point values in the range $[-1, 1]$ is not used in this Technical Specification. — *end note*]

4.5 [io2d.defns.aspectratio] **aspect ratio**

ratio of the width to the height of a rectangular area

4.6 [io2d.defns.colorsapce] **color space**

unambiguous mapping of values to colorimetric colors

4.7 [io2d.defns.gradientstop] **gradient stop**

point at which a color gradient changes from one color to the next

- 4.8** [io2d.defns.visdata]
visual data
 data representing color, transparency, or some combination thereof
- 4.9** [io2d.defns.graphicsdata]
graphics data
 ⟨graphics data⟩ *visual data* (4.8) stored in an unspecified form
- 4.10** [io2d.defns.channel]
channel
 component of *visual data* (4.8) with a defined bit size
- 4.11** [io2d.defns.colorchannel]
color channel
 component of *visual data* (4.8) representing color
- 4.12** [io2d.defns.alphachannel]
alpha channel
 component of *visual data* (4.8) representing transparency
- 4.13** [io2d.defns.visdatafmt]
visual data format
 specification that defines a total bit size, a set of one or more *channels* (4.10), and each *channel*'s role, bit size, and location relative to the upper (high-order) bit
- 4.14** [io2d.defns.premultipliedformat]
premultiplied format
 format with *color channels* (4.11) and an *alpha channel* (4.12) where each *color channel* is *normalized* (4.4) and then multiplied by the *normalized alpha channel* value [Example: Given the 32-bit non-premultiplied RGBA pixel with 8 bits per channel {255, 0, 0, 127} (half-transparent red), when normalized it would become {1.0f, 0.0f, 0.0f, 0.5f}. As such, in premultiplied, normalized format it would become {0.5f, 0.0f, 0.0f, 0.5f} as a result of multiplying each of the three color channels by the alpha channel value. — end example]
- 4.15** [io2d.defns.visdataelem]
visual data element
 item of *visual data* (4.8) with a defined *visual data format* (4.13)
- 4.16** [io2d.defns.pixel]
pixel
 discrete, rectangular *visual data element* (4.15)
- 4.17** [io2d.defns.graphics.raster]
graphics data
 ⟨raster graphics data⟩ *visual data* (4.8) stored as *pixels* (4.16) that is accessible as-if it was an array of rows of pixels beginning with the pixel at the *integral point* (0, 0) (4.3)
- 4.18** [io2d.defns.additivecolor]
additive color
 color defined by the emissive intensity of its *color channels* (4.11)
- 4.19** [io2d.defns.colormodel]
color model
 ideal, mathematical representation of colors which often uses *color channels* (4.11)

- 4.20** [io2d.defns.rgbcolormodel]
RGB color model
 ⟨RGB⟩ *additive* (4.18) *color model* (4.19) using red, green, and blue *color channels* (4.11)
- 4.21** [io2d.defns.rgbacolormodel]
RGBA color model
 ⟨RGBA⟩ *RGB color model* (4.20) with an *alpha channel* (4.12)
- 4.22** [io2d.defns.srgbcolorspace]
sRGB color space
 ⟨sRGB⟩ *additive* (4.18) *color space* (4.6) defined in IEC 61966-2-1 that is based on an *RGB color model* (4.20)
- 4.23** [io2d.defns.startpt]
start point
 point that begins a *segment* (4.28)
- 4.24** [io2d.defns.endpt]
end point
 point that ends a *segment* (4.28)
- 4.25** [io2d.defns.controlpt]
control point
 point, other than the start point and the end point, that is used in defining a curve
- 4.26** [io2d.defns.bezier.quadratic]
Bézier curve
 ⟨quadratic⟩ curve defined by the equation $f(t) = (1-t)^2 \times P_0 + 2 \times t \times (1-t) \times P_1 + t^2 \times t \times P_2$ where t is in the range $[0, 1]$, P_0 is the *start point* (4.23), P_1 is the *control point* (4.25), and P_2 is end point (4.24)
- 4.27** [io2d.defns.bezier.cubic]
Bézier curve
 ⟨cubic⟩ curve defined by the equation $f(t) = (1-t)^3 \times P_0 + 3 \times t \times (1-t)^2 \times P_1 + 3 \times t^2 \times (1-t) \times P_2 + t^3 \times t \times P_3$ where t is in the range $[0, 1]$, P_0 is the *start point* (4.23), P_1 is the first *control point* (4.25), P_2 is the second *control point*, and P_3 is the end point (4.24)
- 4.28** [io2d.defns.seg]
segment
 line, *Bézier curve* (4.26, 4.27), or arc
- 4.29** [io2d.defns.initialseg]
initial segment
segment (4.28) in a *figure* (4.39) whose *start point* (4.23) is not defined as being the *end point* (4.24) of another segment in the figure [*Note*: It is possible for the initial segment and final segment to be the same segment. — *end note*]
- 4.30** [io2d.defns.newfigpt]
new figure point
 point that is the *start point* (4.23) of the *initial segment* (4.29)
- 4.31** [io2d.defns.finalseg]
final segment
segment (4.28) in a *figure* (4.39) whose *end point* (4.24) does not define the *start point* (4.23) of any other segment [*Note*: It is possible for the initial segment and final segment to be the same segment. — *end note*]

- 4.32** [io2d.defns.currentpt]
current point
 point used as the *start point* (4.23) of a *segment* (4.28)
- 4.33** [io2d.defns.openfigure]
open figure
 figure (4.39) with one or more *segments* (4.28) where the *new figure point* (4.30) is not used to define the *end point* (4.24) of the figure's *final segment* (4.31) [*Note*: Even if the start point of the initial segment and the end point of the final segment are assigned the same coordinates, the figure is still an open figure. This is because the final segment's end point is not defined as being the new figure point but instead merely happens to have the same value as that point. — *end note*]
- 4.34** [io2d.defns.closedfigure]
closed figure
 figure (4.39) with one or more *segments* (4.28) where the *new figure point* (4.30) is used to define the *end point* (4.24) of the figure's *final segment* (4.31)
- 4.35** [io2d.defns.degenerateseg]
degenerate segment
segment (4.28) that has the same values for its *start point* (4.23), *end point* (4.24), and, if any, *control points* (4.25)
- 4.36** [io2d.defns.command.closefig]
command
 (close figure command) instruction that creates a line *segment* (4.28) with a *start point* (4.23) of *current point* (4.32) and an *end point* (4.24) of *new figure point* (4.30)
- 4.37** [io2d.defns.command.newfig]
command
 (new figure command) an instruction that creates a new *path* (4.40)
- 4.38** [io2d.defns.figitem]
figure item
segment (4.28), *new figure command* (4.37), *close figure command* (4.36), or *path command* (4.42)
- 4.39** [io2d.defns.figure]
figure
 collection of *figure items* (4.38) where the *end point* (4.24) of each *segment* (4.28) in the collection, except the *final segment* (4.31), defines the *start point* (4.23) of exactly one other segment in the collection
- 4.40** [io2d.defns.path]
path
 collection of *figures* (4.39)
- 4.41** [io2d.defns.pathtransform]
path transformation matrix
 affine transformation matrix used to apply affine transformations to the points in a *path* (4.40)
- 4.42** [io2d.defns.pathcommand]
path command
 instruction that modifies the *path transformation matrix* (4.41)

4.43 [io2d.defns.degenfigure] degenerate figure

figure (4.39) containing a *new figure command* (4.37), zero or more *degenerate segments* (4.35), zero or more *path commands* (4.42), and, optionally, a *close figure command* (4.36)

4.44 [io2d.defns.graphicssubsystem] graphics subsystem

collection of unspecified operating system and library functionality used to render and display 2D computer graphics

4.45 [io2d.defns.graphicsresource] graphics resource

⟨graphics resource⟩ object of unspecified type used by an implementation [*Note*: By its definition a graphics resource is an implementation detail. Often it will be a graphics subsystem object (e.g. a graphics device or a render target) or an aggregate composed of multiple graphics subsystem objects. However the only requirement placed upon a graphics resource is that the implementation is able to use it to provide the functionality required of the graphics resource. — *end note*]

4.46 [io2d.defns.graphicsresource.graphicsdata] graphics resource

⟨graphics data graphics resource⟩ object of unspecified type used by an implementation to provide access to, and allow manipulation of, *visual data* (4.8)

4.47 [io2d.defns.pixmap] pixmap

raster *graphics data graphics resource* (4.46)

4.48 [io2d.defns.filter] filter

mathematical function that determines the *visual data* (4.8) value of a point for a *graphics data graphics resource* (4.46)

4.49 [io2d.defns.compositionalgorithm] composition algorithm

algorithm that combines a source *visual data element* (4.15) and a destination *visual data element* producing a *visual data element* that has the same *visual data format* (4.13) as the destination *visual data element*

4.50 [io2d.defns.compose] compose

combine part or all of a source *graphics data graphics resource* (4.46) with a destination *graphics data graphics resource* in the manner specified by a *composition algorithm* (4.49)

4.51 [io2d.defns.composingoperation] composing operation

operation that performs *composing* (4.50)

4.52 [io2d.defns.artifact] artifact

error in the results of the application of a *composing operation* (4.51)

4.53 [io2d.defns.sample]
sample
 use a *filter* (4.48) to obtain the *visual data* (4.8) for a given point from a *graphics data graphics resource* (4.46)

4.54 [io2d.defns.alias]
aliasing
 presence of visual *artifacts* (4.52) in the results of rendering due to *sampling* (4.53) imperfections

4.55 [io2d.defns.antialias]
anti-aliasing
 application of a function or algorithm while *composing* (4.50) to reduce *aliasing* (4.54) [*Note:* Certain algorithms can produce “better” results, i.e. results with fewer artifacts or with less pronounced artifacts, when rendering text with anti-aliasing due to the nature of text rendering. As such, it often makes sense to provide the ability to choose one type of anti-aliasing for text rendering and another for all other rendering and to provide different sets of anti-aliasing types to choose from for each of the two operations. — *end note*]

4.56 [io2d.defns.graphicsstatedata]
graphics state data
 data which specify how some part of the process of rendering, or of a *composing operation* (4.51), shall be performed in part or in whole

4.57 [io2d.defns.render]
render
 transform a *path* (4.40) into graphics data in the manner specified by a set of *graphics state data* (4.56)

4.58 [io2d.defns.renderingoperation]
rendering operation
 operation that performs *rendering* (4.57)

4.59 [io2d.defns.renderingandcomposingop]
rendering and composing operation
 operation that is either a *composing operation* (4.51), or a *rendering operation* (4.58) followed by a *composing operation*

5 Error reporting

[io2d.err.report]

- ¹ 2D graphics library functions that can produce errors occasionally provide two overloads: one that throws an exception to report errors and another that reports errors using an `error_code` object. This provides for situations where errors are not truly exceptional.
- ² report errors as follows, unless otherwise specified:
- ³ When an error prevents the function from meeting its specifications:
 - (3.1) — Functions that do not take argument of type `error_code&` throw an exception of type `system_error` or of a `system_error`-derived type. The exception object shall include the enumerator specified by the function as part of its observable state.
 - (3.2) — Functions that take an argument of type `error_code&` assigns the specified enumerator to the provided `error_code` object and then returns.
- ⁴ Failure to allocate storage is reported by throwing an exception as described in [res.on.exception.handling] in N4618.
- ⁵ Destructor operations defined in this Technical Specification shall not throw exceptions. Every destructor in this Technical Specification shall behave as if it had a non-throwing exception specification.
- ⁶ If no error occurs in a function that takes an argument of type `error_code&`, `error_code::clear` shall be called on the `error_code` object immediately before the function returns.

6 Header <experimental/io2d> synopsis [io2d.syn]

```

namespace std { namespace experimental {
    namespace io2d { inline namespace v1 {

        using bounding_box = basic_bounding_box<default_graphics_math>;
        using brush = basic_brush<default_graphics_surfaces>;
        using brush_props = basic_brush_props<default_graphics_surfaces>;
        using circle = basic_circle<default_graphics_math>;
        using clip_props = basic_clip_props<default_graphics_surfaces>;
        using dashes = basic_dashes<default_graphics_surfaces>;
        using display_point = basic_display_point<default_graphics_math>;
        using figure_items = basic_figure_items<default_graphics_surfaces>;
        using image_surface = basic_image_surface<default_graphics_surfaces>;
        using interpreted_path = basic_interpreted_path<default_graphics_surfaces>;
        using mask_props = basic_mask_props<default_graphics_surfaces>;
        using matrix_2d = basic_matrix_2d<default_graphics_math>;
        using output_surface = basic_output_surface<default_graphics_surfaces>;
        using path_builder = basic_path_builder<default_graphics_surfaces>;
        using point_2d = basic_point_2d<default_graphics_math>;
        using render_props = basic_render_props<default_graphics_surfaces>;
        using stroke_props = basic_stroke_props<default_graphics_surfaces>;
        using unmanaged_output_surface = basic_unmanaged_output_surface<default_graphics_surfaces>;
    } } }

```


7 Colors

[io2d.colors]

7.1 Introduction to color

[io2d.colors.intro]

- ¹ Color involves many disciplines and has been the subject of many papers, treatises, experiments, studies, and research work in general.
- ² While color is an important part of computer graphics, it is only necessary to understand a few concepts from the study of color for computer graphics.
- ³ A color model defines color mathematically without regard to how humans actually perceive color. These color models are composed of some combination of channels which each channel representing alpha or an ideal color. Color models are useful for working with color computationally, such as in composing operations, because their channel values are homogeneously spaced.
- ⁴ A color space, for purposes of computer graphics, is the result of mapping the ideal color channels from a color model, after making any necessary adjustment for alpha, to color channels that are calibrated to align with human perception of colors. Since the perception of color varies from person to person, color spaces use the science of colorimetry to define those perceived colors in order to obtain uniformity to the extent possible. As such, the uniform display of the colors in a color space on different output devices is possible. The values of color channels in a color space are not necessarily homogeneously spaced because of human perception of color.
- ⁵ Color models are often termed *linear* while color spaces are often termed *gamma corrected*. The mapping of a color model, such as the RGB color model, to a color space, such as the sRGB color space, is often the application of gamma correction.
- ⁶ Gamma correction is the process of transforming homogeneously spaced visual data to visual data that, when displayed, matches the intent of the untransformed visual data.
- ⁷ For example a color that is 50% of the maximum intensity of red when encoded as homogeneously spaced visual data, will likely have a different intensity value when it has been gamma corrected so that a human looking at on a computer display will see it as being 50% of the maximum intensity of red that the computer display is capable of producing. Without gamma correction, it would likely have appeared as though it was closer to the maximum intensity than the untransformed data intended it to be.
- ⁸ In addition to color channels, colors in computer graphics often have an alpha channel. The value of the alpha channel represents transparency of the color channels when they are combined with other visual data using certain composing algorithms. When using alpha, it should be used in a premultiplied format in order to obtain the desired results when applying multiple composing algorithms that utilize alpha.

7.2 Color usage requirements

[io2d.colors.reqs]

- ¹ During rendering and composing operations, color data is linear and, when it has an alpha channel associated with it, in premultiplied format. Implementations shall make any necessary conversions to ensure this.

7.3 Class `rgba_color`

[io2d.rgbacolor]

7.3.1 `rgba_color` overview

[io2d.rgbacolor.intro]

- ¹ The class `rgba_color` describes a four channel color in premultiplied format.
- ² There are three color channels, red, green, and blue, each of which is a `float`.
- ³ There is also an alpha channel, which is a `float`.
- ⁴ Legal values for each channel are in the range `[0.0f, 1.0f]`.

7.3.2 `rgba_color` synopsis

[io2d.rgbacolor.synopsis]

```

namespace std::experimental::io2d::v1 {
    class rgba_color {
    public:
        // 7.3.3, construct/copy/move/destroy:
        constexpr rgba_color() noexcept;
        template <class T>
        constexpr rgba_color(T r, T g, T b, T a = static_cast<T>(0xFF)) noexcept;
        template <class U>
        constexpr rgba_color(U r, U g, U b, U a = static_cast<U>(1.0f)) noexcept;

        // 7.3.4, modifiers:
        template <class T>
        constexpr void r(T val) noexcept;
        template <class U>
        constexpr void r(U val) noexcept;
        template <class T>
        constexpr void g(T val) noexcept;
        template <class U>
        constexpr void g(U val) noexcept;
        template <class T>
        constexpr void b(T val) noexcept;
        template <class U>
        constexpr void b(U val) noexcept;
        template <class T>
        constexpr void a(T val) noexcept;
        template <class U>
        constexpr void a(U val) noexcept;

        // 7.3.5, observers:
        constexpr float r() const noexcept;
        constexpr float g() const noexcept;
        constexpr float b() const noexcept;
        constexpr float a() const noexcept;

        // 7.3.6, static members:
        static const rgba_color alice_blue;
        static const rgba_color antique_white;
        static const rgba_color aqua;
        static const rgba_color aquamarine;
        static const rgba_color azure;
        static const rgba_color beige;
        static const rgba_color bisque;
        static const rgba_color black;
        static const rgba_color blanched_almond;
        static const rgba_color blue;
        static const rgba_color blue_violet;
        static const rgba_color brown;
        static const rgba_color burly_wood;
        static const rgba_color cadet_blue;
        static const rgba_color chartreuse;
        static const rgba_color chocolate;
        static const rgba_color coral;
        static const rgba_color cornflower_blue;
        static const rgba_color cornsilk;
    };
}

```

```
static const rgba_color crimson;
static const rgba_color cyan;
static const rgba_color dark_blue;
static const rgba_color dark_cyan;
static const rgba_color dark_goldenrod;
static const rgba_color dark_gray;
static const rgba_color dark_green;
static const rgba_color dark_grey;
static const rgba_color dark_khaki;
static const rgba_color dark_magenta;
static const rgba_color dark_olive_green;
static const rgba_color dark_orange;
static const rgba_color dark_orchid;
static const rgba_color dark_red;
static const rgba_color dark_salmon;
static const rgba_color dark_sea_green;
static const rgba_color dark_slate_blue;
static const rgba_color dark_slate_gray;
static const rgba_color dark_slate_grey;
static const rgba_color dark_turquoise;
static const rgba_color dark_violet;
static const rgba_color deep_pink;
static const rgba_color deep_sky_blue;
static const rgba_color dim_gray;
static const rgba_color dim_grey;
static const rgba_color dodger_blue;
static const rgba_color firebrick;
static const rgba_color floral_white;
static const rgba_color forest_green;
static const rgba_color fuchsia;
static const rgba_color gainsboro;
static const rgba_color ghost_white;
static const rgba_color gold;
static const rgba_color goldenrod;
static const rgba_color gray;
static const rgba_color green;
static const rgba_color green_yellow;
static const rgba_color grey;
static const rgba_color honeydew;
static const rgba_color hot_pink;
static const rgba_color indian_red;
static const rgba_color indigo;
static const rgba_color ivory;
static const rgba_color khaki;
static const rgba_color lavender;
static const rgba_color lavender_blush;
static const rgba_color lawn_green;
static const rgba_color lemon_chiffon;
static const rgba_color light_blue;
static const rgba_color light_coral;
static const rgba_color light_cyan;
static const rgba_color light_goldenrod_yellow;
static const rgba_color light_gray;
static const rgba_color light_green;
static const rgba_color light_grey;
```

```
static const rgba_color light_pink;
static const rgba_color light_salmon;
static const rgba_color light_sea_green;
static const rgba_color light_sky_blue;
static const rgba_color light_slate_gray;
static const rgba_color light_slate_grey;
static const rgba_color light_steel_blue;
static const rgba_color light_yellow;
static const rgba_color lime;
static const rgba_color lime_green;
static const rgba_color linen;
static const rgba_color magenta;
static const rgba_color maroon;
static const rgba_color medium_aquamarine;
static const rgba_color medium_blue;
static const rgba_color medium_orchid;
static const rgba_color medium_purple;
static const rgba_color medium_sea_green;
static const rgba_color medium_slate_blue;
static const rgba_color medium_spring_green;
static const rgba_color medium_turquoise;
static const rgba_color medium_violet_red;
static const rgba_color midnight_blue;
static const rgba_color mint_cream;
static const rgba_color misty_rose;
static const rgba_color moccasin;
static const rgba_color navajo_white;
static const rgba_color navy;
static const rgba_color old_lace;
static const rgba_color olive;
static const rgba_color olive_drab;
static const rgba_color orange;
static const rgba_color orange_red;
static const rgba_color orchid;
static const rgba_color pale_goldenrod;
static const rgba_color pale_green;
static const rgba_color pale_turquoise;
static const rgba_color pale_violet_red;
static const rgba_color papaya_whip;
static const rgba_color peach_puff;
static const rgba_color peru;
static const rgba_color pink;
static const rgba_color plum;
static const rgba_color powder_blue;
static const rgba_color purple;
static const rgba_color red;
static const rgba_color rosy_brown;
static const rgba_color royal_blue;
static const rgba_color saddle_brown;
static const rgba_color salmon;
static const rgba_color sandy_brown;
static const rgba_color sea_green;
static const rgba_color sea_shell;
static const rgba_color sienna;
static const rgba_color silver;
```

```

static const rgba_color sky_blue;
static const rgba_color slate_blue;
static const rgba_color slate_gray;
static const rgba_color slate_grey;
static const rgba_color snow;
static const rgba_color spring_green;
static const rgba_color steel_blue;
static const rgba_color tan;
static const rgba_color teal;
static const rgba_color thistle;
static const rgba_color tomato;
static const rgba_color transparent_black;
static const rgba_color turquoise;
static const rgba_color violet;
static const rgba_color wheat;
static const rgba_color white;
static const rgba_color white_smoke;
static const rgba_color yellow;
static const rgba_color yellow_green;

// 7.3.7, operators
template <class T>
constexpr rgba_color& operator==(T rhs) noexcept;
template <class U>
constexpr rgba_color& operator==(U rhs) noexcept;
};

// 7.3.7, operators:
constexpr bool operator==(const rgba_color& lhs, const rgba_color& rhs)
    noexcept;
constexpr bool operator!=(const rgba_color& lhs, const rgba_color& rhs)
    noexcept;
template <class T>
constexpr rgba_color operator*(const rgba_color& lhs, T rhs) noexcept;
template <class U>
constexpr rgba_color operator*(const rgba_color& lhs, U rhs) noexcept;
template <class T>
constexpr rgba_color operator*(T lhs, const rgba_color& rhs) noexcept;
template <class U>
constexpr rgba_color operator*(U lhs, const rgba_color& rhs) noexcept;
}

```

7.3.3 rgba_color constructors and assignment operators [io2d.rgbacolor.cons]

```
constexpr rgba_color() noexcept;
```

¹ *Effects:* Equivalent to: `rgba_color{ 0.0f, 0.0f, 0.0f, 0.0f }`.

```
template <class T>
constexpr rgba_color(T r, T g, T b, T a = static_cast<T>(255)) noexcept;
```

² *Requires:* `r >= 0` and `r <= 255` and `g >= 0` and `g <= 255` and `b >= 0` and `b <= 255` and `a >= 0` and `a <= 255`.

³ *Effects:* Constructs an object of type `rgba_color`. The alpha channel is `a / 255.0F`. The red channel is `r / 255.0F * a / 255.0F`. The green channel is `g / 255.0F * a / 255.0F`. The blue channel is `b / 255.0F * a / 255.0F`.

4 *Remarks:* This constructor shall not participate in overload resolution unless `is_integral_v<T>` is `true`.

```
template <class U>
constexpr rgba_color(U r, U g, U b, U a = static_cast<U>(1.0f)) noexcept;
```

5 *Requires:* `r >= 0.0f` and `r <= 1.0f` and `g >= 0.0f` and `g <= 1.0f` and `b >= 0.0f` and `b <= 1.0f` and `a >= 0.0f` and `a <= 1.0f`.

6 *Effects:* Constructs an object of type `rgba_color`. The alpha channel is `a`. The red channel is `r * a`. The green channel is `g * a`. The blue channel is `b * a`.

7 *Remarks:* This constructor shall not participate in overload resolution unless `is_floating_point_v<U>` is `true`.

7.3.4 `rgba_color` modifiers

[io2d.rgbacolor.modifiers]

```
template <class T>
constexpr void r(T val) noexcept;
```

1 *Requires:* `val >= 0` and `val <= 255`.

2 *Effects:* The red channel is `val / 255.0F * a()`.

3 *Remarks:* This function shall not participate in overload resolution unless `is_integral_v<T>` is `true`.

```
template <class U>
constexpr void r(U val) noexcept;
```

4 *Requires:* `val >= 0.0f` and `val <= 1.0f`.

5 *Effects:* The red channel is `val * a()`.

6 *Remarks:* This function shall not participate in overload resolution unless `is_floating_point_v<U>` is `true`.

```
template <class T>
constexpr void g(T val) noexcept;
```

7 *Requires:* `val >= 0` and `val <= 255`.

8 *Effects:* The green channel is `val / 255.0F * a()`.

9 *Remarks:* This function shall not participate in overload resolution unless `is_integral_v<T>` is `true`.

```
template <class U>
constexpr void g(U val) noexcept;
```

10 *Requires:* `val >= 0.0f` and `val <= 1.0f`.

11 *Effects:* The green channel is `val * a()`.

12 *Remarks:* This function shall not participate in overload resolution unless `is_floating_point_v<U>` is `true`.

```
template <class T>
constexpr void b(T val) noexcept;
```

13 *Requires:* `val >= 0` and `val <= 255`.

14 *Effects:* The blue channel is `val / 255.0F * a()`.

15 *Remarks:* This function shall not participate in overload resolution unless `is_integral_v<T>` is `true`.

```
template <class U>
```

```
constexpr void b(U val) noexcept;
16     Requires: val >= 0.0f and val <= 1.0f.
17     Effects: The blue channel is val * a().
18     Remarks: This function shall not participate in overload resolution unless is_floating_point_v<U> is
    true.

template <class T>
constexpr void a(T val) noexcept;
19     Requires: val >= 0 and val <= 255.
20     Effects: If a() == 0.0f the alpha channel is val / 255.0F, otherwise:
    1. The red channel is set to (r() / a()) * val / 255.0F;
    2. The green channel is set to (g() / a()) * val / 255.0F;
    3. The blue channel is set to (b() / a()) * val / 255.0F;
    4. The alpha channel is set to val / 255.0F.
21     Remarks: This function shall not participate in overload resolution unless is_integral_v<T> is true.

template <class U>
constexpr void a(U val) noexcept;
22     Requires: val >= 0.0f and val <= 1.0f.
23     Effects: If a() == 0.0f the alpha channel is val, otherwise:
    1. The red channel is set to (r() / a()) * val;
    2. The green channel is set to (g() / a()) * val;
    3. The blue channel is set to (b() / a()) * val;
    4. The alpha channel is val.
24     Remarks: This function shall not participate in overload resolution unless is_floating_point_v<U> is
    true.
```

7.3.5 rgba_color observers

[io2d.rgbacolor.observers]

```
constexpr float r() const noexcept;
1     Returns: The red channel.

constexpr float g() const noexcept;
2     Returns: The green channel.

constexpr float b() const noexcept;
3     Returns: The blue channel.

constexpr float a() const noexcept;
4     Returns: The alpha channel.
```

7.3.6 rgba_color static members

[io2d.rgbacolor.statics]

- 1 The alpha value of all of the predefined rgba_color static member object in Table 2 is 1.0F except for transparent_black, which has an alpha value of 0.0F.

Table 2 — rgba_color static members values

Member name	red	green	blue
alice_blue	240	248	255
antique_white	250	235	215
aqua	0	255	255
aquamarine	127	255	212
azure	240	255	255
beige	245	245	220
bisque	255	228	196
black	0	0	0
blanched_almond	255	235	205
blue	0	0	255
blue_violet	138	43	226
brown	165	42	42
burly_wood	222	184	135
cadet_blue	95	158	160
chartreuse	127	255	0
chocolate	210	105	30
coral	255	127	80
cornflower_blue	100	149	237
cornsilk	255	248	220
crimson	220	20	60
cyan	0	255	255
dark_blue	0	0	139
dark_cyan	0	139	139
dark_goldenrod	184	134	11
dark_gray	169	169	169
dark_green	0	100	0
dark_grey	169	169	169
dark_khaki	189	183	107
dark_magenta	139	0	139
dark_olive_green	85	107	47
dark_orange	255	140	0
dark_orchid	153	50	204
dark_red	139	0	0
dark_salmon	233	150	122
dark_sea_green	143	188	142
dark_slate_blue	72	61	139
dark_slate_gray	47	79	79
dark_slate_grey	47	79	79
dark_turquoise	0	206	209
dark_violet	148	0	211
deep_pink	255	20	147
deep_sky_blue	0	191	255
dim_gray	105	105	105
dim_grey	105	105	105
dodger_blue	30	144	255
firebrick	178	34	34
floral_white	255	250	240

Table 2 — `rgba_color` static members values (continued)

Member name	red	green	blue
<code>forest_green</code>	34	139	34
<code>fuchsia</code>	255	0	255
<code>gainsboro</code>	220	220	220
<code>ghost_white</code>	248	248	248
<code>gold</code>	255	215	0
<code>goldenrod</code>	218	165	32
<code>gray</code>	128	128	128
<code>green</code>	0	128	0
<code>green_yellow</code>	173	255	47
<code>grey</code>	128	128	128
<code>honeydew</code>	240	255	240
<code>hot_pink</code>	255	105	180
<code>indian_red</code>	205	92	92
<code>indigo</code>	75	0	130
<code>ivory</code>	255	255	240
<code>khaki</code>	240	230	140
<code>lavender</code>	230	230	250
<code>lavender_blush</code>	255	240	245
<code>lawn_green</code>	124	252	0
<code>lemon_chiffon</code>	255	250	205
<code>light_blue</code>	173	216	230
<code>light_coral</code>	240	128	128
<code>light_cyan</code>	224	255	255
<code>light_goldenrod_yellow</code>	250	250	210
<code>light_gray</code>	211	211	211
<code>light_green</code>	144	238	144
<code>light_grey</code>	211	211	211
<code>light_pink</code>	255	182	193
<code>light_salmon</code>	255	160	122
<code>light_sea_green</code>	32	178	170
<code>light_sky_blue</code>	135	206	250
<code>light_slate_gray</code>	119	136	153
<code>light_slate_grey</code>	119	136	153
<code>light_steel_blue</code>	176	196	222
<code>light_yellow</code>	255	255	224
<code>lime</code>	0	255	0
<code>lime_green</code>	50	205	50
<code>linen</code>	250	240	230
<code>magenta</code>	255	0	255
<code>maroon</code>	128	0	0
<code>medium_aquamarine</code>	102	205	170
<code>medium_blue</code>	0	0	205
<code>medium_orchid</code>	186	85	211
<code>medium_purple</code>	147	112	219
<code>medium_sea_green</code>	60	179	113
<code>medium_slate_blue</code>	123	104	238
<code>medium_spring_green</code>	0	250	154

Table 2 — `rgba_color` static members values (continued)

Member name	red	green	blue
<code>medium_turquoise</code>	72	209	204
<code>medium_violet_red</code>	199	21	133
<code>midnight_blue</code>	25	25	112
<code>mint_cream</code>	245	255	250
<code>misty_rose</code>	255	228	225
<code>moccasin</code>	255	228	181
<code>navajo_white</code>	255	222	173
<code>navy</code>	0	0	128
<code>old_lace</code>	253	245	230
<code>olive</code>	128	128	0
<code>olive_drab</code>	107	142	35
<code>orange</code>	255	69	0
<code>orange_red</code>	255	69	0
<code>orchid</code>	218	112	214
<code>pale_goldenrod</code>	238	232	170
<code>pale_green</code>	152	251	152
<code>pale_turquoise</code>	175	238	238
<code>pale_violet_red</code>	219	112	147
<code>papaya_whip</code>	255	239	213
<code>peach_puff</code>	255	218	185
<code>peru</code>	205	133	63
<code>pink</code>	255	192	203
<code>plum</code>	221	160	221
<code>powder_blue</code>	176	224	230
<code>purple</code>	128	0	128
<code>red</code>	255	0	0
<code>rosy_brown</code>	188	143	143
<code>royal_blue</code>	65	105	225
<code>saddle_brown</code>	139	69	19
<code>salmon</code>	250	128	114
<code>sandy_brown</code>	244	164	96
<code>sea_green</code>	46	139	87
<code>sea_shell</code>	255	245	238
<code>sienna</code>	160	82	45
<code>silver</code>	192	192	192
<code>sky_blue</code>	135	206	235
<code>slate_blue</code>	106	90	205
<code>slate_gray</code>	112	128	144
<code>slate_grey</code>	112	128	144
<code>snow</code>	255	250	250
<code>spring_green</code>	0	255	127
<code>steel_blue</code>	70	130	180
<code>tan</code>	210	180	140
<code>teal</code>	0	128	128
<code>thistle</code>	216	191	216
<code>tomato</code>	255	99	71
<code>transparent_black</code>	0	0	0

Table 2 — `rgba_color` static members values (continued)

Member name	red	green	blue
turquoise	64	244	208
violet	238	130	238
wheat	245	222	179
white	255	255	255
white_smoke	245	245	245
yellow	255	255	0
yellow_green	154	205	50

7.3.7 `rgba_color` operators

[io2d.rgbacolor.ops]

```

template <class T>
constexpr rgba_color& operator*=(T rhs) noexcept;
1      Requires: rhs >= 0 and rhs <= 255.
2      Effects: r(min(r() * rhs / 255.0F, 1.0F)).
3      g(min(g() * rhs / 255.0F, 1.0F)).
4      b(min(b() * rhs / 255.0F, 1.0F)).
5      a(min(a() * rhs / 255.0F, 1.0F)).
      Returns: *this.
6      Remarks: This function shall not participate in overload resolution unless is_integral_v<T> is true.

template <class U>
constexpr rgba_color& operator*=(U rhs) noexcept;
7      Requires: rhs >= 0.0F and rhs <= 1.0F.
8      Effects: r(min(r() * rhs, 1.0F)).
9      g(min(g() * rhs, 1.0F)).
10     b(min(b() * rhs, 1.0F)).
11     a(min(a() * rhs, 1.0F)).
      Returns: *this.
12     Remarks: This function shall not participate in overload resolution unless is_floating_point_v<T> is
      true.

constexpr bool operator==(const rgba_color& lhs, const rgba_color& rhs)
    noexcept;
13     Returns: lhs.r() == rhs.r() && lhs.g() == rhs.g() && lhs.b() == rhs.b() && lhs.a() ==
      rhs.a().

template <class T>
constexpr rgba_color operator*(const rgba_color& lhs, T rhs) noexcept;
14     Requires: rhs >= 0 and rhs <= 255.
15     Returns:
      rgba_color(min(lhs.r() * rhs / 255.0F, 1.0F), min(lhs.g() * rhs / 255.0F, 1.0F),
        min(lhs.b() * rhs / 255.0F, 1.0F), min(lhs.a() * rhs / 255.0F, 1.0F))
16     Remarks: This function shall not participate in overload resolution unless is_integral_v<T> is true.

```

```

template <class U>
constexpr rgba_color& operator*(const rgba_color& lhs, U rhs) noexcept;
17     Requires: rhs >= 0.0F and rhs <= 1.0F.
18     Returns:
        rgba_color(min(lhs.r() * rhs, 1.0F), min(lhs.g() * rhs, 1.0F),
            min(lhs.b() * rhs, 1.0F), min(lhs.a() * rhs, 1.0F))
19     Remarks: This function shall not participate in overload resolution unless is_floating_point_v<U> is
        true.

template <class T>
constexpr rgba_color operator*(T lhs, const rgba_color& rhs) noexcept;
20     Requires: lhs >= 0 and lhs <= 255.
21     Returns:
        rgba_color(min(lhs * rhs.r() / 255.0F, 1.0F), min(lhs * rhs.g() / 255.0F, 1.0F),
            min(lhs * rhs.b() / 255.0F, 1.0F), min(lhs * rhs.a() / 255.0F, 1.0F))
22     Remarks: This function shall not participate in overload resolution unless is_integral_v<T> is true.

template <class U>
constexpr rgba_color& operator*(U lhs, const rgba_color& rhs) noexcept;
23     Requires: lhs >= 0.0F and lhs <= 1.0F.
24     Returns:
        rgba_color(min(lhs * rhs.r(), 1.0F), min(lhs * rhs.g(), 1.0F),
            min(lhs * rhs.b(), 1.0F), min(lhs * rhs.a(), 1.0F))
25     Remarks: This function shall not participate in overload resolution unless is_floating_point_v<U> is
        true.

```

8 Linear algebra

[io2d.linearalgebra]

8.1 Class `basic_point_2d`

[io2d.point2d]

8.1.1 `basic_point_2d` description

[io2d.point2d.intro]

- ¹ The class template `basic_point_2d` is used as both a point and as a two-dimensional Euclidean vector.
- ² It has an *x coordinate* of type `float` and a *y coordinate* of type `float`.

8.1.2 `basic_point_2d` synopsis

[io2d.point2d.synopsis]

```
namespace std::experimental::io2d::v1 {
    template <class GraphicsMath>
    class basic_point_2d {
    public:
        // 8.1.3, constructors:
        basic_point_2d() noexcept;
        basic_point_2d(float x, float y) noexcept;
        basic_point_2d(const typename GraphicsMath::point_2d_data_type& data) noexcept;

        // 8.1.4, modifiers:
        void x(float val) noexcept;
        void y(float val) noexcept;

        // 8.1.5, observers:
        float x() const noexcept;
        float y() const noexcept;
        float dot(const basic_point_2d& other) const noexcept;
        float magnitude() const noexcept;
        float magnitude_squared() const noexcept;
        float angular_direction() const noexcept;
        basic_point_2d to_unit() const noexcept;

        // 8.1.6, member operators:
        basic_point_2d& operator+=(const basic_point_2d& rhs) noexcept;
        basic_point_2d& operator+=(float rhs) noexcept;
        basic_point_2d& operator-=(const basic_point_2d& rhs) noexcept;
        basic_point_2d& operator-=(float rhs) noexcept;
        basic_point_2d& operator*=(const basic_point_2d& rhs) noexcept;
        basic_point_2d& operator*=(float rhs) noexcept;
        basic_point_2d& operator/=(const basic_point_2d& rhs) noexcept;
        basic_point_2d& operator/=(float rhs) noexcept;
    };

    // 8.1.7, non-member operators:
    template <class GraphicsMath>
    bool operator==(const basic_point_2d<GraphicsMath>& lhs,
        const basic_point_2d<GraphicsMath>& rhs) noexcept;
    template <class GraphicsMath>
    bool operator!=(const basic_point_2d<GraphicsMath>& lhs,
        const basic_point_2d<GraphicsMath>& rhs) noexcept;
    template <class GraphicsMath>
```

```

basic_point_2d<GraphicsMath> operator+(const basic_point_2d<GraphicsMath>& val) noexcept;
template <class GraphicsMath>
basic_point_2d<GraphicsMath> operator+(const basic_point_2d<GraphicsMath>& lhs,
    const basic_point_2d<GraphicsMath>& rhs) noexcept;
template <class GraphicsMath>
basic_point_2d<GraphicsMath> operator-(const basic_point_2d<GraphicsMath>& val) noexcept;
template <class GraphicsMath>
basic_point_2d<GraphicsMath> operator-(const basic_point_2d<GraphicsMath>& lhs,
    const basic_point_2d<GraphicsMath>& rhs) noexcept;
template <class GraphicsMath>
basic_point_2d<GraphicsMath> operator*(const basic_point_2d<GraphicsMath>& lhs,
    float rhs) noexcept;
template <class GraphicsMath>
basic_point_2d<GraphicsMath> operator*(float lhs,
    const basic_point_2d<GraphicsMath>& rhs) noexcept;
template <class GraphicsMath>
basic_point_2d<GraphicsMath> operator*(const basic_point_2d<GraphicsMath>& lhs,
    const basic_point_2d<GraphicsMath>& rhs) noexcept;
template <class GraphicsMath>
basic_point_2d<GraphicsMath> operator/(const basic_point_2d<GraphicsMath>& lhs,
    float rhs) noexcept;
template <class GraphicsMath>
basic_point_2d<GraphicsMath> operator/(float lhs,
    const basic_point_2d<GraphicsMath>& rhs) noexcept;
template <class GraphicsMath>
basic_point_2d<GraphicsMath> operator/(const basic_point_2d<GraphicsMath>& lhs,
    const basic_point_2d<GraphicsMath>& rhs) noexcept;
}

```

8.1.3 basic_point_2d constructors

[io2d.point2d.cons]

```
basic_point_2d() noexcept;
```

1 *Effects:* Equivalent to `basic_point_2d{ 0.0f, 0.0f }`.

```
basic_point_2d(float x, float y) noexcept;
```

2 *Effects:* Constructs an object of type `basic_point_2d`.

3 The x coordinate is x.

4 The y coordinate is y.

```
basic_point_2d(const typename GraphicsMath::point_2d_data_type& data) noexcept;
```

5 *Effects:* Constructs an object of type `basic_point_2d`.

6 <TODO>The coordinates are data.

8.1.4 basic_point_2d modifiers

[io2d.point2d.modifiers]

```
void x(float val) noexcept;
```

1 *Effects:* <TODO>

```
void y(float val) noexcept;
```

2 *Effects:* <TODO>

8.1.5 basic_point_2d observers

[io2d.point2d.observers]

```

float x() const noexcept;
1   Returns: <TODO>

float y() const noexcept;
2   Returns: <TODO>

float dot(const basic_point_2d& other) const noexcept;
3   Returns: <TODO>x * other.x + y * other.y.

float magnitude() const noexcept;
4   Returns: Equivalent to: sqrt(dot(*this));

float magnitude_squared() const noexcept;
5   Returns: Equivalent to: dot(*this);

float angular_direction() const noexcept
6   Returns: <TODO>atan2(y, x) if it is greater than or equal to 0.0f.
7   <TODO>Otherwise, atan2(y, x) + two_pi<float>.
8   [ Note: The purpose of adding two_pi<float> if the result is negative is to produce values in the range
    [0.0f, two_pi<float>). — end note ]

basic_point_2d to_unit() const noexcept;
9   Returns: <TODO>basic_point_2d{ x / magnitude(), y / magnitude()}.

```

8.1.6 basic_point_2d member operators [io2d.point2d.member.ops]

```

basic_point_2d& operator+=(const basic_point_2d& rhs) noexcept;
1   Effects: *this = *this + rhs.
2   Returns: *this.

basic_point_2d& operator-=(const basic_point_2d& rhs) noexcept;
3   Effects: Equivalent to: *this = *this - rhs;.
4   Returns: *this.

basic_point_2d& operator*=(float rhs) noexcept;
basic_point_2d& operator*=(const basic_point_2d& rhs) noexcept;
5   Effects: Equivalent to: *this = *this * rhs;.
6   Returns: *this.

basic_point_2d& operator/=(float rhs) noexcept;
basic_point_2d& operator/=(const basic_point_2d& rhs) noexcept;
7   Effects: Equivalent to: *this = *this / rhs;.
8   Returns: *this.

```

8.1.7 basic_point_2d non-member operators [io2d.point2d.ops]

```

bool operator==(const basic_point_2d& lhs, const basic_point_2d& rhs) noexcept;
1   Returns: <TODO>lhs.x == rhs.x && lhs.y == rhs.y.

```

```

bool operator!=(const basic_point_2d& lhs, const basic_point_2d& rhs) noexcept;
2   Returns: !(lhs == rhs).

basic_point_2d operator+(const basic_point_2d& val) noexcept;
3   Returns: val.

basic_point_2d operator+(const basic_point_2d& lhs, const basic_point_2d& rhs) noexcept;
4   Returns: <TODO>basic_point_2d{ lhs.x + rhs.x, lhs.y + rhs.y }.

basic_point_2d operator-(const basic_point_2d& val) noexcept;
5   Returns: <TODO>basic_point_2d{ -val.x, -val.y }.

basic_point_2d operator-(const basic_point_2d& lhs, const basic_point_2d& rhs) noexcept;
6   Returns: <TODO>basic_point_2d{ lhs.x - rhs.x, lhs.y - rhs.y }.

basic_point_2d operator*(const basic_point_2d& lhs, const basic_point_2d& rhs) noexcept;
7   Returns: <TODO>basic_point_2d{ lhs.x * rhs.x, lhs.y * rhs.y }.

basic_point_2d operator*(const basic_point_2d& lhs, float rhs) noexcept;
8   Returns: <TODO>basic_point_2d{ lhs.x * rhs, lhs.y * rhs }.

basic_point_2d operator*(float lhs, const basic_point_2d& rhs) noexcept;
9   Returns: <TODO>basic_point_2d{ lhs * rhs.x, lhs * rhs.y }.

basic_point_2d operator/(const basic_point_2d& lhs, const basic_point_2d& rhs) noexcept;
10  Requires: <TODO>rhs.x is not 0.0f and rhs.y is not 0.0f.
11  Returns: <TODO>basic_point_2d{ lhs.x / rhs.x, lhs.y / rhs.y }.

basic_point_2d operator/(const basic_point_2d& lhs, float rhs) noexcept;
12  Requires: rhs is not 0.0f.
13  Returns: <TODO>basic_point_2d{ lhs.x / rhs, lhs.y / rhs }.

basic_point_2d operator/(float lhs, const basic_point_2d& rhs) noexcept;
14  Requires: <TODO>rhs.x is not 0.0f and rhs.y is not 0.0f.
15  Returns: <TODO>basic_point_2d{ lhs / rhs.x, lhs / rhs.y }.

```

8.2 Class `basic_matrix_2d`

[io2d.matrix2d]

8.2.1 `basic_matrix_2d` description

[io2d.matrix2d.intro]

- 1 The class template `basic_matrix_2d` represents a three row by three column matrix. Its purpose is to perform affine transformations.
- 2 The matrix is composed of nine `float` values: `m00`, `m01`, `m02`, `m10`, `m11`, `m12`, `m20`, `m21`, and `m22`. The ordering of these `float` values in the `basic_matrix_2d` class is unspecified.
- 3 The specification of the `basic_matrix_2d` class, as described in this subclause, uses the following ordering:


```

[ [ m00 m01 m02 ] ]
[ [ m10 m11 m12 ] ]
[ [ m20 m21 m22 ] ]

```
- 4 [Note: The naming convention and the layout shown above are consistent with a row-major layout. Though the naming convention is fixed, the unspecified layout allows for a column-major layout (or any other layout,

though row-major and column-major are the only layouts typically used). — *end note*]

- ⁵ The performance of any mathematical operation upon a `basic_matrix_2d` shall be carried out as-if the omitted third column data members were present with the values prescribed in the previous paragraph.

8.2.2 `basic_matrix_2d` synopsis

[io2d.matrix2d.synopsis]

```
namespace std::experimental::io2d::v1 {
    template <class GraphicsMath>
    class basic_matrix_2d {
    public:
        // 8.2.3, constructors:
        basic_matrix_2d() noexcept;
        basic_matrix_2d(float v00, float v01, float v10, float v11, float v20, float v21) noexcept;
        basic_matrix_2d(const typename GraphicsMath::matrix_2d_data_type& val) noexcept;

        // 8.2.4, static factory functions:
        static basic_matrix_2d init_translate(const basic_point_2d<GraphicsMath>& val) noexcept;
        static basic_matrix_2d init_scale(const basic_point_2d<GraphicsMath>& val) noexcept;
        static basic_matrix_2d init_rotate(float radians) noexcept;
        static basic_matrix_2d init_rotate(float radians,
            const basic_point_2d<GraphicsMath>& origin) noexcept;
        static basic_matrix_2d init_reflect(float radians) noexcept;
        static basic_matrix_2d init_shear_x(float factor) noexcept;
        static basic_matrix_2d init_shear_y(float factor) noexcept;

        // 8.2.5, modifiers:
        void m00(float val) noexcept;
        void m01(float val) noexcept;
        void m10(float val) noexcept;
        void m11(float val) noexcept;
        void m20(float val) noexcept;
        void m21(float val) noexcept;
        basic_matrix_2d& translate(const basic_point_2d<GraphicsMath>& v) noexcept;
        basic_matrix_2d& scale(const basic_point_2d<GraphicsMath>& v) noexcept;
        basic_matrix_2d& rotate(float radians) noexcept;
        basic_matrix_2d& rotate(float radians, const basic_point_2d<GraphicsMath>& origin) noexcept;
        basic_matrix_2d& reflect(float radians) noexcept;
        basic_matrix_2d& shear_x(float factor) noexcept;
        basic_matrix_2d& shear_y(float factor) noexcept;

        // 8.2.6, observers:
        float m00() const noexcept;
        float m01() const noexcept;
        float m10() const noexcept;
        float m11() const noexcept;
        float m20() const noexcept;
        float m21() const noexcept;
        bool is_finite() const noexcept;
        bool is_invertible() const noexcept;
        float determinant() const noexcept;
        basic_matrix_2d inverse() const noexcept;
        basic_point_2d<GraphicsMath> transform_pt(const basic_point_2d<GraphicsMath>& pt)
            const noexcept;

        // 8.2.7, member operators:
```

```

    basic_matrix_2d& operator*=(const basic_matrix_2d& other) noexcept;
};

// 8.2.8, member operators:
template <class GraphicsMath>
basic_matrix_2d<GraphicsMath> operator*(const basic_matrix_2d<GraphicsMath>& lhs,
    const basic_matrix_2d<GraphicsMath>& rhs) noexcept;
template <class GraphicsMath>
basic_point_2d<GraphicsMath> operator*(const basic_point_2d<GraphicsMath>& lhs,
    const basic_matrix_2d<GraphicsMath>& rhs) noexcept;
template <class GraphicsMath>
bool operator==(const basic_matrix_2d<GraphicsMath>& lhs,
    const basic_matrix_2d<GraphicsMath>& rhs) noexcept;
template <class GraphicsMath>
bool operator!=(const basic_matrix_2d<GraphicsMath>& lhs,
    const basic_matrix_2d<GraphicsMath>& rhs) noexcept;
}

```

8.2.3 basic_matrix_2d constructors

[io2d.matrix2d.cons]

```
basic_matrix_2d() noexcept;
```

1 *Effects:* Equivalent to: `basic_matrix_2d{ 1.0f, 0.0f, 0.0f, 1.0f, 0.0f, 0.0f }`.

2 [*Note:* The resulting matrix is the identity matrix. — *end note*]

```
basic_matrix_2d(float v00, float v01, float v10, float v11,
    float v20, float v21) noexcept;
```

3 *Effects:* Constructs an object of type `basic_matrix_2d`.

4 `<TODO>m00 == v00, m01 == v01, m02 == 0.0f, m10 == v10, m11 == v11, m12 == 0.0f, m20 == v20, m21 == v21, m22 == 1.0f.`

```
basic_matrix_2d(const typename GraphicsMath::matrix_2d_data_type& val) noexcept;
```

5 *Effects:* Constructs an object of type `basic_matrix_2d`.

6 `<TODO>`

8.2.4 basic_matrix_2d static factory functions

[io2d.matrix2d.staticfactories]

```
static basic_matrix_2d init_translate(basic_point_2d<GraphicsMath> value) noexcept;
```

1 *Returns:* `<TODO>basic_matrix_2d(1.0f, 0.0f, 0.0f, 1.0f, value.x, value.y).`

```
static basic_matrix_2d init_scale(basic_point_2d<GraphicsMath> value) noexcept;
```

2 *Returns:* `<TODO>basic_matrix_2d(value.x, 0.0f, 0.0f, value.y, 0.0f, 0.0f).`

```
static basic_matrix_2d init_rotate(float radians) noexcept;
```

3 *Returns:* `basic_matrix_2d(cos(radians), -sin(radians), sin(radians), cos(radians), 0.0f, 0.0f).`

```
static basic_matrix_2d init_rotate(float radians, basic_point_2d<GraphicsMath> origin) noexcept;
```

4 *Effects:* Equivalent to:

```
    return basic_matrix_2d(
        basic_matrix_2d::init_translate(origin).rotate(radians).translate(-origin));
```

```
static basic_matrix_2d init_reflect(float radians) noexcept;
```

5 *Returns:* `basic_matrix_2d(cos(radians * 2.0f), sin(radians * 2.0f), sin(radians * 2.0f),
-cos(radians * 2.0f), 0.0f, 0.0f)`

`static basic_matrix_2d init_shear_x(float factor) noexcept;`

6 *Returns:* `basic_matrix_2d(1.0f, 0.0f, factor, 1.0f, 0.0f, 0.0f).`

`static basic_matrix_2d init_shear_y(float factor) noexcept;`

7 *Returns:* `basic_matrix_2d(1.0f, factor, 0.0f, 1.0f, 0.0f, 0.0f)`

8.2.5 `basic_matrix_2d` modifiers [io2d.matrix2d.modifiers]

`basic_matrix_2d& translate(basic_point_2d<GraphicsMath> val) noexcept;`

1 *Effects:* Equivalent to: `*this = *this * init_translate(val);`

2 *Returns:* `*this.`

`basic_matrix_2d& scale(basic_point_2d<GraphicsMath> val) noexcept;`

3 *Effects:* Equivalent to: `*this = *this * init_scale(val);`

4 *Returns:* `*this.`

`basic_matrix_2d& rotate(float radians) noexcept;`

5 *Effects:* Equivalent to: `*this = *this * init_rotate(radians);`

6 *Returns:* `*this.`

`basic_matrix_2d& rotate(float radians, basic_point_2d<GraphicsMath> origin) noexcept;`

7 *Effects:* Equivalent to: `*this = *this * init_rotate(radians, origin);`

8 *Returns:* `*this.`

`basic_matrix_2d& reflect(float radians) noexcept;`

9 *Effects:* Equivalent to: `*this = *this * init_reflect(radians);`

10 *Returns:* `*this.`

`basic_matrix_2d& shear_x(float factor) noexcept;`

11 *Effects:* Equivalent to: `*this = *this * init_shear_x(factor);`

12 *Returns:* `*this.`

`basic_matrix_2d& shear_y(float factor) noexcept;`

13 *Effects:* Equivalent to: `*this = *this * init_shear_y(factor);`

14 *Returns:* `*this.`

8.2.6 `basic_matrix_2d` observers [io2d.matrix2d.observers]

`bool is_finite() const noexcept;`

1 *Returns:* `true` if the observable behavior of all of the following expressions evaluates to `true`:

(1.1) — `isfinite(m00);`

(1.2) — `isfinite(m01);`

(1.3) — `isfinite(m10);`

(1.4) — `isfinite(m11);`

(1.5) — `isfinite(m20);`

(1.6) — `isfinite(m21);`

2 Otherwise returns `false`.

3 [*Note:* The specification of `isfinite` in N4618 does not include the `constexpr` specifier. Regardless, the requirements stated in [library.c] and [c.math.fpclass] in N4618 make it possible to implement a `constexpr` function that produces the observable behavior of `isfinite`. As a result, this function can be implemented as a `constexpr` function. — *end note*]

```
bool is_invertible() const noexcept;
```

4 *Requires:* `is_finite()` is true.

5 *Returns:* `determinant() != 0.0f`.

```
basic_matrix_2d inverse() const noexcept;
```

6 *Requires:* `is_invertible()` is true.

7 *Returns:* Let `inverseDeterminant` be `1.0f / determinant()`.

<TODO>

```
return basic_matrix_2d{
    (m11 * 1.0f - 0.0f * m21) * inverseDeterminant,
    -(m01 * 1.0f - 0.0f * m21) * inverseDeterminant,
    -(m10 * 1.0f - 0.0f * m20) * inverseDeterminant,
    (m00 * 1.0f - 0.0f * m20) * inverseDeterminant,
    (m10 * m21 - m11 * m20) * inverseDeterminant,
    -(m00 * m21 - m01 * m20) * inverseDeterminant
};
```

```
float determinant() const noexcept;
```

8 *Requires:* `is_finite()` is true.

9 *Returns:* <TODO>`m00 * m11 - m01 * m10`.

```
basic_point_2d<GraphicsMath> transform_pt(basic_point_2d<GraphicsMath> pt) const noexcept;
```

10 *Returns:* <TODO>`basic_point_2d<GraphicsMath>((m00 * pt.x + m10 * pt.y) + m20, (m01 * pt.x + m11 * pt.y) + m21)`.

8.2.7 `basic_matrix_2d` member operators [io2d.matrix2d.member.ops]

```
basic_matrix_2d& operator*=(const basic_matrix_2d& rhs) noexcept;
```

- 1 *Effects:* Equivalent to: `*this = *this * rhs;`
- 2 *Returns:* `*this`.

8.2.8 `basic_matrix_2d` non-member operators [io2d.matrix2d.ops]

```
basic_matrix_2d operator*(const basic_matrix_2d& lhs, const basic_matrix_2d& rhs)
noexcept;
```

- 1 *Returns:* <TODO>
- ```
basic_matrix_2d{
 lhs.m00 * rhs.m00 + lhs.m01 * rhs.m10,
 lhs.m00 * rhs.m01 + lhs.m01 * rhs.m11,
 lhs.m10 * rhs.m00 + lhs.m11 * rhs.m10,
 lhs.m10 * rhs.m01 + lhs.m11 * rhs.m11,
```

```

 lhs.m20 * rhs.m00 + lhs.m21 * rhs.m10 + lhs.m20,
 lhs.m20 * rhs.m01 + lhs.m21 * rhs.m11 + lhs.m21
 }

```

```

basic_point_2d<GraphicsMath> operator*(basic_point_2d<GraphicsMath> v, const basic_matrix_2d& m) noexcept;

```

2     *Returns:* Equivalent to: `m.transform_pt(v)`.

```

bool operator==(const basic_matrix_2d& lhs, const basic_matrix_2d& rhs) noexcept;

```

3     *Returns:* <TODO>

```

 lhs.m00 == rhs.m00 && lhs.m01 == rhs.m01 &&
 lhs.m10 == rhs.m10 && lhs.m11 == rhs.m11 &&
 lhs.m20 == rhs.m20 && lhs.m21 == rhs.m21

```

```

bool operator!=(const basic_matrix_2d& lhs, const basic_matrix_2d& rhs) noexcept;

```

4     *Returns:* Equivalent to: `!(lhs == rhs)`.

## 9 Geometry

[io2d.geometry]

### 9.1 Class basic\_bounding\_box

[io2d.bounding\_box]

#### 9.1.1 basic\_bounding\_box description

[io2d.bounding\_box.intro]

- <sup>1</sup> The class template `basic_bounding_box` describes a `bounding_box`.
- <sup>2</sup> It has an *x coordinate* of type `float`, a *y coordinate* of type `float`, a *width* of type `float`, and a *height* of type `float`.

#### 9.1.2 basic\_bounding\_box synopsis

[io2d.bounding\_box.synopsis]

```
namespace std::experimental::io2d::v1 {
 template <class GraphicsMath>
 class basic_bounding_box {
 public:
 // 9.1.3, constructors:
 basic_bounding_box() noexcept;
 basic_bounding_box(float x, float y, float width, float height) noexcept;
 basic_bounding_box(const basic_point_2d<GraphicsMath>& tl,
 const basic_point_2d<GraphicsMath>& br) noexcept;
 basic_bounding_box(const typename GraphicsMath::bounding_box_data_type& val) noexcept;

 // 9.1.4, modifiers:
 void x(float val) noexcept;
 void y(float val) noexcept;
 void width(float val) noexcept;
 void height(float val) noexcept;
 void top_left(const basic_point_2d<GraphicsMath>& val) noexcept;
 void bottom_right(const basic_point_2d<GraphicsMath>& val) noexcept;

 // 9.1.5, observers:
 float x() const noexcept;
 float y() const noexcept;
 float width() const noexcept;
 float height() const noexcept;
 basic_point_2d<GraphicsMath> top_left() const noexcept;
 basic_point_2d<GraphicsMath> bottom_right() const noexcept;
 };

 // 9.1.6, operators:
 template <class GraphicsMath>
 bool operator==(const basic_bounding_box<GraphicsMath>& lhs,
 const basic_bounding_box<GraphicsMath>& rhs) noexcept;
 template <class GraphicsMath>
 bool operator!=(const basic_bounding_box<GraphicsMath>& lhs,
 const basic_bounding_box<GraphicsMath>& rhs) noexcept;
}
```

#### 9.1.3 basic\_bounding\_box constructors

[io2d.bounding\_box.cons]

`basic_bounding_box()` `noexcept`;

- <sup>1</sup> *Effects:* Equivalent to `basic_bounding_box{ 0.0f, 0.0f, 0.0f, 0.0f }`.

```
basic_bounding_box(float x, float y, float w, float h) noexcept;
```

2     *Requires:*  $w$  is not less than 0.0f and  $h$  is not less than 0.0f.

3     *Effects:* Constructs an object of type `basic_bounding_box`.

4     The  $x$  coordinate is  $x$ . The  $y$  coordinate is  $y$ . The width is  $w$ . The height is  $h$ .

```
basic_bounding_box(const basic_point_2d<GraphicsMath>& tl,
 const basic_point_2d<GraphicsMath>& br) noexcept;
```

5     *Effects:* Constructs an object of type `basic_bounding_box`.

6     <TODO>The  $x$  coordinate is  $tl.x$ . The  $y$  coordinate is  $tl.y$ . The width is  $\max(0.0f, br.x - tl.x)$ .  
The height is  $\max(0.0f, br.y - tl.y)$ .

#### 9.1.4 `basic_bounding_box` modifiers

[io2d.bounding\_box.modifiers]

```
void x(float val) noexcept;
```

1     *Effects:* The  $x$  coordinate is  $val$ .

```
void y(float val) noexcept;
```

2     *Effects:* The  $y$  coordinate is  $val$ .

```
void width(float val) noexcept;
```

3     *Effects:* The width is  $val$ .

```
void height(float val) noexcept;
```

4     *Effects:* The height is  $val$ .

```
void top_left(const basic_point_2d<GraphicsMath>& val) noexcept;
```

5     *Effects:* <TODO>The  $x$  coordinate is  $val.x$ .

6     <TODO>The  $y$  coordinate is  $val.y$ .

```
void bottom_right(const basic_point_2d<GraphicsMath>& val) noexcept;
```

7     *Effects:* <TODO>The width is  $\max(0.0f, val.x - x())$ .

8     <TODO>The height is  $\max(0.0f, value.y - y())$ .

#### 9.1.5 `basic_bounding_box` observers

[io2d.bounding\_box.observers]

```
float x() const noexcept;
```

1     *Returns:* The  $x$  coordinate.

```
float y() const noexcept;
```

2     *Returns:* The  $y$  coordinate.

```
float width() const noexcept;
```

3     *Returns:* The width.

```
float height() const noexcept;
```

4     *Returns:* The height.

```
basic_point_2d<GraphicsMath> top_left() const noexcept;
```

- 5 *Returns:* A `basic_point_2d<GraphicsMath>` object constructed with the x coordinate as its first argument and the y coordinate as its second argument.

```
basic_point_2d<GraphicsMath> bottom_right() const noexcept;
```

- 6 *Returns:* A `basic_point_2d<GraphicsMath>` object constructed with the width added to the x coordinate as its first argument and the height added to the y coordinate as its second argument.

### 9.1.6 `basic_bounding_box` operators [io2d.bounding\_box.ops]

```
bool operator==(const basic_bounding_box<GraphicsMath>& lhs,
 const basic_bounding_box<GraphicsMath>& rhs) noexcept;
```

- 1 *Returns:* <TODO>
- ```
    lhs.x() == rhs.x() && lhs.y() == rhs.y() &&
    lhs.width() == rhs.width() && lhs.height() == rhs.height()
```

9.2 Class `basic_circle` [io2d.circle]

9.2.1 `basic_circle` description [io2d.circle.intro]

- 1 The class template `basic_circle` describes a circle.
- 2 It has a *center* of type `basic_point_2d<GraphicsMath>` and a *radius* of type `float`.

9.2.2 `basic_circle` synopsis [io2d.circle.synopsis]

```
namespace std::experimental::io2d::v1 {
    template <class GraphicsMath>
    class basic_circle {
    public:
        // 9.2.3, constructors:
        basic_circle() noexcept;
        basic_circle(const basic_point_2d<GraphicsMath>& ctr, float rad) noexcept;
        basic_circle(const typename GraphicsMath::circle_data_type& val) noexcept;

        // 9.2.4, modifiers:
        void center(const basic_point_2d<GraphicsMath>& ctr) noexcept;
        void radius(float r) noexcept;

        // 9.2.5, observers:
        basic_point_2d<GraphicsMath> center() const noexcept;
        float radius() const noexcept;
    };

    // 9.2.6, operators:
    template <class GraphicsMath>
    bool operator==(const basic_circle<GraphicsMath>& lhs,
                   const basic_circle<GraphicsMath>& rhs) noexcept;
    template <class GraphicsMath>
    bool operator!=(const basic_circle<GraphicsMath>& lhs,
                   const basic_circle<GraphicsMath>& rhs) noexcept;
}
```

9.2.3 `basic_circle` constructors [io2d.circle.cons]

```
basic_circle() noexcept;
```

- 1 *Effects:* Equivalent to: `basic_circle({ 0.0f, 0.0f }, 0.0f).`


```
basic_circle(const basic_point_2d<GraphicsMath>& ctr, float r) noexcept;
```

Requires: $r \geq 0.0f$.

2 *Effects:* Constructs an object of type `basic_circle`.

3 The center is `ctr`. The radius is `r`.

9.2.4 `basic_circle` modifiers

[io2d.circle.modifiers]

```
void center(const basic_point_2d<GraphicsMath>& ctr) noexcept;
```

1 *Effects:* The center is `ctr`.

```
void radius(float r) noexcept;
```

Requires: $r \geq 0.0f$.

2 *Effects:* The radius is `r`.

9.2.5 `basic_circle` observers

[io2d.circle.observers]

```
basic_point_2d<GraphicsMath> center() const noexcept;
```

1 *Returns:* The center.

```
float radius() const noexcept;
```

2 *Returns:* The radius.

9.2.6 `basic_circle` operators

[io2d.circle.ops]

```
bool operator==(const basic_circle<GraphicsMath>& lhs,
                const basic_circle<GraphicsMath>& rhs) noexcept;
```

1 *Returns:* `lhs.center() == rhs.center() && lhs.radius() == rhs.radius();`

10 Text rendering and display [io2d.text]

- ¹ [*Note:* Text rendering and matters related to it, such as font support, will be added at a later date. This section is a placeholder. The integration of text rendering is expected to result in the addition of member functions to the surface class and changes to other parts of the text. — *end note*]

11 Paths

[io2d.paths]

11.1 Overview of paths

[io2d.paths.overview]

- ¹ Paths define geometric objects which can be stroked (Table 19), filled, masked, and used to define a clip area (See: 13.12.1).
- ² A path contains zero or more figures.
- ³ A figure is composed of at least one segment.
- ⁴ A figure may contain degenerate segments. When a path is interpreted (11.3.16), degenerate segments are removed from figures. [*Note*: If a path command exists or is inserted between segments, it's possible that points which might have compared equal will no longer compare equal as a result of interpretation (11.3.16). — *end note*]
- ⁵ Paths provide vector graphics functionality. As such they are particularly useful in situations where an application is intended to run on a variety of platforms whose output devices (??) span a large gamut of sizes, both in terms of measurement units and in terms of a horizontal and vertical pixel count, in that order.
- ⁶ A `basic_interpreted_path` object is an immutable resource wrapper containing a path (11.4). A `basic_interpreted_path` object is created by interpreting the path contained in a `basic_path_builder` object. It can also be default constructed, in which case the `basic_interpreted_path` object contains no figures. [*Note*: `basic_interpreted_path` objects provide significant optimization opportunities for implementations. Because they are immutable and opaque, they are intended to be used to store a path in the most efficient representation available. — *end note*]

11.2 Path examples (Informative)

[io2d.paths.example]

11.2.1 Overview

[io2d.paths.example.intro]

- ¹ Paths are composed of zero or more figures. The following examples show the basics of how paths work in practice.
- ² Every example is placed within the following code at the indicated spot. This code is shown here once to avoid repetition:

```
#include <experimental/io2d>

using namespace std;
using namespace std::experimental::io2d;

int main() {
    auto imgSfc = make_image_surface(format::argb32, 300, 200);
    brush backBrush{ rgba_color::black };
    brush foreBrush{ rgba_color::white };
    render_props aliased{ antialias::none };
    path_builder pb{};
    imgSfc.paint(backBrush);

    // Example code goes here.

    // Example code ends.

    imgSfc.save(filesystem::path("example.png"), image_file_format::png);
}
```

```
    return 0;
}
```

11.2.2 Example 1

[io2d.paths.examples.one]

- ¹ Example 1 consists of a single figure, forming a trapezoid:

```
pb.new_figure({ 80.0f, 20.0f }); // Begins the figure.
pb.line({ 220.0f, 20.0f }); // Creates a line from the [80, 20] to [220, 20].
pb.rel_line({ 60.0f, 160.0f }); // Line from [220, 20] to
// [220 + 60, 160 + 20]. The "to" point is relative to the starting point.
pb.rel_line({ -260.0f, 0.0f }); // Line from [280, 180] to
// [280 - 260, 180 + 0].
pb.close_figure(); // Creates a line from [20, 180] to [80, 20]
// (the new figure point), which makes this a closed figure.
imgSfc.stroke(foreBrush, pb, nullopt, nullopt, nullopt, aliased);
```

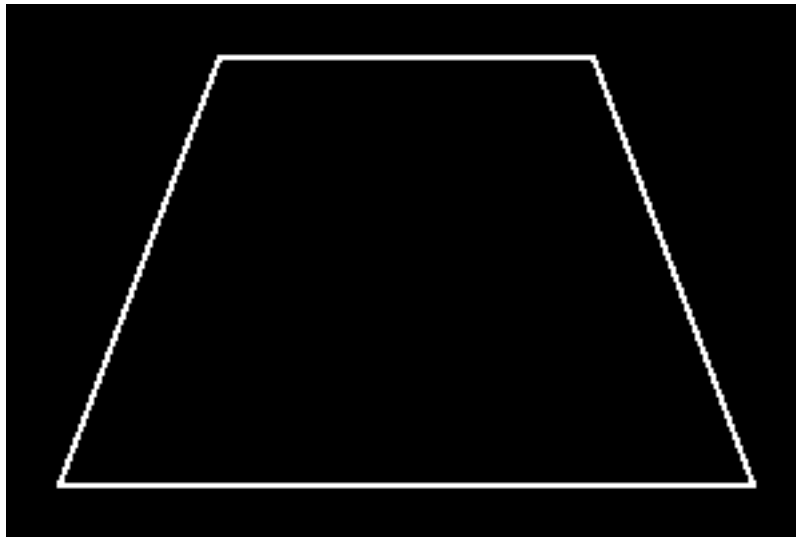


Figure 1 — Example 1 result

11.2.3 Example 2

[io2d.paths.examples.two]

- ¹ Example 2 consists of two figures. The first is a rectangular open figure (on the left) and the second is a rectangular closed figure (on the right):

```
pb.new_figure({ 20.0f, 20.0f }); // Begin the first figure.
pb.rel_line({ 100.0f, 0.0f });
pb.rel_line({ 0.0f, 160.0f });
pb.rel_line({ -100.0f, 0.0f });
pb.rel_line({ 0.0f, -160.0f });

pb.new_figure({ 180.0f, 20.0f }); // End the first figure and begin the
// second figure.
pb.rel_line({ 100.0f, 0.0f });
pb.rel_line({ 0.0f, 160.0f });
pb.rel_line({ -100.0f, 0.0f });
pb.close_figure(); // End the second figure.
imgSfc.stroke(foreBrush, pb, nullopt, stroke_props{ 10.0f }, nullopt, aliased);
```

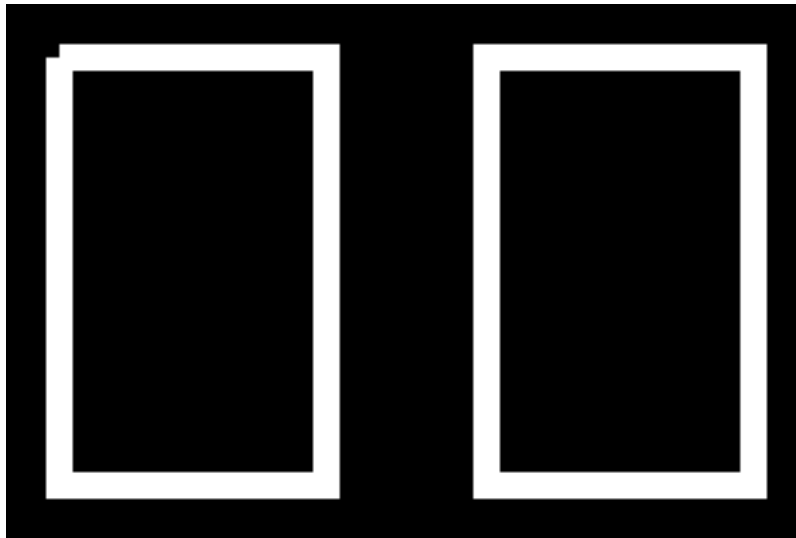


Figure 2 — Example 2 result

- ² The resulting image from example 2 shows the difference between an open figure and a closed figure. Each figure begins and ends at the same point. The difference is that with the closed figure, that the rendering of the point where the initial segment and final segment meet is controlled by the `line_join` value in the `stroke_props` class, which in this case is the default value of `line_join::miter`. In the open figure, the rendering of that point receives no special treatment such that each segment at that point is rendered using the `line_cap` value in the `stroke_props` class, which in this case is the default value of `line_cap::none`.
- ³ That difference between rendering as a `line_join` versus rendering as two `line_caps` is what causes the notch to appear in the open segment. Segments are rendered such that half of the stroke width is rendered on each side of the point being evaluated. With no line cap, each segment begins and ends exactly at the point specified.
- ⁴ So for the open figure, the first line begins at `point_2d{ 20.0f, 20.0f }` and the last line ends there. Given the stroke width of `10.0f`, the visible result for the first line is a rectangle with an upper left corner of `point_2d{ 20.0f, 15.0f }` and a lower right corner of `point_2d{ 120.0f, 25.0f }`. The last line appears as a rectangle with an upper left corner of `point_2d{ 15.0f, 20.0f }` and a lower right corner of `point_2d{ 25.0f, 180.0f }`. This produces the appearance of a square gap between `point_2d{ 15.0f, 15.0f }` and `point_2d{20.0f, 20.0f }`.
- ⁵ For the closed figure, adjusting for the coordinate differences, the rendering facts are the same as for the open figure except for one key difference: the point where the first line and last line meet is rendered as a line join rather than two line caps, which, given the default value of `line_join::miter`, produces a miter, adding that square area to the rendering result.

11.2.4 Example 3

[io2d.paths.examples.three]

- ¹ Example 3 demonstrates open and closed figures each containing either a quadratic curve or a cubic curve.

```
pb.new_figure({ 20.0f, 20.0f });
pb.rel_quadratic_curve({ 60.0f, 120.0f }, { 60.0f, -120.0f });
pb.rel_new_figure({ 20.0f, 0.0f });
pb.rel_quadratic_curve({ 60.0f, 120.0f }, { 60.0f, -120.0f });
pb.close_figure();
pb.new_figure({ 20.0f, 150.0f });
```

```

pb.rel_cubic_curve({ 40.0f, -120.0f }, { 40.0f, 120.0f * 2.0f },
  { 40.0f, -120.0f });
pb.rel_new_figure({ 20.0f, 0.0f });
pb.rel_cubic_curve({ 40.0f, -120.0f }, { 40.0f, 120.0f * 2.0f },
  { 40.0f, -120.0f });
pb.close_figure();
imgSfc.stroke(foreBrush, pb, nullopt, nullopt, nullopt, aliased);

```

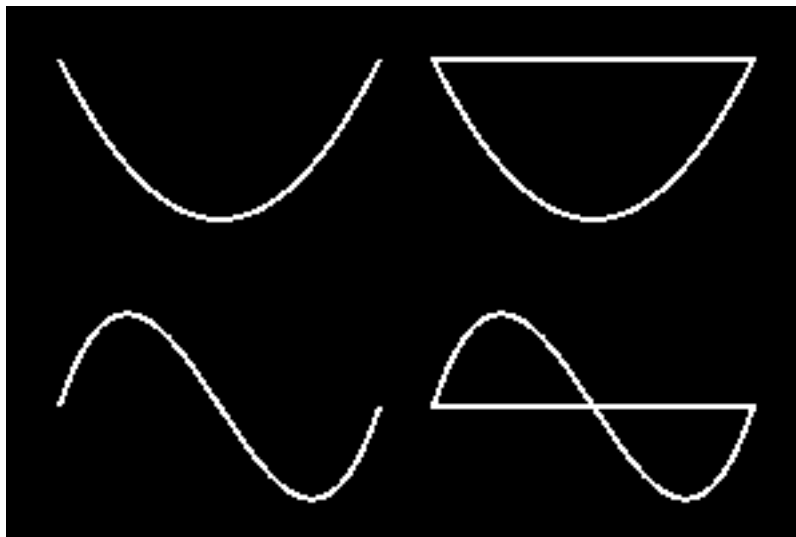


Figure 3 — Path example 3

- ² [Note: `pb.quadratic_curve({ 80.0f, 140.0f }, { 140.0f, 20.0f })`; would be the absolute equivalent of the first curve in example 3. — end note]

11.2.5 Example 4

[io2d.paths.examples.four]

- ¹ Example 4 shows how to draw "C++" using figures.
- ² For the "C", it is created using an arc. A scaling matrix is used to make it slightly elliptical. It is also desirable that the arc has a fixed center point, `point_2d{ 85.0f, 100.0f }`. The inverse of the scaling matrix is used in combination with the `point_for_angle` function to determine the point at which the arc should begin in order to get achieve this fixed center point. The "C" is then stroked.
- ³ Unlike the "C", which is created using an open figure that is stroked, each "+" is created using a closed figure that is filled. To avoid filling the "C", `pb.clear()`; is called to empty the container. The first "+" is created using a series of lines and is then filled.
- ⁴ Taking advantage of the fact that `path_builder` is a container, rather than create a brand new figure for the second "+", a translation matrix is applied by inserting a `figure_items::change_matrix` figure item before the `figure_items::new_figure` object in the existing plus, reverting back to the old matrix immediately after the and then filling it again.

```

// Create the "C".
const matrix_2d scl = matrix_2d::init_scale({ 0.9f, 1.1f });
auto pt = scl.inverse().transform_pt({ 85.0f, 100.0f }) +
  point_for_angle(half_pi<float> / 2.0f, 50.0f);
pb.matrix(scl);
pb.new_figure(pt);

```

```

pb.arc({ 50.0f, 50.0f }, three_pi_over_two<float>, half_pi<float> / 2.0f);
imgSfc.stroke(foreBrush, pb, nullopt, stroke_props{ 10.0f });
// Create the first "+".
pb.clear();
pb.new_figure({ 130.0f, 105.0f });
pb.rel_line({ 0.0f, -10.0f });
pb.rel_line({ 25.0f, 0.0f });
pb.rel_line({ 0.0f, -25.0f });
pb.rel_line({ 10.0f, 0.0f });
pb.rel_line({ 0.0f, 25.0f });
pb.rel_line({ 25.0f, 0.0f });
pb.rel_line({ 0.0f, 10.0f });
pb.rel_line({ -25.0f, 0.0f });
pb.rel_line({ 0.0f, 25.0f });
pb.rel_line({ -10.0f, 0.0f });
pb.rel_line({ 0.0f, -25.0f });
pb.close_figure();
imgSfc.fill(foreBrush, pb);
// Create the second "+".
pb.insert(pb.begin(), figure_items::change_matrix(
    matrix_2d::init_translate({ 80.0f, 0.0f })));
imgSfc.fill(foreBrush, pb);

```

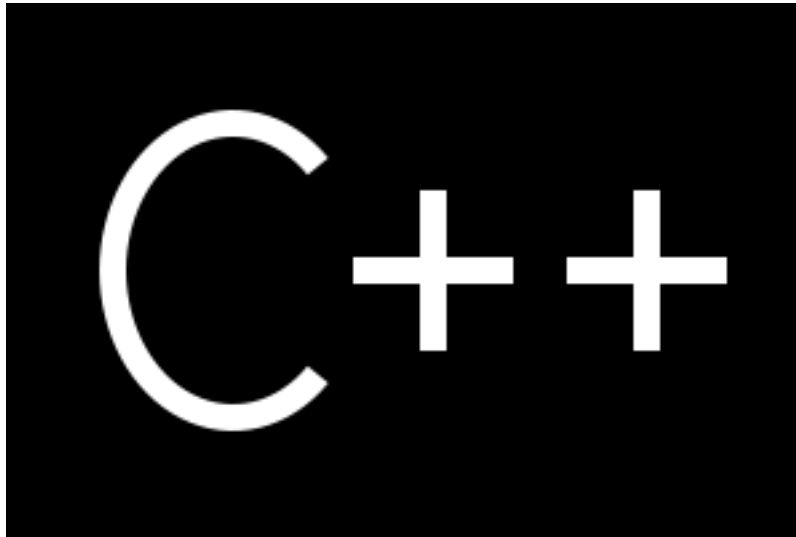


Figure 4 — Path example 4

11.3 Figure items

[io2d.paths.items]

11.3.1 Introduction

[io2d.paths.items.intro]

- ¹ The member classes of the class template `basic_figure_items` describe figure items.
- ² A figure begins with an `abs_new_figure` or `rel_new_figure` object. A figure ends when:
 - (2.1) — a `close_figure` object is encountered;
 - (2.2) — a `abs_new_figure` or `rel_new_figure` object is encountered; or
 - (2.3) — there are no more figure items in the path.

- ³ The `basic_path_builder` class is a sequential container that contains a path. It provides a simple interface for building a path but a path can be created using any container that stores `basic_figure_items::figure_item` objects.

11.3.2 `basic_figure_items` synopsis

[io2d.paths.items.synopsis]

```
namespace std::experimental::io2d::v1 {
    template <class GraphicsSurfaces>
    class basic_figure_items {
    public:
        class abs_new_figure;
        class rel_new_figure;
        class close_figure;
        class abs_matrix;
        class rel_matrix;
        class revert_matrix;
        class abs_cubic_curve;
        class abs_line;
        class abs_quadratic_curve;
        class arc;
        class rel_cubic_curve;
        class rel_line;
        class rel_quadratic_curve;

        class abs_new_figure {
        public:
            // 11.3.3.1, construct:
            abs_new_figure();
            explicit abs_new_figure(const basic_point_2d<typename
                GraphicsSurfaces::graphics_math_type>& pt);
            abs_new_figure(const abs_new_figure& other);
            abs_new_figure(abs_new_figure&& other) noexcept;

            // 11.3.3.2, assign:
            abs_new_figure& operator=(const abs_new_figure& other);
            abs_new_figure& operator=(abs_new_figure&& other) noexcept;

            // 11.3.3.3, modifiers:
            void at(const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& pt) noexcept;

            // 11.3.3.4, observers:
            basic_point_2d<typename GraphicsSurfaces::graphics_math_type> at() const noexcept;
        };

        class rel_new_figure {
        public:
            // 11.3.4.1, construct:
            rel_new_figure() noexcept;
            explicit rel_new_figure(const basic_point_2d<typename
                GraphicsSurfaces::graphics_math_type>& pt) noexcept;
            rel_new_figure(const rel_new_figure& other);
            rel_new_figure(rel_new_figure&& other) noexcept;

            // 11.3.4.2, assign:
            rel_new_figure& operator=(const rel_new_figure& other);
            rel_new_figure& operator=(rel_new_figure&& other) noexcept;
        };
    };
};
```



```

// 11.3.4.3, modifiers:
void at(const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& pt) noexcept;

// 11.3.4.4, observers:
basic_point_2d<typename GraphicsSurfaces::graphics_math_type> at() const noexcept;
};

class close_figure {
public:
// 11.3.5.1, construct:
constexpr close_figure() noexcept;
};

class abs_matrix {
public:
// 11.3.6.1, construct:
abs_matrix() noexcept;
explicit abs_matrix(const matrix_2d& m);
abs_matrix(abs_matrix&& other) noexcept;

// 11.3.6.2, assign:
abs_matrix& operator=(const abs_matrix& other);
abs_matrix& operator=(abs_matrix&& other) noexcept;

// 11.3.6.3, modifiers:
void matrix(const basic_matrix_2d<typename
    GraphicsSurfaces::graphics_math_type>& m) noexcept;

// 11.3.6.4, observers:
basic_matrix_2d<typename GraphicsSurfaces::graphics_math_type> matrix() const noexcept;
};

class rel_matrix {
public:
// 11.3.7.1, construct:
rel_matrix() noexcept;
explicit rel_matrix(const basic_matrix_2d<typename
    GraphicsSurfaces::graphics_math_type>& m) noexcept;
rel_matrix(const rel_matrix& other);
rel_matrix(rel_matrix&& other) noexcept;

// 11.3.7.2, assign:
rel_matrix& operator=(const rel_matrix& other);
rel_matrix& operator=(rel_matrix&& other) noexcept;

// 11.3.7.3, modifiers:
void matrix(const basic_matrix_2d<typename
    GraphicsSurfaces::graphics_math_type>& m) noexcept;

// 11.3.7.4, observers:
basic_matrix_2d<typename GraphicsSurfaces::graphics_math_type> matrix() const noexcept;
};

class revert_matrix {

```

```

public:
    // 11.3.8.1, construct:
    revert_matrix() noexcept;
};

class abs_line {
public:
    // 11.3.9.1, construct:
    abs_line() noexcept;
    explicit abs_line(const basic_point_2d<typename
        GraphicsSurfaces::graphics_math_type>& pt) noexcept;
    abs_line(const abs_line& other);
    abs_line(abs_line&& other) noexcept;

    // 11.3.9.2, assign:
    abs_line& operator=(const abs_line& other);
    abs_line& operator=(abs_line&& other) noexcept;

    // 11.3.9.3, modifiers:
    void to(const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& pt) noexcept;

    // 11.3.9.4, observers:
    basic_point_2d<typename GraphicsSurfaces::graphics_math_type> to() const noexcept;
};

class rel_line {
public:
    // 11.3.10.1, construct:
    rel_line() noexcept;
    explicit rel_line(const basic_point_2d<typename
        GraphicsSurfaces::graphics_math_type>& pt) noexcept;
    rel_line(const rel_line& other);
    rel_line(rel_line&& other) noexcept;

    // 11.3.10.2, assign:
    rel_line& operator=(rel_line&& other) noexcept;
    rel_line& operator=(const rel_line& other);

    // 11.3.10.3, modifiers:
    void to(const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& pt) noexcept;

    // 11.3.10.4, observers:
    basic_point_2d<typename GraphicsSurfaces::graphics_math_type> to() const noexcept;
};

class abs_quadratic_curve {
public:
    // 11.3.11.1, construct:
    abs_quadratic_curve() noexcept;
    abs_quadratic_curve(const basic_point_2d<typename
        GraphicsSurfaces::graphics_math_type>& cpt, const basic_point_2d<typename
        GraphicsSurfaces::graphics_math_type>& ept) noexcept;
    abs_quadratic_curve(const abs_quadratic_curve& other);
    abs_quadratic_curve(abs_quadratic_curve&& other) noexcept;

```

```

// 11.3.11.2, assign:
abs_quadratic_curve& operator=(abs_quadratic_curve&& other) noexcept;
abs_quadratic_curve& operator=(const abs_quadratic_curve& other);

// 11.3.11.3, modifiers:
void control_pt(const basic_point_2d<typename
    GraphicsSurfaces::graphics_math_type>& cpt) noexcept;
void end_pt(const basic_point_2d<typename
    GraphicsSurfaces::graphics_math_type>& ept) noexcept;

// 11.3.11.4, observers:
basic_point_2d<typename GraphicsSurfaces::graphics_math_type> control_pt() const noexcept;
basic_point_2d<typename GraphicsSurfaces::graphics_math_type> end_pt() const noexcept;
};

class rel_quadratic_curve {
public:
    // 11.3.12.1, construct:
    rel_quadratic_curve() noexcept;
    rel_quadratic_curve(const basic_point_2d<typename
        GraphicsSurfaces::graphics_math_type>& cpt, const basic_point_2d<typename
        GraphicsSurfaces::graphics_math_type>& ept) noexcept;
    rel_quadratic_curve(const rel_quadratic_curve& other);
    rel_quadratic_curve(rel_quadratic_curve&& other) noexcept;

    // 11.3.12.2, assign:
    rel_quadratic_curve& operator=(rel_quadratic_curve&& other) noexcept;
    rel_quadratic_curve& operator=(const rel_quadratic_curve& other);

    // 11.3.12.3, modifiers:
    void control_pt(const basic_point_2d<typename
        GraphicsSurfaces::graphics_math_type>& cpt) noexcept;
    void end_pt(const basic_point_2d<typename
        GraphicsSurfaces::graphics_math_type>& ept) noexcept;

    // 11.3.12.4, observers:
    basic_point_2d<typename GraphicsSurfaces::graphics_math_type> control_pt() const noexcept;
    basic_point_2d<typename GraphicsSurfaces::graphics_math_type> end_pt() const noexcept;
};

class abs_cubic_curve {
public:
    // 11.3.13.1, construct:
    abs_cubic_curve() noexcept;
    abs_cubic_curve(const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& cpt1,
        const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& cpt2,
        const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& ept) noexcept;
    abs_cubic_curve(const abs_cubic_curve& other);
    abs_cubic_curve(abs_cubic_curve&& other) noexcept;

    // 11.3.13.2, assign:
    abs_cubic_curve& operator=(const abs_cubic_curve& other);
    abs_cubic_curve& operator=(abs_cubic_curve&& other) noexcept;

    // 11.3.13.3, modifiers:

```

```

void control_pt1(const basic_point_2d<typename
    GraphicsSurfaces::graphics_math_type>& cpt) noexcept;
void control_pt2(const basic_point_2d<typename
    GraphicsSurfaces::graphics_math_type>& cpt) noexcept;
void end_pt(const basic_point_2d<typename
    GraphicsSurfaces::graphics_math_type>& ept) noexcept;

// 11.3.13.4, observers:
basic_point_2d<typename GraphicsSurfaces::graphics_math_type> control_pt1() const noexcept;
basic_point_2d<typename GraphicsSurfaces::graphics_math_type> control_pt2() const noexcept;
basic_point_2d<typename GraphicsSurfaces::graphics_math_type> end_pt() const noexcept;
};

class rel_cubic_curve {
public:
    // 11.3.14.1, construct:
    rel_cubic_curve() noexcept;
    rel_cubic_curve(const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& cpt1,
        const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& cpt2,
        const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& ept) noexcept;
    rel_cubic_curve(const rel_cubic_curve& other);
    rel_cubic_curve(rel_cubic_curve&& other) noexcept;

    // 11.3.14.2, assign:
    rel_cubic_curve& operator=(const rel_cubic_curve& other);
    rel_cubic_curve& operator=(rel_cubic_curve&& other) noexcept;

    // 11.3.14.3, modifiers:
    void control_pt1(const basic_point_2d<typename
        GraphicsSurfaces::graphics_math_type>& cpt) noexcept;
    void control_pt2(const basic_point_2d<typename
        GraphicsSurfaces::graphics_math_type>& cpt) noexcept;
    void end_pt(const basic_point_2d<typename
        GraphicsSurfaces::graphics_math_type>& ept) noexcept;

    // 11.3.14.4, observers:
    basic_point_2d<typename GraphicsSurfaces::graphics_math_type> control_pt1() const noexcept;
    basic_point_2d<typename GraphicsSurfaces::graphics_math_type> control_pt2() const noexcept;
    basic_point_2d<typename GraphicsSurfaces::graphics_math_type> end_pt() const noexcept;
};

class arc {
public:
    // 11.3.15.2, construct:
    arc() noexcept;
    arc(const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& rad, float rot, float sang) noexcept;
    arc(const arc& other);
    arc(arc&& other) noexcept;

    // 11.3.15.3, assign:
    arc& operator=(const arc& other);
    arc& operator=(arc&& other) noexcept;

    // 11.3.15.4, modifiers:
    void radius(const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& rad) noexcept;

```

```

void rotation(float rot) noexcept;
void start_angle(float sang) noexcept;

// 11.3.15.5, observers:
basic_point_2d<typename GraphicsSurfaces::graphics_math_type> radius() const noexcept;
float rotation() const noexcept;
float start_angle() const noexcept;
basic_point_2d<typename GraphicsSurfaces::graphics_math_type> center(const basic_point_2d<typename
    GraphicsSurfaces::graphics_math_type>& cpt, const basic_matrix_2d<typename
    GraphicsSurfaces::graphics_math_type>& m = basic_matrix_2d<typename
    GraphicsSurfaces::graphics_math_type>{ }) const noexcept;
basic_point_2d<typename GraphicsSurfaces::graphics_math_type> end_pt(const basic_point_2d<typename
    GraphicsSurfaces::graphics_math_type>& cpt, const basic_matrix_2d<typename
    GraphicsSurfaces::graphics_math_type>& m = basic_matrix_2d<typename
    GraphicsSurfaces::graphics_math_type>{ }) const noexcept;
};

using figure_item = variant<abs_cubic_curve, abs_line, abs_matrix, abs_new_figure,
    abs_quadratic_curve, arc, close_figure, rel_cubic_curve, rel_line, rel_matrix,
    rel_new_figure, rel_quadratic_curve, revert_matrix>;
};

// 11.3.3.5, abs_new_figure operators:
template <class GraphicsSurfaces>
bool operator==(const typename basic_figure_items<GraphicsSurfaces>::abs_new_figure& lhs,
    const typename basic_figure_items<GraphicsSurfaces>::abs_new_figure& rhs) noexcept;
template <class GraphicsSurfaces>
bool operator!=(const typename basic_figure_items<GraphicsSurfaces>::abs_new_figure& lhs,
    const typename basic_figure_items<GraphicsSurfaces>::abs_new_figure& rhs) noexcept;

// 11.3.4.5, rel_new_figure operators:
template <class GraphicsSurfaces>
bool operator==(const typename basic_figure_items<GraphicsSurfaces>::rel_new_figure& lhs,
    const typename basic_figure_items<GraphicsSurfaces>::rel_new_figure& rhs) noexcept;
template <class GraphicsSurfaces>
bool operator!=(const typename basic_figure_items<GraphicsSurfaces>::rel_new_figure& lhs,
    const typename basic_figure_items<GraphicsSurfaces>::rel_new_figure& rhs) noexcept;

// 11.3.5.2, close_figure operators:
template <class GraphicsSurfaces>
bool operator==(const typename basic_figure_items<GraphicsSurfaces>::close_figure& lhs,
    const typename basic_figure_items<GraphicsSurfaces>::close_figure& rhs) noexcept;
template <class GraphicsSurfaces>
bool operator!=(const typename basic_figure_items<GraphicsSurfaces>::close_figure& lhs,
    const typename basic_figure_items<GraphicsSurfaces>::close_figure& rhs) noexcept;

// 11.3.6.5, abs_matrix operators:
template <class GraphicsSurfaces>
bool operator==(const typename basic_figure_items<GraphicsSurfaces>::abs_matrix& lhs,
    const typename basic_figure_items<GraphicsSurfaces>::abs_matrix& rhs) noexcept;
template <class GraphicsSurfaces>
bool operator!=(const typename basic_figure_items<GraphicsSurfaces>::abs_matrix& lhs,
    const typename basic_figure_items<GraphicsSurfaces>::abs_matrix& rhs) noexcept;

// 11.3.7.5, rel_matrix operators:

```

```

template <class GraphicsSurfaces>
bool operator==(const typename basic_figure_items<GraphicsSurfaces>::rel_matrix& lhs,
    const typename basic_figure_items<GraphicsSurfaces>::rel_matrix& rhs) noexcept;
template <class GraphicsSurfaces>
bool operator!=(const typename basic_figure_items<GraphicsSurfaces>::rel_matrix& lhs,
    const typename basic_figure_items<GraphicsSurfaces>::rel_matrix& rhs) noexcept;

// 11.3.8.2, revert_matrix operators:
template <class GraphicsSurfaces>
bool operator==(const typename basic_figure_items<GraphicsSurfaces>::revert_matrix& lhs,
    const typename basic_figure_items<GraphicsSurfaces>::revert_matrix& rhs) noexcept;
template <class GraphicsSurfaces>
bool operator!=(const typename basic_figure_items<GraphicsSurfaces>::revert_matrix& lhs,
    const typename basic_figure_items<GraphicsSurfaces>::revert_matrix& rhs) noexcept;

// 11.3.9.5, abs_line operators:
template <class GraphicsSurfaces>
bool operator==(const typename basic_figure_items<GraphicsSurfaces>::abs_line& lhs,
    const typename basic_figure_items<GraphicsSurfaces>::abs_line& rhs) noexcept;
template <class GraphicsSurfaces>
bool operator!=(const typename basic_figure_items<GraphicsSurfaces>::abs_line& lhs,
    const typename basic_figure_items<GraphicsSurfaces>::abs_line& rhs) noexcept;

// 11.3.10.5, rel_line operators:
template <class GraphicsSurfaces>
bool operator==(const typename basic_figure_items<GraphicsSurfaces>::rel_line& lhs,
    const typename basic_figure_items<GraphicsSurfaces>::rel_line& rhs) noexcept;
template <class GraphicsSurfaces>
bool operator!=(const typename basic_figure_items<GraphicsSurfaces>::rel_line& lhs,
    const typename basic_figure_items<GraphicsSurfaces>::rel_line& rhs) noexcept;

// 11.3.11.5, abs_quadratic_curve operators:
template <class GraphicsSurfaces>
bool operator==(const typename basic_figure_items<GraphicsSurfaces>::abs_quadratic_curve& lhs,
    const typename basic_figure_items<GraphicsSurfaces>::abs_quadratic_curve& rhs) noexcept;
template <class GraphicsSurfaces>
bool operator!=(const typename basic_figure_items<GraphicsSurfaces>::abs_quadratic_curve& lhs,
    const typename basic_figure_items<GraphicsSurfaces>::abs_quadratic_curve& rhs) noexcept;

// 11.3.12.5, rel_quadratic_curve operators:
template <class GraphicsSurfaces>
bool operator==(const typename basic_figure_items<GraphicsSurfaces>::rel_quadratic_curve& lhs,
    const typename basic_figure_items<GraphicsSurfaces>::rel_quadratic_curve& rhs) noexcept;
template <class GraphicsSurfaces>
bool operator!=(const typename basic_figure_items<GraphicsSurfaces>::rel_quadratic_curve& lhs,
    const typename basic_figure_items<GraphicsSurfaces>::rel_quadratic_curve& rhs) noexcept;

// 11.3.13.5, abs_cubic_curve operators:
template <class GraphicsSurfaces>
bool operator==(const typename basic_figure_items<GraphicsSurfaces>::abs_cubic_curve& lhs,
    const typename basic_figure_items<GraphicsSurfaces>::abs_cubic_curve& rhs) noexcept;
template <class GraphicsSurfaces>
bool operator!=(const typename basic_figure_items<GraphicsSurfaces>::abs_cubic_curve& lhs,
    const typename basic_figure_items<GraphicsSurfaces>::abs_cubic_curve& rhs) noexcept;

```

```

// 11.3.14.5, rel_cubic_curve operators:
template <class GraphicsSurfaces>
bool operator==(const typename basic_figure_items<GraphicsSurfaces>::rel_cubic_curve& lhs,
    const typename basic_figure_items<GraphicsSurfaces>::rel_cubic_curve& rhs) noexcept;
template <class GraphicsSurfaces>
bool operator!=(const typename basic_figure_items<GraphicsSurfaces>::rel_cubic_curve& lhs,
    const typename basic_figure_items<GraphicsSurfaces>::rel_cubic_curve& rhs) noexcept;

// 11.3.15.6, arc operators:
template <class GraphicsSurfaces>
bool operator==(const typename basic_figure_items<GraphicsSurfaces>::arc& lhs,
    const typename basic_figure_items<GraphicsSurfaces>::arc& rhs) noexcept;
template <class GraphicsSurfaces>
bool operator!=(const typename basic_figure_items<GraphicsSurfaces>::arc& lhs,
    const typename basic_figure_items<GraphicsSurfaces>::arc& rhs) noexcept;
}

```

11.3.3 Class `abs_new_figure` [io2d.absnewfigure]

- 1 The class `abs_new_figure` describes a figure item that is a new figure command.
- 2 It has an *at point* of type `point_2d`.

11.3.3.1 `abs_new_figure` constructors [io2d.absnewfigure.cons]

```
abs_new_figure();
```

- 1 *Effects:* Equivalent to: `abs_new_figure{basic_point_2d<typename GraphicsSurfaces::graphics_math_type>()};`

```
explicit abs_new_figure(const basic_point_2d<typename
    GraphicsSurfaces::graphics_math_type>& pt);
```

- 2 *Effects:* Constructs an object of type `abs_new_figure`.
- 3 The *at point* is `pt`.

```
abs_new_figure(const abs_new_figure& other);
abs_new_figure(abs_new_figure&& other) noexcept;
```

- 4 *Effects:* Constructs an object of type `abs_new_figure`. In the second form, `other` is left in a valid state with an unspecified value.
- 5 The *at point* is `other.at()`.

11.3.3.2 `abs_new_figure` assignment operators [io2d.absnewfigure.assign]

```
abs_new_figure& operator=(const abs_new_figure& other);
```

- 1 *Effects:* If `*this` and `other` are not the same object, modifies `*this` such that `*this.at()` is `other.at()`
- 2 If `*this` and `other` are the same object, the member has no effect.
- 3 *Returns:* `*this`

```
abs_new_figure& operator=(abs_new_figure&& other) noexcept;
```

- 4 *Effects:* <TODO>
- 5 *Returns:* `*this`

11.3.3.3 `abs_new_figure` modifiers [io2d.absnewfigure.modifiers]

```
void at(const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& pt) noexcept;
```

1 *Effects:* The at point is pt.

11.3.3.4 abs_new_figure observers [io2d.absnewfigure.observers]

```
basic_point_2d<typename GraphicsSurfaces::graphics_math_type> at() const noexcept;
```

1 *Returns:* The at point.

11.3.3.5 abs_new_figure operators [io2d.absnewfigure.ops]

```
template <class GraphicsSurfaces>
bool operator==(const typename basic_figure_items<GraphicsSurfaces>::abs_new_figure& lhs,
    const typename basic_figure_items<GraphicsSurfaces>::abs_new_figure& rhs) noexcept;
```

1 *Returns:* lhs.at() == rhs.at().

11.3.4 Class rel_new_figure [io2d.relnewfigure]

1 The class rel_new_figure describes a figure item that is a new figure command.

2 It has an *at point* of type point_2d.

11.3.4.1 rel_new_figure constructors [io2d.relnewfigure.cons]

```
rel_new_figure() noexcept;
```

1 *Effects:* Equivalent to: rel_new_figure{basic_point_2d<typename GraphicsSurfaces::graphics_math_type>()};

```
explicit rel_new_figure(const basic_point_2d<typename
    GraphicsSurfaces::graphics_math_type>& pt) noexcept;
```

2 *Effects:* Constructs an object of type rel_new_figure.

3 The at point is pt.

```
rel_new_figure(const rel_new_figure& other);
rel_new_figure(rel_new_figure&& other) noexcept;
```

4 *Effects:* Constructs an object of type rel_new_figure. In the second form, other is left in a valid state with an unspecified value.

5 The at point is other.at().

11.3.4.2 rel_new_figure assignment operators [io2d.relnewfigure.assign]

```
rel_new_figure& operator=(const rel_new_figure& other);
```

1 *Effects:* If *this and other are not the same object, modifies *this such that *this.at() is other.at()

2 If *this and other are the same object, the member has no effect.

3 *Returns:* *this

```
rel_new_figure& operator=(rel_new_figure&& other) noexcept;
```

4 *Effects:* <TODO>

5 *Returns:* *this

11.3.4.3 rel_new_figure modifiers [io2d.relnewfigure.modifiers]

```
void at(const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& pt) noexcept;
```


1 *Effects:* The at point is pt.

11.3.4.4 rel_new_figure observers [io2d.relnewfigure.observers]

```
basic_point_2d<typename GraphicsSurfaces::graphics_math_type> at() const noexcept;
```

1 *Returns:* The at point.

11.3.4.5 rel_new_figure operators [io2d.relnewfigure.ops]

```
template <class GraphicsSurfaces>
bool operator==(const typename basic_figure_items<GraphicsSurfaces>::rel_new_figure& lhs,
    const typename basic_figure_items<GraphicsSurfaces>::rel_new_figure& rhs) noexcept;
```

1 *Returns:* lhs.at() == rhs.at().

11.3.5 Class close_figure [io2d.closefigure]

1 The class close_figure describes a figure item that is a close figure command.

11.3.5.1 close_figure constructors [io2d.closefigure.cons]

```
close_figure();
```

1 *Effects:* Constructs an object of type close_figure.

11.3.5.2 close_figure operators [io2d.closefigure.ops]

```
template <class GraphicsSurfaces>
bool operator==(const typename basic_figure_items<GraphicsSurfaces>::close_figure& lhs,
    const typename basic_figure_items<GraphicsSurfaces>::close_figure& rhs) noexcept;
```

1 *Returns:* true.

11.3.6 Class abs_matrix [io2d.absmatrix]

1 The class abs_matrix describes a figure item that is a path command.

2 It has a transform matrix of type basic_matrix_2d.

11.3.6.1 abs_matrix constructors [io2d.absmatrix.cons]

```
abs_matrix() noexcept;
```

1 *Effects:* Equivalent to: abs_matrix{ basic_matrix_2d() };

```
explicit abs_matrix(const basic_matrix_2d<typename
    GraphicsSurfaces::graphics_math_type>& m) noexcept;
```

2 *Requires:* m.is_invertible() is true.

3 *Effects:* Constructs an object of type abs_matrix.

4 The transform matrix is m.

```
abs_matrix(const abs_matrix& other);
abs_matrix(abs_matrix&& other) noexcept;
```

5 *Effects:* Constructs an object of type abs_matrix. In the second form, other is left in a valid state with an unspecified value.

11.3.6.2 abs_matrix assignment operators [io2d.absmatrix.assign]

```
abs_matrix& operator=(const abs_matrix& other);
```

1 *Effects:* If `*this` and `other` are not the same object, modifies `*this` such that `*this.matrix()` is `other.matrix()`

2 If `*this` and `other` are the same object, the member has no effect.

3 *Returns:* `*this`

```
abs_matrix& operator=(abs_matrix&& other) noexcept;
```

4 *Effects:* <TODO>

5 *Returns:* `*this`

11.3.6.3 `abs_matrix` modifiers [io2d.absmatrix.modifiers]

```
void matrix(const basic_matrix_2d<typename GraphicsSurfaces::graphics_math_type>& m) noexcept;
```

1 *Requires:* `m.is_invertible()` is true.

2 *Effects:* The transform matrix is `m`.

11.3.6.4 `abs_matrix` observers [io2d.absmatrix.observers]

```
basic_matrix_2d<typename GraphicsSurfaces::graphics_math_type> matrix() const noexcept;
```

1 *Returns:* The transform matrix.

11.3.6.5 `abs_matrix` operators [io2d.absmatrix.ops]

```
template <class GraphicsSurfaces>
bool operator==(const typename basic_figure_items<GraphicsSurfaces>::abs_matrix& lhs,
    const typename basic_figure_items<GraphicsSurfaces>::abs_matrix& rhs) noexcept;
```

1 *Returns:* `lhs.matrix() == rhs.matrix()`.

11.3.7 Class `rel_matrix` [io2d.relmatrix]

1 The class `rel_matrix` describes a figure item that is a path command.

2 It has a transform matrix of type `basic_matrix_2d`.

11.3.7.1 `rel_matrix` constructors [io2d.relmatrix.cons]

```
rel_matrix() noexcept;
```

1 *Effects:* Equivalent to: `rel_matrix{ basic_matrix_2d() };`

```
explicit rel_matrix(const basic_matrix_2d<typename
    GraphicsSurfaces::graphics_math_type>& m) noexcept;
```

2 *Requires:* `m.is_invertible()` is true.

3 *Effects:* Constructs an object of type `rel_matrix`.

4 The transform matrix is `m`.

```
rel_matrix(const rel_matrix& other);
rel_matrix(rel_matrix&& other) noexcept;
```

5 *Effects:* Constructs an object of type `rel_matrix`. In the second form, `other` is left in a valid state with an unspecified value.

11.3.7.2 `rel_matrix` assignment operators [io2d.relmatrix.assign]

```
rel_matrix& operator=(const rel_matrix& other);
```

1 *Effects:* If `*this` and `other` are not the same object, modifies `*this` such that `*this.matrix()` is `other.matrix()`

2 If `*this` and `other` are the same object, the member has no effect.

3 *Returns:* `*this`

```
rel_matrix& operator=(rel_matrix&& other) noexcept;
```

4 *Effects:* <TODO>

5 *Returns:* `*this`

11.3.7.3 rel_matrix modifiers [io2d.relmatrix.modifiers]

```
void matrix(const basic_matrix_2d<typename GraphicsSurfaces::graphics_math_type>& m) noexcept;
```

1 *Requires:* `m.is_invertible()` is true.

2 *Effects:* The transform matrix is `m`.

11.3.7.4 rel_matrix observers [io2d.relmatrix.observers]

```
basic_matrix_2d<typename GraphicsSurfaces::graphics_math_type> matrix() const noexcept;
```

1 *Returns:* The transform matrix.

11.3.7.5 rel_matrix operators [io2d.relmatrix.ops]

```
template <class GraphicsSurfaces>
bool operator==(const typename basic_figure_items<GraphicsSurfaces>::rel_matrix& lhs,
    const typename basic_figure_items<GraphicsSurfaces>::rel_matrix& rhs) noexcept;
```

1 *Returns:* `lhs.matrix() == rhs.matrix()`.

11.3.8 Class revert_matrix [io2d.revertmatrix]

1 The class `revert_matrix` describes a figure item that is a path command.

11.3.8.1 revert_matrix constructors [io2d.revertmatrix.cons]

```
revert_matrix() noexcept;
```

1 *Effects:* Constructs an object of type `revert_matrix`.

11.3.8.2 revert_matrix operators [io2d.revertmatrix.ops]

```
template <class GraphicsSurfaces>
bool operator==(const typename basic_figure_items<GraphicsSurfaces>::revert_matrix& lhs,
    const typename basic_figure_items<GraphicsSurfaces>::revert_matrix& rhs) noexcept;
```

1 *Returns:* true.

11.3.9 Class abs_line [io2d.absline]

1 The class `abs_line` describes a figure item that is a segment.

2 It has an *end point* of type `basic_point_2d`.

11.3.9.1 abs_line constructors [io2d.absline.cons]

```
abs_line() noexcept;
```

1 *Effects:* Equivalent to: `abs_line{ basic_point_2d() };`

```
explicit abs_line(const basic_point_2d<typename
    GraphicsSurfaces::graphics_math_type>& pt) noexcept;
```

2 *Effects:* Constructs an object of type `abs_line`.

3 The end point is `pt`.

```
abs_line(const abs_line& other);
abs_line(abs_line&& other) noexcept;
```

4 *Effects:* Constructs an object of type `abs_line`. In the second form, `other` is left in a valid state with an unspecified value.

5 The end point is `other.to()`.

11.3.9.2 `abs_line` assignment operators

[io2d.absline.assign]

```
abs_line& operator=(const abs_line& other);
```

1 *Effects:* If `*this` and `other` are not the same object, modifies `*this` such that `*this.to()` is `other.to()`

2 If `*this` and `other` are the same object, the member has no effect.

3 *Returns:* `*this`

```
abs_line& operator=(abs_line&& other) noexcept;
```

4 *Effects:* <TODO>

5 *Returns:* `*this`

11.3.9.3 `abs_line` modifiers

[io2d.absline.modifiers]

```
void to(const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& pt) noexcept;
```

1 *Effects:* The end point is `pt`.

11.3.9.4 `abs_line` observers

[io2d.absline.observers]

```
basic_point_2d<typename GraphicsSurfaces::graphics_math_type> to() const noexcept;
```

1 *Returns:* The end point.

11.3.9.5 `abs_line` operators

[io2d.absline.ops]

```
template <class GraphicsSurfaces>
bool operator==(const typename basic_figure_items<GraphicsSurfaces>::abs_line& lhs,
    const typename basic_figure_items<GraphicsSurfaces>::abs_line& rhs) noexcept;
```

1 *Returns:* `lhs.to() == rhs.to()`.

11.3.10 Class `rel_line`

[io2d.relline]

1 The class `rel_line` describes a figure item that is a segment.

2 It has an *end point* of type `basic_point_2d`.

11.3.10.1 `rel_line` constructors

[io2d.relline.cons]

```
rel_line() noexcept;
```

1 *Effects:* Equivalent to: `rel_line{ basic_point_2d() };`

```
explicit rel_line(const basic_point_2d<typename
    GraphicsSurfaces::graphics_math_type>& pt) noexcept;
```

2 *Effects:* Constructs an object of type `rel_line`.

3 The end point is `pt`.

```
rel_line(const rel_line& other);
rel_line(rel_line&& other) noexcept;
```

4 *Effects:* Constructs an object of type `rel_line`. In the second form, `other` is left in a valid state with an unspecified value.

5 The end point is `other.to()`.

11.3.10.2 `rel_line` assignment operators [io2d.relline.assign]

```
rel_line& operator=(const rel_line& other);
```

1 *Effects:* If `*this` and `other` are not the same object, modifies `*this` such that `*this.to()` is `other.to()`

2 If `*this` and `other` are the same object, the member has no effect.

3 *Returns:* `*this`

```
rel_line& operator=(rel_line&& other) noexcept;
```

4 *Effects:* <TODO>

5 *Returns:* `*this`

11.3.10.3 `rel_line` modifiers [io2d.relline.modifiers]

```
void to(const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& pt) noexcept;
```

1 *Effects:* The end point is `pt`.

11.3.10.4 `rel_line` observers [io2d.relline.observers]

```
basic_point_2d<typename GraphicsSurfaces::graphics_math_type> to() const noexcept;
```

1 *Returns:* The end point.

11.3.10.5 `rel_line` operators [io2d.relline.ops]

```
template <class GraphicsSurfaces>
bool operator==(const typename basic_figure_items<GraphicsSurfaces>::rel_line& lhs,
    const typename basic_figure_items<GraphicsSurfaces>::rel_line& rhs) noexcept;
```

1 *Returns:* `lhs.to() == rhs.to()`.

11.3.11 Class `abs_quadratic_curve` [io2d.absquadraticcurve]

1 The class `abs_quadratic_curve` describes a figure item that is a segment.

2 It has a *control point* of type `basic_point_2d` and an *end point* of type `basic_point_2d`.

11.3.11.1 `abs_quadratic_curve` constructors [io2d.absquadraticcurve.cons]

```
abs_quadratic_curve() noexcept;
```

1 *Effects:* Equivalent to: `abs_quadratic_curve{ basic_point_2d(), basic_point_2d() };`

```
abs_quadratic_curve(const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& cpt,
    const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& ept) noexcept;
```

2 *Effects:* Constructs an object of type `abs_quadratic_curve`.

3 The control point is `cpt`.

4 The end point is `ept`.

```
abs_quadratic_curve(const abs_quadratic_curve& other);
abs_quadratic_curve(abs_quadratic_curve&& other) noexcept;
```

5 *Effects:* Constructs an object of type `abs_quadratic_curve`. In the second form, `other` is left in a valid state with an unspecified value.

6 The control point is `other.control_pt()`.

7 The end point is `other.end_pt()`.

11.3.11.2 `abs_quadratic_curve` assignment operators [io2d.absquadraticcurve.assign]

```
abs_quadratic_curve& operator=(const abs_quadratic_curve& other);
```

1 *Effects:* If `*this` and `other` are not the same object, modifies `*this` such that `*this.control_pt()` is `other.control_pt()` and `*this.end_pt()` is `other.end_pt()`

2 If `*this` and `other` are the same object, the member has no effect.

3 *Returns:* `*this`

```
abs_quadratic_curve& operator=(abs_quadratic_curve&& other) noexcept;
```

4 *Effects:* <TODO>

5 *Returns:* `*this`

11.3.11.3 `abs_quadratic_curve` modifiers [io2d.absquadraticcurve.modifiers]

```
void control_pt(const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& cpt) noexcept;
```

1 *Effects:* The control point is `cpt`.

```
void end_pt(const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& ept) noexcept;
```

2 *Effects:* The end point is `ept`.

11.3.11.4 `abs_quadratic_curve` observers [io2d.absquadraticcurve.observers]

```
basic_point_2d<typename GraphicsSurfaces::graphics_math_type> control_pt() const noexcept;
```

1 *Returns:* The control point.

```
basic_point_2d<typename GraphicsSurfaces::graphics_math_type> end_pt() const noexcept;
```

2 *Returns:* The end point.

11.3.11.5 `abs_quadratic_curve` operators [io2d.absquadraticcurve.ops]

```
template <class GraphicsSurfaces>
bool operator==(const typename basic_figure_items<GraphicsSurfaces>::abs_quadratic_curve& lhs,
  const typename basic_figure_items<GraphicsSurfaces>::abs_quadratic_curve& rhs) noexcept;
```

1 *Returns:* `lhs.control_pt() == rhs.control_pt() && lhs.end_pt() == rhs.end_pt()`.

11.3.12 Class `rel_quadratic_curve` [io2d.relquadraticcurve]

1 The class `rel_quadratic_curve` describes a figure item that is a segment.

2 It has a *control point* of type `basic_point_2d` and an *end point* of type `basic_point_2d`.

11.3.12.1 rel_quadratic_curve constructors [io2d.relquadraticcurve.cons]

```
rel_quadratic_curve() noexcept;
```

1 *Effects:* Equivalent to: `rel_quadratic_curve{ basic_point_2d(), basic_point_2d() }`;

```
rel_quadratic_curve(const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& cpt,
    const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& ept) noexcept;
```

2 *Effects:* Constructs an object of type `rel_quadratic_curve`.

3 The control point is `cpt`.

4 The end point is `ept`.

```
rel_quadratic_curve(const rel_quadratic_curve& other);
rel_quadratic_curve(rel_quadratic_curve&& other) noexcept;
```

5 *Effects:* Constructs an object of type `rel_quadratic_curve`. In the second form, `other` is left in a valid state with an unspecified value.

6 The control point is `other.control_pt()`.

7 The end point is `other.end_pt()`.

11.3.12.2 rel_quadratic_curve assignment operators [io2d.relquadraticcurve.assign]

```
rel_quadratic_curve& operator=(const rel_quadratic_curve& other);
```

1 *Effects:* If `*this` and `other` are not the same object, modifies `*this` such that `*this.control_pt()` is `other.control_pt()` and `*this.end_pt()` is `other.end_pt()`

2 If `*this` and `other` are the same object, the member has no effect.

3 *Returns:* `*this`

```
rel_quadratic_curve& operator=(rel_quadratic_curve&& other) noexcept;
```

4 *Effects:* <TODO>

5 *Returns:* `*this`

11.3.12.3 rel_quadratic_curve modifiers [io2d.relquadraticcurve.modifiers]

```
void control_pt(const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& cpt) noexcept;
```

1 *Effects:* The control point is `cpt`.

```
void end_pt(const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& ept) noexcept;
```

2 *Effects:* The end point is `ept`.

11.3.12.4 rel_quadratic_curve observers [io2d.relquadraticcurve.observers]

```
basic_point_2d<typename GraphicsSurfaces::graphics_math_type> control_pt() const noexcept;
```

1 *Returns:* The control point.

```
basic_point_2d<typename GraphicsSurfaces::graphics_math_type> end_pt() const noexcept;
```

2 *Returns:* The end point.

11.3.12.5 rel_quadratic_curve operators [io2d.relquadraticcurve.ops]

```
template <class GraphicsSurfaces>
bool operator==(const typename basic_figure_items<GraphicsSurfaces>::rel_quadratic_curve& lhs,
    const typename basic_figure_items<GraphicsSurfaces>::rel_quadratic_curve& rhs) noexcept;
```

1 *Returns:* lhs.control_pt() == rhs.control_pt() && lhs.end_pt() == rhs.end_pt().

11.3.13 Class abs_cubic_curve [io2d.abscubiccurve]

1 The class abs_cubic_curve describes a figure item that is a segment.

2 It has a *first control point* of type basic_point_2d, a *second control point* of type basic_point_2d, and an end point of type basic_point_2d.

11.3.13.1 abs_cubic_curve constructors [io2d.abscubiccurve.cons]

abs_cubic_curve() noexcept;

1 *Effects:* Equivalent to abs_cubic_curve{ basic_point_2d(), basic_point_2d(), basic_point_2d() }.

abs_cubic_curve(const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& cpt1,
const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& cpt2,
const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& ept) noexcept;

2 *Effects:* Constructs an object of type abs_cubic_curve.

3 The first control point is cpt1.

4 The second control point is cpt2.

5 The end point is ept.

abs_cubic_curve(const abs_cubic_curve& other);
abs_cubic_curve(abs_cubic_curve&& other) noexcept;

6 *Effects:* Constructs an object of type abs_cubic_curve. In the second form, other is left in a valid state with an unspecified value.

7 The first control point is other.control_pt1().

8 The second control point is other.control_pt2().

9 The end point is other.end_pt().

11.3.13.2 abs_cubic_curve assignment operators [io2d.abscubiccurve.assign]

abs_cubic_curve& operator=(const abs_cubic_curve& other);

1 *Effects:* If *this and other are not the same object, modifies *this such that *this.control_pt1() is other.control_pt1(), *this.control_pt2() is other.control_pt2() and *this.end_pt() is other.end_pt().

2 If *this and other are the same object, the member has no effect.

3 *Returns:* *this

abs_cubic_curve& operator=(abs_cubic_curve&& other) noexcept;

4 *Effects:* <TODO>

5 *Returns:* *this

11.3.13.3 abs_cubic_curve modifiers [io2d.abscubiccurve.modifiers]

void control_pt1(const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& cpt) noexcept;

1 *Effects:* The first control point is cpt.

void control_pt2(const basic_point_2d<typename


```
GraphicsSurfaces::graphics_math_type>& cpt) noexcept;
```

2 *Effects:* The second control point is `cpt`.

```
void end_pt(const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& ept) noexcept;
```

3 *Effects:* The end point is `ept`.

11.3.13.4 abs_cubic_curve observers [io2d.abscubiccurve.observers]

```
basic_point_2d<typename GraphicsSurfaces::graphics_math_type> control_pt1() const noexcept;
```

1 *Returns:* The first control point.

```
basic_point_2d<typename GraphicsSurfaces::graphics_math_type> control_pt2() const noexcept;
```

2 *Returns:* The second control point.

```
basic_point_2d<typename GraphicsSurfaces::graphics_math_type> end_pt() const noexcept;
```

3 *Returns:* The end point.

11.3.13.5 abs_cubic_curve operators [io2d.abscubiccurve.ops]

```
template <class GraphicsSurfaces>
```

```
bool operator==(const typename basic_figure_items<GraphicsSurfaces>::abs_cubic_curve& lhs,  
               const typename basic_figure_items<GraphicsSurfaces>::abs_cubic_curve& rhs) noexcept;
```

1 *Returns:*

```
    lhs.control_pt1() == rhs.control_pt1() &&  
    lhs.control_pt2() == rhs.control_pt2() &&  
    lhs.end_pt() && rhs.end_pt()
```

11.3.14 Class rel_cubic_curve [io2d.relcubiccurve]

1 The class `rel_cubic_curve` describes a figure item that is a segment.

2 It has a *first control point* of type `basic_point_2d`, a *second control point* of type `basic_point_2d`, and an end point of type `basic_point_2d`.

11.3.14.1 rel_cubic_curve constructors [io2d.relcubiccurve.cons]

```
rel_cubic_curve() noexcept;
```

1 *Effects:* Equivalent to `rel_cubic_curve{ basic_point_2d(), basic_point_2d(), basic_point_2d() }`.

```
rel_cubic_curve(const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& cpt1,  
               const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& cpt2,  
               const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& ept) noexcept;
```

2 *Effects:* Constructs an object of type `rel_cubic_curve`.

3 The first control point is `cpt1`.

4 The second control point is `cpt2`.

5 The end point is `ept`.

```
rel_cubic_curve(const rel_cubic_curve& other);  
rel_cubic_curve(rel_cubic_curve&& other) noexcept;
```

6 *Effects:* Constructs an object of type `rel_cubic_curve`. In the second form, `other` is left in a valid state with an unspecified value.

- 7 The first control point is `other.control_pt1()`.
 8 The second control point is `other.control_pt2()`.
 9 The end point is `other.end_pt()`.

11.3.14.2 `rel_cubic_curve` assignment operators [io2d.relcubiccurve.assign]

```
rel_cubic_curve& operator=(const rel_cubic_curve& other);
```

- 1 *Effects:* If `*this` and `other` are not the same object, modifies `*this` such that `*this.control_pt1()` is `other.control_pt1()`, `*this.control_pt2()` is `other.control_pt2()` and `*this.end_pt()` is `other.end_pt()`.
 2 If `*this` and `other` are the same object, the member has no effect.
 3 *Returns:* `*this`

```
rel_cubic_curve& operator=(rel_cubic_curve&& other) noexcept;
```

- 4 *Effects:* <TODO>
 5 *Returns:* `*this`

11.3.14.3 `rel_cubic_curve` modifiers [io2d.relcubiccurve.modifiers]

```
void control_pt1(const basic_point_2d<typename  
GraphicsSurfaces::graphics_math_type>& cpt) noexcept;
```

- 1 *Effects:* The first control point is `cpt`.

```
void control_pt2(const basic_point_2d<typename  
GraphicsSurfaces::graphics_math_type>& cpt) noexcept;
```

- 2 *Effects:* The second control point is `cpt`.

```
void end_pt(const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& ept) noexcept;
```

- 3 *Effects:* The end point is `ept`.

11.3.14.4 `rel_cubic_curve` observers [io2d.relcubiccurve.observers]

```
basic_point_2d<typename GraphicsSurfaces::graphics_math_type> control_pt1() const noexcept;
```

- 1 *Returns:* The first control point.

```
basic_point_2d<typename GraphicsSurfaces::graphics_math_type> control_pt2() const noexcept;
```

- 2 *Returns:* The second control point.

```
basic_point_2d<typename GraphicsSurfaces::graphics_math_type> end_pt() const noexcept;
```

- 3 *Returns:* The end point.

11.3.14.5 `rel_cubic_curve` operators [io2d.relcubiccurve.ops]

```
template <class GraphicsSurfaces>  
bool operator==(const typename basic_figure_items<GraphicsSurfaces>::rel_cubic_curve& lhs,  
const typename basic_figure_items<GraphicsSurfaces>::rel_cubic_curve& rhs) noexcept;
```

- 1 *Returns:*
 `lhs.control_pt1() == rhs.control_pt1() &&`
 `lhs.control_pt2() == rhs.control_pt2() &&`
 `lhs.end_pt() && rhs.end_pt()`

11.3.15 Class arc**[io2d.arc]****11.3.15.1 In general****[io2d.arc.general]**

- 1 The class `arc` describes a figure item that is a segment.
- 2 It has a *radius* of type `basic_point_2d`, a *rotation* of type `float`, and a *start angle* of type `float`.
- 3 It forms a portion of the circumference of a circle. The centre of the circle is implied by the start point, the radius and the start angle of the arc.

11.3.15.2 arc constructors**[io2d.arc.cons]**

```
arc() noexcept;
```

- 1 *Effects:* Equivalent to: `arc{ basic_point_2d(10.0f, 10.0f), pi<float>, pi<float> };`

```
arc(const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& rad,  
     float rot, float sang) noexcept;
```

- 2 *Effects:* Constructs an object of type `arc`.
- 3 The radius is `rad`.
- 4 The rotation is `rot`.
- 5 The start angle is `sang`.

```
arc(const arc& other);  
arc(arc&& other) noexcept;
```

- 6 *Effects:* Constructs an object of type `arc`. In the second form, `other` is left in a valid state with an unspecified value.
- 7 The radius is `other.radius()`.
- 8 The rotation is `other.rotation()`.
- 9 The start angle is `other.start_angle()`.

11.3.15.3 arc assignment operators**[io2d.arc.assign]**

```
arc& operator=(const arc& other);
```

- 1 *Effects:* If `*this` and `other` are not the same object, modifies `*this` such that `*this.radius()` is `other.radius()`, `*this.rotation()` is `other.rotation()` and `*this.start_angle()` is `other.start_angle()`.
- 2 If `*this` and `other` are the same object, the member has no effect.
- 3 *Returns:* `*this`

```
arc& operator=(arc&& other) noexcept;
```

- 4 *Effects:* <TODO>
- 5 *Returns:* `*this`

11.3.15.4 arc modifiers**[io2d.arc.modifiers]**

```
void radius(const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& rad) noexcept;
```

- 1 *Effects:* The radius is `rad`.

```
constexpr void rotation(float rot) noexcept;
```

- 2 *Effects:* The rotation is `rot`.

```
void start_angle(float sang) noexcept;
```

3 *Effects:* The start angle is sang.

11.3.15.5 arc observers

[io2d.arc.observers]

```
basic_point_2d<typename GraphicsSurfaces::graphics_math_type> radius() const noexcept;
```

1 *Returns:* The radius.

```
float rotation() const noexcept;
```

2 *Returns:* The rotation.

```
float start_angle() const noexcept;
```

3 *Returns:* The start angle.

```
basic_point_2d<typename GraphicsSurfaces::graphics_math_type> center(const basic_point_2d<typename
GraphicsSurfaces::graphics_math_type>& cpt, const basic_matrix_2d<typename
GraphicsSurfaces::graphics_math_type>& m = basic_matrix_2d<typename
GraphicsSurfaces::graphics_math_type>{ }) const noexcept;
```

4 *Returns:* As-if:

```
auto lmtx = m;
lmtx.m20 = 0.0f;
lmtx.m21 = 0.0f;
auto centerOffset = point_for_angle(two_pi<float> - start_angle(), radius());
centerOffset.y = -centerOffset.y;
return cpt - centerOffset * lmtx;
```

```
basic_point_2d<typename GraphicsSurfaces::graphics_math_type> end_pt(const basic_point_2d<typename
GraphicsSurfaces::graphics_math_type>& cpt, const basic_matrix_2d<typename
GraphicsSurfaces::graphics_math_type>& m = basic_matrix_2d<typename
GraphicsSurfaces::graphics_math_type>{ }) const noexcept;
```

5 *Returns:* As-if:

```
auto lmtx = m;
auto tfrm = matrix_2d::init_rotate(start_angle() + rotation());
lmtx.m20 = 0.0f;
lmtx.m21 = 0.0f;
auto pt = (radius() * tfrm);
pt.y = -pt.y;
return cpt + pt * lmtx;
```

11.3.15.6 arc operators

[io2d.arc.ops]

```
template <class GraphicsSurfaces>
```

```
bool operator==(const typename basic_figure_items<GraphicsSurfaces>::arc& lhs,
const typename basic_figure_items<GraphicsSurfaces>::arc& rhs) noexcept;
```

1 *Returns:*

```
lhs.radius() == rhs.radius() && lhs.rotation() == rhs.rotation() &&
lhs.start_angle() && rhs.start_angle()
```

11.3.16 Path interpretation

[io2d.paths.interpretation]

1 This subclause describes how to interpret a path for use in a rendering and composing operation.

- 2 Interpreting a path consists of sequentially evaluating the figure items contained in the figures in the path and transforming them into zero or more figures as-if in the manner specified in this subclause.
- 3 The interpretation of a path requires the state data specified in Table 3.

Table 3 — Path interpretation state data

Name	Description	Type	Initial value
<code>mtx</code>	Path transformation matrix	<code>matrix_2d</code>	<code>matrix_2d{ }</code>
<code>currPt</code>	Current point	<code>point_2d</code>	<i>unspecified</i>
<code>lnfPt</code>	Last new figure point	<code>point_2d</code>	<i>unspecified</i>
<code>mtxStk</code>	Matrix stack	<code>stack<matrix_2d></code>	<code>stack<matrix_2d>{ }</code>

- 4 When interpreting a path, until a `figure_items::abs_new_figure` figure item is reached, a path shall only contain path command figure items; no diagnostic is required. If a figure is a degenerate figure, none of its figure items have any effects, with two exceptions:
 - (4.1) — the path's `figure_items::abs_new_figure` or `figure_items::rel_new_figure` figure item sets the value of `currPt` as-if the figure item was interpreted; and,
 - (4.2) — any path command figure items are evaluated with full effect.
- 5 The effects of a figure item contained in a `figure_items::figure_item` object when that object is being evaluated during path interpretation are described in Table 4.
- 6 If evaluation of a figure item contained in a `figure_items::figure_item` during path interpretation results in the figure item becoming a degenerate segment, its effects are ignored and interpretation continues as-if that figure item did not exist.

Table 4 — Figure item interpretation effects

Figure item	Effects
<code>figure_items::abs_new_figure p</code>	Creates a new figure. Sets <code>currPt</code> to <code>mtx.transform_pt({ 0.0f, 0.0f }) + p.at()</code> . Sets <code>lnfPt</code> to <code>currPt</code> .
<code>figure_items::rel_new_figure p</code>	Let <code>mm</code> equal <code>mtx</code> . Let <code>mm.m20</code> equal <code>0.0f</code> . Let <code>mm.m21</code> equal <code>0.0f</code> . Creates a new figure. Sets <code>currPt</code> to <code>currPt + p.at() * mm</code> . Sets <code>lnfPt</code> to <code>currPt</code> .
<code>figure_items::close_figure p</code>	Creates a line from <code>currPt</code> to <code>lnfPt</code> . Makes the current figure a closed figure. Creates a new figure. Sets <code>currPt</code> to <code>lnfPt</code> .
<code>figure_items::abs_matrix p</code>	Calls <code>mtxStk.push(mtx)</code> . Sets <code>mtx</code> to <code>p.matrix()</code> .
<code>figure_items::rel_matrix p</code>	Calls <code>mtxStk.push(mtx)</code> . Sets <code>mtx</code> to <code>mtx * p.matrix()</code> .
<code>figure_items::revert_matrix p</code>	If <code>mtxStk.empty()</code> is <code>false</code> , sets <code>mtx</code> to <code>mtxStk.top()</code> then calls <code>mtxStk.pop()</code> . Otherwise sets <code>mtx</code> to its initial value as specified in Table 3.
<code>figure_items::abs_line p</code>	Let <code>pt</code> equal <code>mtx.transform_pt(p.to() - currPt) + currPt</code> . Creates a line from <code>currPt</code> to <code>pt</code> . Sets <code>currPt</code> to <code>pt</code> .
<code>figure_items::rel_line p</code>	Let <code>mm</code> equal <code>mtx</code> . Let <code>mm.m20</code> equal <code>0.0f</code> . Let <code>mm.m21</code> equal <code>0.0f</code> . Let <code>pt</code> equal <code>currPt + p.to() * mm</code> . Creates a line from <code>currPt</code> to <code>pt</code> . Sets <code>currPt</code> to <code>pt</code> .

Table 4 — Figure item interpretation effects (continued)

Figure item	Effects
<code>figure_items::abs_quadratic_curve</code> <code>p</code>	Let <code>cpt</code> equal <code>mtx.transform_pt(p.control_pt() - currPt) + currPt</code> . Let <code>ept</code> equal <code>mtx.transform_pt(p.end_pt() - currPt) + currPt</code> . Creates a quadratic Bézier curve from <code>currPt</code> to <code>ept</code> using <code>cpt</code> as the curve's control point. Sets <code>currPt</code> to <code>ept</code> .
<code>figure_items::rel_quadratic_curve</code> <code>p</code>	Let <code>mm</code> equal <code>mtx</code> . Let <code>mm.m20</code> equal <code>0.0f</code> . Let <code>mm.m21</code> equal <code>0.0f</code> . Let <code>cpt</code> equal <code>currPt + p.control_pt() * mm</code> . Let <code>ept</code> equal <code>currPt + p.control_pt() * mm + p.end_pt() * mm</code> . Creates a quadratic Bézier curve from <code>currPt</code> to <code>ept</code> using <code>cpt</code> as the curve's control point. Sets <code>currPt</code> to <code>ept</code> .
<code>figure_items::abs_cubic_curve</code> <code>p</code>	Let <code>cpt1</code> equal <code>mtx.transform_pt(p.control_pt1() - currPt) + currPt</code> . Let <code>cpt2</code> equal <code>mtx.transform_pt(p.control_pt2() - currPt) + currPt</code> . Let <code>ept</code> equal <code>mtx.transform_pt(p.end_pt() - currPt) + currPt</code> . Creates a cubic Bézier curve from <code>currPt</code> to <code>ept</code> using <code>cpt1</code> as the curve's first control point and <code>cpt2</code> as the curve's second control point. Sets <code>currPt</code> to <code>ept</code> .
<code>figure_items::rel_cubic_curve</code> <code>p</code>	Let <code>mm</code> equal <code>mtx</code> . Let <code>mm.m20</code> equal <code>0.0f</code> . Let <code>mm.m21</code> equal <code>0.0f</code> . Let <code>cpt1</code> equal <code>currPt + p.control_pt1() * mm</code> . Let <code>cpt2</code> equal <code>currPt + p.control_pt1() * mm + p.control_pt2() * mm</code> . Let <code>ept</code> equal <code>currPt + p.control_pt1() * mm + p.control_pt2() * mm + p.end_pt() * mm</code> . Creates a cubic Bézier curve from <code>currPt</code> to <code>ept</code> using <code>cpt1</code> as the curve's first control point and <code>cpt2</code> as the curve's second control point. Sets <code>currPt</code> to <code>ept</code> .
<code>figure_items::arc</code> <code>p</code>	Let <code>mm</code> equal <code>mtx</code> . Let <code>mm.m20</code> equal <code>0.0f</code> . Let <code>mm.m21</code> equal <code>0.0f</code> . Creates an arc. It begins at <code>currPt</code> , which is at <code>p.start_angle()</code> radians on the arc and rotates <code>p.rotation()</code> radians. If <code>p.rotation()</code> is positive, rotation is counterclockwise, otherwise it is clockwise. The center of the arc is located at <code>p.center(currPt, mm)</code> . The arc ends at <code>p.end_pt(currPt, mm)</code> . Sets <code>currPt</code> to <code>p.end_pt(currPt, mm)</code> . [<i>Note:</i> <code>p.radius()</code> , which specifies the radius of the arc, is implicitly included in the above statement of effects by the specifications of the center of the arc and the end of the arcs. The use of the current point as the origin for the application of the path transformation matrix is also implicitly included by the same specifications. — <i>end note</i>]

11.4 Class `basic_interpreted_path`

[io2d.pathgroup]

- ¹ The class `basic_interpreted_path` contains the data that result from interpreting (11.3.16) a sequence of `figure_items::figure_item` objects.
- ² A `basic_interpreted_path` object is used by most rendering and composing operations.

11.4.1 `basic_interpreted_path` synopsis

[io2d.pathgroup.synopsis]

```
namespace std::experimental::io2d::v1 {
    template <class GraphicsSurfaces>
    class basic_interpreted_path {
    public:
        // 11.4.2, construct:
        basic_interpreted_path() noexcept;
        template <class Allocator>
```

```

        explicit basic_interpreted_path(const basic_path_builder<GraphicsSurfaces, Allocator>& pb);
        template <class ForwardIterator>
        basic_interpreted_path(ForwardIterator first, ForwardIterator last);
        explicit basic_interpreted_path(initializer_list<typename
            basic_figure_items<GraphicsSurfaces>::figure_item> il);
    };
}

```

11.4.2 basic_interpreted_path constructors

[io2d.pathgroup.cons]

```
basic_interpreted_path() noexcept;
```

- 1 *Effects:* Constructs a `basic_interpreted_path` that contains an empty path.

```

template <class Allocator>
explicit basic_interpreted_path(const basic_path_builder<GraphicsSurfaces, Allocator>& pb);

```

- 2 *Effects:* Equivalent to: `basic_interpreted_path{ begin(pb), end(pb) };`.

```

template <class ForwardIterator>
basic_interpreted_path(ForwardIterator first, ForwardIterator last);

```

- 3 *Effects:* Constructs an object of type `interpreted_path`.

- 4 The contained path is as-if it was the result of interpreting a path containing the values of the elements from `first` to the last element before `last`.

```

explicit basic_interpreted_path(initializer_list<typename
    basic_figure_items<GraphicsSurfaces>::figure_item> il);

```

- 5 *Effects:* <TODO>

11.5 Class basic_path_builder

[io2d.pathbuilder]

- 1 The class `basic_path_builder` is a container that stores and manipulates objects of type `figure_items::figure_item` from which `interpreted_path` objects are created.
- 2 A `basic_path_builder` is a contiguous container. (See [container.requirements.general] in N4618.)
- 3 The collection of `figure_items::figure_item` objects in a path builder is referred to as its path.

11.5.1 basic_path_builder synopsis

[io2d.pathbuilder.synopsis]

```

namespace std::experimental::io2d::v1 {
    template <class GraphicsSurfaces,
        class Allocator = ::std::allocator<typename
            basic_figure_items<GraphicsSurfaces>::figure_item>>
    class basic_path_builder {
    public:
        using value_type           = typename basic_figure_items<GraphicsSurfaces>::figure_item;
        using allocator_type       = Allocator;
        using reference             = value_type&;
        using const_reference      = const value_type&;
        using size_type            = implementation-defined. // See [container.requirements] in N4618.
        using difference_type      = implementation-defined. // See [container.requirements] in N4618.
        using iterator             = implementation-defined. // See [container.requirements] in N4618.
        using const_iterator       = implementation-defined. // See [container.requirements] in N4618.
        using reverse_iterator     = std::reverse_iterator<iterator>;
        using const_reverse_iterator = std::reverse_iterator<const_iterator>;

        // 11.5.3, construct, copy, move, destroy:

```

```

basic_path_builder() noexcept(noexcept(Allocator()));
explicit basic_path_builder(const Allocator&) noexcept;
explicit basic_path_builder(size_type n, const Allocator& = Allocator());
basic_path_builder(size_type n, const value_type& value, const Allocator& = Allocator());
template <class InputIterator>
basic_path_builder(InputIterator first, InputIterator last, const Allocator& = Allocator());
basic_path_builder(const basic_path_builder& x);
basic_path_builder(basic_path_builder&&) noexcept;
basic_path_builder(const basic_path_builder&, const Allocator&);
basic_path_builder(basic_path_builder&&, const Allocator&);
basic_path_builder(initializer_list<value_type>, const Allocator& = Allocator());
~basic_path_builder();
basic_path_builder& operator=(const basic_path_builder& x);
basic_path_builder& operator=(basic_path_builder&& x) noexcept(
    allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
    allocator_traits<Allocator>::is_always_equal::value);
basic_path_builder& operator=(initializer_list<value_type>);
template <class InputIterator>
void assign(InputIterator first, InputIterator last);
void assign(size_type n, const value_type& u);
void assign(initializer_list<value_type>);
allocator_type get_allocator() const noexcept;

// 11.5.4, capacity
bool empty() const noexcept;
size_type size() const noexcept;
size_type max_size() const noexcept;
size_type capacity() const noexcept;
void resize(size_type sz);
void resize(size_type sz, const value_type& c);
void reserve(size_type n);
void shrink_to_fit();

// element access:
reference operator[](size_type n);
const_reference operator[](size_type n) const;
const_reference at(size_type n) const;
reference at(size_type n);
reference front();
const_reference front() const;
reference back();
const_reference back() const;

// 11.5.5, modifiers:
void new_figure(const basic_point_2d<typename
    GraphicsSurfaces::graphics_math_type>& pt) noexcept;
void rel_new_figure(const basic_point_2d<typename
    GraphicsSurfaces::graphics_math_type>& pt) noexcept;
void close_figure() noexcept;
void matrix(const basic_matrix_2d<typename
    GraphicsSurfaces::graphics_math_type>& m) noexcept;
void rel_matrix(const basic_matrix_2d<typename
    GraphicsSurfaces::graphics_math_type>& m) noexcept;
void revert_matrix() noexcept;
void line(const basic_point_2d<typename GraphicsSurfaces::graphics_math_type>& pt) noexcept;

```



```

void rel_line(const basic_point_2d<typename
    GraphicsSurfaces::graphics_math_type>& dpt) noexcept;
void quadratic_curve(const basic_point_2d<typename
    GraphicsSurfaces::graphics_math_type>& pt0, const basic_point_2d<typename
    GraphicsSurfaces::graphics_math_type>& pt2) noexcept;
void rel_quadratic_curve(const basic_point_2d<typename
    GraphicsSurfaces::graphics_math_type>& pt0, const basic_point_2d<typename
    GraphicsSurfaces::graphics_math_type>& pt2) noexcept;
void cubic_curve(const basic_point_2d<typename
    GraphicsSurfaces::graphics_math_type>& pt0, const basic_point_2d<typename
    GraphicsSurfaces::graphics_math_type>& pt1, const basic_point_2d<typename
    GraphicsSurfaces::graphics_math_type>& pt2) noexcept;
void rel_cubic_curve(const basic_point_2d<typename
    GraphicsSurfaces::graphics_math_type>& dpt0, const basic_point_2d<typename
    GraphicsSurfaces::graphics_math_type>& dpt1, const basic_point_2d<typename
    GraphicsSurfaces::graphics_math_type>& dpt2) noexcept;
void arc(const basic_point_2d<typename
    GraphicsSurfaces::graphics_math_type>& rad, float rot, float sang = pi<float>) noexcept;
template <class... Args>
reference emplace_back(Args&&... args);
void push_back(const value_type& x);
void push_back(value_type&& x);
void pop_back();
template <class... Args>
iterator emplace(const_iterator position, Args&&... args);
iterator insert(const_iterator position, const value_type& x);
iterator insert(const_iterator position, value_type&& x);
iterator insert(const_iterator position, size_type n, const value_type& x);
template <class InputIterator>
iterator insert(const_iterator position, InputIterator first, InputIterator last);
iterator insert(const_iterator position,
    initializer_list<value_type> il);
iterator erase(const_iterator position);
iterator erase(const_iterator first, const_iterator last);
void swap(basic_path_builder&) noexcept(
    allocator_traits<Allocator>::propagate_on_container_swap::value ||
    allocator_traits<Allocator>::is_always_equal::value);
void clear() noexcept;

// 11.5.6, iterators:
iterator begin() noexcept;
const_iterator begin() const noexcept;
const_iterator cbegin() const noexcept;
iterator end() noexcept;
const_iterator end() const noexcept;
const_iterator cend() const noexcept;
reverse_iterator rbegin() noexcept;
const_reverse_iterator rbegin() const noexcept;
const_reverse_iterator crbegin() const noexcept;
reverse_iterator rend() noexcept;
const_reverse_iterator rend() const noexcept;
const_reverse_iterator crend() const noexcept;
};

template <class GraphicsSurfaces, class Allocator>

```

```

    bool operator==(const basic_path_builder<GraphicsSurfaces, Allocator>& lhs,
        const basic_path_builder<GraphicsSurfaces, Allocator>& rhs) noexcept;
    template <class GraphicsSurfaces, class Allocator>
    bool operator!=(const basic_path_builder<GraphicsSurfaces, Allocator>& lhs,
        const basic_path_builder<GraphicsSurfaces, Allocator>& rhs) noexcept;
    template <class GraphicsSurfaces, class Allocator>
    void swap(basic_path_builder<GraphicsSurfaces, Allocator>& lhs,
        basic_path_builder<GraphicsSurfaces, Allocator>& rhs) noexcept(noexcept(lhs.swap(rhs)));
}

```

11.5.2 basic_path_builder container requirements [io2d.pathbuilder.containerrequirements]

- ¹ This class is a sequence container, as defined in [containers] in N4618, and all sequence container requirements that apply specifically to `vector` shall also apply to this class.

11.5.3 basic_path_builder constructors, copy, and assignment [io2d.pathbuilder.cons]

```
basic_path_builder() noexcept(noexcept(Allocator()));
```

- ¹ *Effects:* Constructs an empty `basic_path_builder`.

```
explicit basic_path_builder(const Allocator&);
```

- ² *Effects:* Constructs an empty `basic_path_builder`, using the specified allocator.

- ³ *Complexity:* Constant.

```
explicit basic_path_builder(size_type n, const Allocator& = Allocator());
```

- ⁴ *Effects:* Constructs a `basic_path_builder` with `n` default-inserted elements using the specified allocator.

- ⁵ *Complexity:* Linear in `n`.

```
basic_path_builder(size_type n, const value_type& value,
    const Allocator& = Allocator());
```

- ⁶ *Requires:* `value_type` shall be CopyInsertable into `*this`.

- ⁷ *Effects:* Constructs a `basic_path_builder` with `n` copies of `value`, using the specified allocator.

- ⁸ *Complexity:* Linear in `n`.

```
template <class InputIterator>
basic_path_builder(InputIterator first, InputIterator last,
    const Allocator& = Allocator());
```

- ⁹ *Effects:* Constructs a `basic_path_builder` equal to the range `[first, last)`, using the specified allocator.

- ¹⁰ *Complexity:* Makes only N calls to the copy constructor of `value_type` (where N is the distance between `first` and `last`) and no reallocations if iterators `first` and `last` are of forward, bidirectional, or random access categories. It makes order N calls to the copy constructor of `value_type` and order $\log(N)$ reallocations if they are just input iterators.

11.5.4 basic_path_builder capacity [io2d.pathbuilder.capacity]

```
size_type capacity() const noexcept;
```

- ¹ *Returns:* The total number of elements that the path builder can hold without requiring reallocation.

```
void reserve(size_type n);
```

Requires: `value_type` shall be `MoveInsertable` into `*this`.

Effects: A directive that informs a path builder of a planned change in size, so that it can manage the storage allocation accordingly. After `reserve()`, `capacity()` is greater or equal to the argument of `reserve` if reallocation happens; and equal to the previous value of `capacity()` otherwise. Reallocation happens at this point if and only if the current capacity is less than the argument of `reserve()`. If an exception is thrown other than by the move constructor of a non-`CopyInsertable` type, there are no effects.

Complexity: It does not change the size of the sequence and takes at most linear time in the size of the sequence.

Throws: `length_error` if `n > max_size()`.¹

Remarks: Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence. No reallocation shall take place during insertions that happen after a call to `reserve()` until the time when an insertion would make the size of the vector greater than the value of `capacity()`.

```
void shrink_to_fit();
```

Requires: `value_type` shall be `MoveInsertable` into `*this`.

Effects: `shrink_to_fit` is a non-binding request to reduce `capacity()` to `size()`. [*Note:* The request is non-binding to allow latitude for implementation-specific optimizations. — *end note*] It does not increase `capacity()`, but may reduce `capacity()` by causing reallocation. If an exception is thrown other than by the move constructor of a non-`CopyInsertable` `value_type` there are no effects.

Complexity: Linear in the size of the sequence.

Remarks: Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence. If no reallocation happens, they remain valid.

```
void swap(basic_path_builder&)
    noexcept(allocator_traits<Allocator>::propagate_on_container_swap::value ||
             allocator_traits<Allocator>::is_always_equal::value);
```

Effects: Exchanges the contents and `capacity()` of `*this` with that of `x`.

Complexity: Constant time.

```
resize
```

```
void resize(size_type sz);
```

Effects: If `sz < size()`, erases the last `size() - sz` elements from the sequence. Otherwise, appends `sz - size()` default-inserted elements to the sequence.

Requires: `value_type` shall be `MoveInsertable` and `DefaultInsertable` into `*this`.

Remarks: If an exception is thrown other than by the move constructor of a non-`CopyInsertable` `value_type` there are no effects.

```
resize
```

```
void resize(size_type sz, const value_type& c);
```

Effects: If `sz < size()`, erases the last `size() - sz` elements from the sequence. Otherwise, appends `sz - size()` copies of `c` to the sequence.

Requires: `value_type` shall be `CopyInsertable` into `*this`.

Remarks: If an exception is thrown there are no effects.

¹ `reserve()` uses `Allocator::allocate()` which may throw an appropriate exception.

11.5.5 basic_path_builder modifiers

[io2d.pathbuilder.modifiers]

```
void new_figure(point_2d pt) noexcept;
```

- 1 *Effects:* Adds a `figure_items::figure_item` object constructed from `figure_items::abs_new_figure(pt)` to the end of the path.

```
void rel_new_figure(point_2d pt) noexcept;
```

- 2 *Effects:* Adds a `figure_items::figure_item` object constructed from `figure_items::rel_new_figure(pt)` to the end of the path.

```
void close_figure() noexcept;
```

- 3 *Requires:* The current point contains a value.

- 4 *Effects:* Adds a `figure_items::figure_item` object constructed from `figure_items::close_figure()` to the end of the path.

```
void matrix(const matrix_2d& m) noexcept;
```

- 5 *Requires:* The matrix `m` shall be invertible.

- 6 *Effects:* Adds a `figure_items::figure_item` object constructed from `(figure_items::abs_matrix(m))` to the end of the path.

```
void rel_matrix(const matrix_2d& m) noexcept;
```

- 7 *Requires:* The matrix `m` shall be invertible.

- 8 *Effects:* Adds a `figure_items::figure_item` object constructed from `(figure_items::rel_matrix(m))` to the end of the path.

```
void revert_matrix() noexcept;
```

- 9 *Effects:* Adds a `figure_items::figure_item` object constructed from `(figure_items::revert_matrix())` to the end of the path.

```
void line(point_2d pt) noexcept;
```

- 10 Adds a `figure_items::figure_item` object constructed from `figure_items::abs_line(pt)` to the end of the path.

```
void rel_line(point_2d dpt) noexcept;
```

- 11 *Effects:* Adds a `figure_items::figure_item` object constructed from `figure_items::rel_line(pt)` to the end of the path.

```
void quadratic_curve(point_2d pt0, point_2d pt1) noexcept;
```

- 12 *Effects:* Adds a `figure_items::figure_item` object constructed from `figure_items::abs_quadratic_curve(pt0, pt1)` to the end of the path.

```
void rel_quadratic_curve(point_2d dpt0, point_2d dpt1)
noexcept;
```

- 13 *Effects:* Adds a `figure_items::figure_item` object constructed from `figure_items::rel_quadratic_curve(dpt0, dpt1)` to the end of the path.

```
void cubic_curve(point_2d pt0, point_2d pt1,
point_2d pt2) noexcept;
```

- 14 *Effects:* Adds a `figure_items::figure_item` object constructed from `figure_items::abs_cubic_curve(pt0, pt1, pt2)` to the end of the path.

```
void rel_cubic_curve(point_2d dpt0, point_2d dpt1,
    point_2d dpt2) noexcept;
```

- 16 *Effects:* Adds a `figure_items::figure_item` object constructed from `figure_items::rel_cubic_curve(dpt0, dpt1, dpt2)` to the end of the path.

```
void arc(point_2d rad, float rot, float sang) noexcept;
```

- 17 *Effects:* Adds a `figure_items::figure_item` object constructed from `figure_items::arc(rad, rot, sang)` to the end of the path.

```
iterator insert(const_iterator position, const value_type& x);
iterator insert(const_iterator position, value_type&& x);
iterator insert(const_iterator position, size_type n, const value_type& x);
template <class InputIterator>
iterator insert(const_iterator position, InputIterator first,
    InputIterator last);
iterator insert(const_iterator position, initializer_list<value_type>);
template <class... Args>
reference emplace_back(Args&&... args);
template <class... Args>
iterator emplace(const_iterator position, Args&&... args);
void push_back(const value_type& x);
void push_back(value_type&& x);
```

- 18 *Remarks:* Causes reallocation if the new size is greater than the old capacity. Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence. If no reallocation happens, all the iterators and references before the insertion point remain valid. If an exception is thrown other than by the copy constructor, move constructor, assignment operator, or move assignment operator of `value_type` or by any `InputIterator` operation there are no effects. If an exception is thrown while inserting a single element at the end and `value_type` is `CopyInsertable` or `is_nothrow_move_constructible_v<value_type>` is `true`, there are no effects. Otherwise, if an exception is thrown by the move constructor of a non-`CopyInsertable` `value_type`, the effects are unspecified.
- 19 *Complexity:* The complexity is linear in the number of elements inserted plus the distance to the end of the path builder.

```
iterator erase(const_iterator position);
iterator erase(const_iterator first, const_iterator last);
void pop_back();
```

- 20 *Effects:* Invalidates iterators and references at or after the point of the erase.
- 21 *Complexity:* The destructor of `value_type` is called the number of times equal to the number of the elements erased, but the assignment operator of `value_type` is called the number of times equal to the number of elements in the path builder after the erased elements.
- 22 *Throws:* Nothing unless an exception is thrown by the copy constructor, move constructor, assignment operator, or move assignment operator of `value_type`.

11.5.6 basic_path_builder iterators

[io2d.pathbuilder.iterators]

```
iterator begin() noexcept;
const_iterator begin() const noexcept;
const_iterator cbegin() const noexcept;
```

- 1 *Returns:* An iterator referring to the first `figure_items::figure_item` item in the path.
- 2 *Remarks:* Changing a `figure_items::figure_item` object or otherwise modifying the path in a

way that violates the preconditions of that `figure_items::figure_item` object or of any subsequent `figure_items::figure_item` object in the path produces undefined behavior when the path is interpreted as described in 11.3.16 unless all of the violations are fixed prior to such interpretation.

```
iterator end() noexcept;
const_iterator end() const noexcept;
const_iterator cend() const noexcept;
```

3 *Returns:* An iterator which is the past-the-end value.

4 *Remarks:* Changing a `figure_items::figure_item` object or otherwise modifying the path in a way that violates the preconditions of that `figure_items::figure_item` object or of any subsequent `figure_items::figure_item` object in the path produces undefined behavior when the path is interpreted as described in 11.3.16 unless all of the violations are fixed prior to such interpretation.

```
reverse_iterator rbegin() noexcept;
const_reverse_iterator rbegin() const noexcept;
const_reverse_iterator crbegin() const noexcept;
```

5 *Returns:* An iterator which is semantically equivalent to `reverse_iterator(end)`.

6 *Remarks:* Changing a `figure_items::figure_item` object or otherwise modifying the path in a way that violates the preconditions of that `figure_items::figure_item` object or of any subsequent `figure_items::figure_item` object in the path produces undefined behavior when the path is interpreted as described in 11.3.16 all of the violations are fixed prior to such interpretation.

```
reverse_iterator rend() noexcept;
const_reverse_iterator rend() const noexcept;
const_reverse_iterator crend() const noexcept;
```

7 *Returns:* An iterator which is semantically equivalent to `reverse_iterator(begin)`.

8 *Remarks:* Changing a `figure_items::figure_item` object or otherwise modifying the path in a way that violates the preconditions of that `figure_items::figure_item` object or of any subsequent `figure_items::figure_item` object in the path produces undefined behavior when the path is interpreted as described in 11.3.16 unless all of the violations are fixed prior to such interpretation.

11.5.7 `basic_path_builder` specialized algorithms [io2d.pathbuilder.special]

```
template <class Allocator>
void swap(basic_path_builder<Allocator>& lhs, basic_path_builder<Allocator>& rhs)
    noexcept(noexcept(lhs.swap(rhs)));
```

1 *Effects:* As if by `lhs.swap(rhs)`.

12 Brushes

[io2d.brushes]

12.1 Overview of brushes

[io2d.brushes.intro]

- 1 Brushes contain visual data and serve as sources of visual data for rendering and composing operations.
- 2 There are four types of brushes:
 - (2.1) — solid color;
 - (2.2) — linear gradient;
 - (2.3) — radial gradient; and,
 - (2.4) — surface.
- 3 Once a brush is created, its visual data is immutable.
- 4 [*Note*: While copy and move operations along with a swap operation can change the visual data that a brush contains, the visual data itself is not modified. — *end note*]
- 5 A brush is used either as a *source brush* or a *mask brush* (13.15.2.2).
- 6 When a brush is used in a rendering and composing operation, if it is used as a source brush, it has a **brush_props** object that describes how the brush is interpreted for purposes of sampling. If it is used as a mask brush, it has a **mask_props** object that describes how the brush is interpreted for purposes of sampling.
- 7 The **basic_brush_props** (13.11.1) and **basic_mask_props** (13.14.1) classes each have a *wrap mode* and a *filter*. The **basic_brush_props** class also has a *brush matrix* and a *fill rule*. The **basic_mask_props** class also has a *mask matrix*. Where possible, the terms that are common between the two classes are referenced without regard to whether the brush is being used as a source brush or a mask brush.
- 8 Solid color brushes are unbounded and as such always produce the same visual data when sampled from, regardless of the requested point.
- 9 Linear gradient and radial gradient brushes share similarities with each other that are not shared by the other types of brushes. This is discussed in more detail elsewhere (12.2).
- 10 Surface brushes are constructed from a **basic_image_surface** object. Their visual data is a pixmap, which has implications on sampling from the brush that are not present in the other brush types.

12.2 Gradient brushes

[io2d.gradients]

12.2.1 Common properties of gradients

[io2d.gradients.common]

- 1 Gradients are formed, in part, from a collection of **gradient_stop** objects.
- 2 The collection of **gradient_stop** objects contribute to defining a brush which, when sampled from, returns a value that is interpolated based on those gradient stops.

12.2.2 Linear gradients

[io2d.gradients.linear]

- 1 A linear gradient is a type of gradient.
- 2 A linear gradient has a *begin point* and an *end point*, each of which are objects of type **basic_point_2d**.
- 3 A linear gradient for which the distance between its begin point and its end point is <TODO>**basic_point_2d::zero()** is a *degenerate linear gradient*.
- 4 All attempts to sample from a a degenerate linear gradient return the color **rgba_color::transparent_black**. The remainder of 12.2 is inapplicable to degenerate linear gradients. [*Note*: Because a point has no

width and this case is only met when the distance is between the begin point and the end point is zero (such that it collapses to a single point), the existence of one or more gradient stops is irrelevant. A linear gradient requires a line segment to define its color(s). Without a line segment, it is not a linear gradient. — *end note*]

- 5 The begin point and end point of a linear gradient define a line segment, with a gradient stop offset value of 0.0f corresponding to the begin point and a gradient stop offset value of 1.0f corresponding to the end point.
- 6 Gradient stop offset values in the range [0.0f, 1.0f] linearly correspond to points on the line segment.
- 7 [*Example:* Given a linear gradient with a begin point of <TODO>`basic_point_2d(0.0f, 0.0f)` and an end point of <TODO>`basic_point_2d(10.0f, 5.0f)`, a gradient stop offset value of 0.6f would correspond to the point <TODO>`basic_point_2d(6.0f, 3.0f)`. — *end example*]
- 8 To determine the offset value of a point p for a linear gradient, perform the following steps:
 - a) Create a line at the begin point of the linear gradient, the *begin line*, and another line at the end point of the linear gradient, the *end line*, with each line being perpendicular to the *gradient line segment*, which is the line segment delineated by the begin point and the end point.
 - b) Using the begin line, p , and the end line, create a line, the p line, which is parallel to the gradient line segment.
 - c) Defining dp as the distance between p and the point where the p line intersects the begin line and dt as the distance between the point where the p line intersects the begin line and the point where the p line intersects the end line, the offset value of p is $dp \div dt$.
 - d) The offset value shall be negative if
 - (8.1) — p is not on the line segment delineated by the point where the p line intersects the begin line and the point where the p line intersects the end line; and,
 - (8.2) — the distance between p and the point where the p line intersects the begin line is less than the distance between p and the point where the p line intersects the end line.

12.2.3 Radial gradients

[io2d.gradients.radial]

- 1 A radial gradient is a type of gradient.
- 2 A radial gradient has a *start circle* and an *end circle*, each of which is defined by a `basic_circle` object.
- 3 A radial gradient is a *degenerate radial gradient* if:
 - (3.1) — its start circle has a negative radius; or,
 - (3.2) — its end circle has a negative radius; or,
 - (3.3) — the distance between the center point of its start circle and the center point of its end circle is <TODO>`basic_point_2d::zero()`; or,
 - (3.4) — its start circle has a radius of 0.0f and its end circle has a radius of 0.0f.
- 4 All attempts to sample from a `brush` object created using a degenerate radial gradient return the color `rgba_color::transparent_black`. The remainder of 12.2 is inapplicable to degenerate radial gradients.
- 5 A gradient stop offset of 0.0f corresponds to all points along the diameter of the start circle or to its center point if it has a radius value of 0.0f.
- 6 A gradient stop offset of 1.0f corresponds to all points along the diameter of the end circle or to its center point if it has a radius value of 0.0f.
- 7 A radial gradient shall be rendered as a continuous series of interpolated circles defined by the following equations:
 - a) $x(o) = x_{start} + o \times (x_{end} - x_{start})$
 - b) $y(o) = y_{start} + o \times (y_{end} - y_{start})$

$$c) \text{ radius}(o) = \text{radius}_{start} + o \times (\text{radius}_{end} - \text{radius}_{start})$$

where o is a gradient stop offset value.

- 8 The range of potential values for o shall be determined by the *wrap mode* (12.1):
- (8.1) — For `wrap_mode::none`, the range of potential values for o is $[0, 1]$.
- (8.2) — For all other `wrap_mode` values, the range of potential values for o is $[\text{numeric_limits}\langle\text{float}\rangle::\text{lowest}(), \text{numeric_limits}\langle\text{float}\rangle::\text{max}()]$.
- 9 The interpolated circles shall be rendered starting from the smallest potential value of o .
- 10 An interpolated circle shall not be rendered if its value for o results in $\text{radius}(o)$ evaluating to a negative value.

12.2.4 Sampling from gradients

[io2d.gradients.sampling]

- 1 For any offset value o , its color value shall be determined according to the following rules:
- If there are less than two gradient stops or if all gradient stops have the same offset value, then the color value of every offset value shall be `rgba_color::transparent_black` and the remainder of these rules are inapplicable.
 - If exactly one gradient stop has an offset value equal to o , o 's color value shall be the color value of that gradient stop and the remainder of these rules are inapplicable.
 - If two or more gradient stops have an offset value equal to o , o 's color value shall be the color value of the gradient stop which has the lowest index value among the set of gradient stops that have an offset value equal to o and the remainder of 12.2.4 is inapplicable.
 - When no gradient stop has the offset value of `0.0f`, then, defining n to be the offset value that is nearest to `0.0f` among the offset values in the set of all gradient stops, if o is in the offset range $[0, n)$, o 's color value shall be `rgba_color::transparent_black` and the remainder of these rules are inapplicable. [*Note:* Since the range described does not include n , it does not matter how many gradient stops have n as their offset value for purposes of this rule. — *end note*]
 - When no gradient stop has the offset value of `1.0f`, then, defining n to be the offset value that is nearest to `1.0f` among the offset values in the set of all gradient stops, if o is in the offset range $(n, 1]$, o 's color value shall be `rgba_color::transparent_black` and the remainder of these rules are inapplicable. [*Note:* Since the range described does not include n , it does not matter how many gradient stops have n as their offset value for purposes of this rule. — *end note*]
 - Each gradient stop has, at most, two adjacent gradient stops: one to its left and one to its right.
 - Adjacency of gradient stops is initially determined by offset values. If two or more gradient stops have the same offset value then index values are used to determine adjacency as described below.
 - For each gradient stop a , the *set of gradient stops to its left* are those gradient stops which have an offset value which is closer to `0.0f` than a 's offset value. [*Note:* This includes any gradient stops with an offset value of `0.0f` provided that a 's offset value is not `0.0f`. — *end note*]
 - For each gradient stop b , the *set of gradient stops to its right* are those gradient stops which have an offset value which is closer to `1.0f` than b 's offset value. [*Note:* This includes any gradient stops with an offset value of `1.0f` provided that b 's offset value is not `1.0f`. — *end note*]
 - A gradient stop which has an offset value of `0.0f` does not have an adjacent gradient stop to its left.
 - A gradient stop which has an offset value of `1.0f` does not have an adjacent gradient stop to its right.
 - If a gradient stop a 's set of gradient stops to its left consists of exactly one gradient stop, that gradient stop is the gradient stop that is adjacent to a on its left.
 - If a gradient stop b 's set of gradient stops to its right consists of exactly one gradient stop, that gradient

stop is the gradient stop that is adjacent to b on its right.

- n) If two or more gradient stops have the same offset value then the gradient stop with the lowest index value is the only gradient stop from that set of gradient stops which can have a gradient stop that is adjacent to it on its left and the gradient stop with the highest index value is the only gradient stop from that set of gradient stops which can have a gradient stop that is adjacent to it on its right. This rule takes precedence over all of the remaining rules.
- o) If a gradient stop can have an adjacent gradient stop to its left, then the gradient stop which is adjacent to it to its left is the gradient stop from the set of gradient stops to its left which has an offset value which is closest to its offset value. If two or more gradient stops meet that criteria, then the gradient stop which is adjacent to it to its left is the gradient stop which has the highest index value from the set of gradient stops to its left which are tied for being closest to its offset value.
- p) If a gradient stop can have an adjacent gradient stop to its right, then the gradient stop which is adjacent to it to its right is the gradient stop from the set of gradient stops to its right which has an offset value which is closest to its offset value. If two or more gradient stops meet that criteria, then the gradient stop which is adjacent to it to its right is the gradient stop which has the lowest index value from the set of gradient stops to its right which are tied for being closest to its offset value.
- q) Where the value of o is in the range $[0, 1]$, its color value shall be determined by interpolating between the gradient stop, r , which is the gradient stop whose offset value is closest to o without being less than o and which can have an adjacent gradient stop to its left, and the gradient stop that is adjacent to r on r 's left. The acceptable forms of interpolating between color values is set forth later in this section.
- r) Where the value of o is outside the range $[0, 1]$, its color value depends on the value of wrap mode:

(1.1) — If wrap mode is `wrap_mode::none`, the color value of o shall be `rgba_color::transparent_black`.

(1.2) — If wrap mode is `wrap_mode::pad`, if o is negative then the color value of o shall be the same as-if the value of o was `0.0f`, otherwise the color value of o shall be the same as-if the value of o was `1.0f`.

(1.3) — If wrap mode is `wrap_mode::repeat`, then `1.0f` shall be added to or subtracted from o until o is in the range $[0, 1]$, at which point its color value is the color value for the modified value of o as determined by these rules. [*Example:* Given $o == 2.1$, after application of this rule $o == 0.1$ and the color value of o shall be the same value as-if the initial value of o was `0.1`.

Given $o == -0.3$, after application of this rule $o == 0.7$ and the color value of o shall be the same as-if the initial value of o was `0.7`. — *end example*]

(1.4) — If wrap mode is `wrap_mode::reflect`, o shall be set to the absolute value of o , then `2.0f` shall be subtracted from o until o is in the range $[0, 2]$, then if o is in the range $(1, 2]$ then o shall be set to `1.0f - (o - 1.0f)`, at which point its color value is the color value for the modified value of o as determined by these rules. [*Example:* Given $o == 2.8$, after application of this rule $o == 0.8$ and the color value of o shall be the same value as-if the initial value of o was `0.8`.

Given $o == 3.6$, after application of this rule $o == 0.4$ and the color value of o shall be the same value as-if the initial value of o was `0.4`.

Given $o == -0.3$, after application of this rule $o == 0.3$ and the color value of o shall be the same as-if the initial value of o was `0.3`.

Given $o == -5.8$, after application of this rule $o == 0.2$ and the color value of o shall be the same as-if the initial value of o was `0.2`. — *end example*]

² Interpolation between the color values of two adjacent gradient stops is performed linearly on each color channel.

12.3 Enum class `wrap_mode`

[io2d.wrapmode]

12.3.1 `wrap_mode` summary

[io2d.wrapmode.summary]

- ¹ The `wrap_mode` enum class describes how a point's visual data is determined if it is outside the bounds of the *source brush* (13.15.2.2) when sampling.
- ² Depending on the source brush's `filter` value, the visual data of several points may be required to determine the appropriate visual data value for the point that is being sampled. In this case, each point is sampled according to the source brush's `wrap_mode` value with two exceptions:
 1. If the point to be sampled is within the bounds of the source brush and the source brush's `wrap_mode` value is `wrap_mode::none`, then if the source brush's `filter` value requires that one or more points which are outside of the bounds of the source brush be sampled, each of those points is sampled as-if the source brush's `wrap_mode` value is `wrap_mode::pad` rather than `wrap_mode::none`.
 2. If the point to be sampled is within the bounds of the source brush and the source brush's `wrap_mode` value is `wrap_mode::none`, then if the source brush's `filter` value requires that one or more points which are inside of the bounds of the source brush be sampled, each of those points is sampled such that the visual data that is returned is the equivalent of `rgba_color::transparent_black`.
- ³ If a point to be sampled does not have a defined visual data element and the search for the nearest point with defined visual data produces two or more points with defined visual data that are equidistant from the point to be sampled, the returned visual data shall be an unspecified value which is the visual data of one of those equidistant points. Where possible, implementations should choose the among the equidistant points that have an *x* axisvalue and a *y* axisvalue that is nearest to 0.0f.
- ⁴ See Table 5 for the meaning of each `wrap_mode` enumerator.

12.3.2 `wrap_mode` synopsis

[io2d.wrapmode.synopsis]

```
namespace std::experimental::io2d::v1 {
    enum class wrap_mode {
        none,
        repeat,
        reflect,
        pad
    };
}
```

12.3.3 `wrap_mode` enumerators

[io2d.wrapmode.enumerators]

Table 5 — `wrap_mode` enumerator meanings

Enumerator	Meaning
<code>none</code>	If the point to be sampled is outside of the bounds of the source brush, the visual data that is returned is the equivalent of <code>rgba_color::transparent_black</code> .
<code>repeat</code>	If the point to be sampled is outside of the bounds of the source brush, the visual data that is returned is the visual data that would have been returned if the source brush was infinitely large and repeated itself in a left-to-right-left-to-right and top-to-bottom-top-to-bottom fashion.

Table 5 — `wrap_mode` enumerator meanings (continued)

Enumerator	Meaning
<code>reflect</code>	If the point to be sampled is outside of the bounds of the source brush, the visual data that is returned is the visual data that would have been returned if the source brush was infinitely large and repeated itself in a left-to-right-to-left-to-right and top-to-bottom-to-top-to-bottom fashion.
<code>pad</code>	If the point to be sampled is outside of the bounds of the source brush, the visual data that is returned is the visual data that would have been returned for the nearest defined point that is inside the bounds of the source brush.

12.4 Enum class filter

[io2d.filter]

12.4.1 filter summary

[io2d.filter.summary]

- ¹ The `filter` enum class specifies the type of filter to use when sampling from a pixmap.
- ² Three of the `filter` enumerators, `filter::fast`, `filter::good`, and `filter::best`, specify desired characteristics of the filter, leaving the choice of a specific filter to the implementation.
The other two, `filter::nearest` and `filter::bilinear`, each specify a particular filter that shall be used.
- ³ [*Note*: The only type of brush that has a pixmap as its underlying graphics data graphics resource is a brush with a brush type of `brush_type::surface`. — *end note*]
- ⁴ See Table 6 for the meaning of each `filter` enumerator.

12.4.2 filter synopsis

[io2d.filter.synopsis]

```
namespace std::experimental::io2d::v1 {
    enum class filter {
        fast,
        good,
        best,
        nearest,
        bilinear
    };
}
```

12.4.3 filter enumerators

[io2d.filter.enumerators]

Table 6 — `filter` enumerator meanings

Enumerator	Meaning
<code>fast</code>	The filter that corresponds to this value is implementation-defined. The implementation shall ensure that the time complexity of the chosen filter is not greater than the time complexity of the filter that corresponds to <code>filter::good</code> . [<i>Note</i> : By choosing this value, the user is hinting that performance is more important than quality. — <i>end note</i>]

Table 6 — `filter` enumerator meanings (continued)

Enumerator	Meaning
<code>good</code>	The filter that corresponds to this value is implementation-defined. The implementation shall ensure that the time complexity of the chosen formula is not greater than the time complexity of the formula for <code>filter::best</code> . [<i>Note</i> : By choosing this value, the user is hinting that quality and performance are equally important. — <i>end note</i>]
<code>best</code>	The filter that corresponds to this value is implementation-defined. [<i>Note</i> : By choosing this value, the user is hinting that quality is more important than performance. — <i>end note</i>]
<code>nearest</code>	Nearest-neighbor interpolation filtering
<code>bilinear</code>	Bilinear interpolation filtering

12.5 Enum class `brush_type`

[io2d.brushtype]

12.5.1 `brush_type` summary

[io2d.brushtype.summary]

- ¹ The `brush_type` enum class denotes the type of a `brush` object.
- ² See Table 7 for the meaning of each `brush_type` enumerator.

12.5.2 `brush_type` synopsis

[io2d.brushtype.synopsis]

```
namespace std::experimental::io2d::v1 {
    enum class brush_type {
        solid_color,
        surface,
        linear,
        radial
    };
}
```

12.5.3 `brush_type` enumerators

[io2d.brushtype.enumerators]

Table 7 — `brush_type` enumerator meanings

Enumerator	Meaning
<code>solid_color</code>	The brush object is a solid color brush.
<code>surface</code>	The brush object is a surface brush.
<code>linear</code>	The brush object is a linear gradient brush.
<code>radial</code>	The brush object is a radial gradient brush.

12.6 Class `gradient_stop`

[io2d.gradientstop]

12.6.1 Overview

[io2d.gradientstop.intro]

- ¹ The class `gradient_stop` describes a gradient stop that is used by gradient brushes.
- ² It has an *offset* of type `float` and an *offset color* of type `rgba_color`.

12.6.2 `gradient_stop` synopsis

[io2d.gradientstop.synopsis]

```

namespace std::experimental::io2d::v1 {
    class gradient_stop {
    public:
        // 12.6.3, construct:
        constexpr gradient_stop() noexcept;
        constexpr gradient_stop(float o, rgba_color c) noexcept;<TODO><Implementation mismatch>

        // 12.6.4, modifiers:
        constexpr void offset(float o) noexcept;
        constexpr void color(rgba_color c) noexcept;

        // 12.6.5, observers:
        constexpr float offset() const noexcept;
        constexpr rgba_color color() const noexcept;
    };
    // 12.6.6, operators:
    constexpr bool operator==(const gradient_stop& lhs, const gradient_stop& rhs)
        noexcept;
    constexpr bool operator!=(const gradient_stop& lhs, const gradient_stop& rhs)
        noexcept;
}

```

12.6.3 gradient_stop constructors

[io2d.gradientstop.cons]

```
constexpr gradient_stop() noexcept;
```

1 *Effects:* Equivalent to: `gradient_stop(0.0f, rgba_color::transparent_black)`.

```
constexpr gradient_stop(float o, rgba_color c) noexcept;
```

2 *Requires:* `o >= 0.0f` and `o <= 1.0f`.

3 *Effects:* Constructs a `gradient_stop` object.

4 The offset is `o` rounded to the nearest multiple of `0.00001f`. The offset color is `c`.

12.6.4 gradient_stop modifiers

[io2d.gradientstop.modifiers]

```
constexpr void offset(float o) noexcept;
```

1 *Requires:* `o >= 0.0f` and `o <= 1.0f`.

2 *Effects:* The offset is `o` rounded to the nearest multiple of `0.00001f`.

```
constexpr void color(rgba_color c) noexcept;
```

3 *Effects:* The offset color is `c`.

12.6.5 gradient_stop observers

[io2d.gradientstop.observers]

```
constexpr float offset() const noexcept;
```

1 *Returns:* The offset.

```
constexpr rgba_color color() const noexcept;
```

2 *Returns:* The offset color.

12.6.6 gradient_stop operators

[io2d.gradientstop.ops]

```
constexpr bool operator==(const gradient_stop& lhs, const gradient_stop& rhs)
    noexcept;
```

¹ *Returns:* lhs.offset() == rhs.offset() && lhs.color() == rhs.color();

12.7 Class basic_brush

[io2d.brush]

12.7.1 basic_brush summary

[io2d.brush.intro]

- ¹ The class `basic_brush` describes an opaque wrapper for graphics data.
- ² A `basic_brush` object is usable with any `surface` or `surface`-derived object.<TODO>
- ³ A `basic_brush` object's graphics data is immutable. It is observable only by the effect that it produces when the brush is used as a *source brush* or as a *mask brush* (13.15.2.2).
- ⁴ A `basic_brush` object has a brush type of `brush_type`, which indicates which type of brush it is (Table 7).
- ⁵ As a result of technological limitations and considerations, a `basic_brush` object's graphics data may have less precision than the data from which it was created.

12.7.2 basic_brush synopsis

[io2d.brush.synopsis]

```
namespace std::experimental::io2d::v1 {
template <class GraphicsSurfaces>
class basic_brush {
public:
    using graphics_math_type = typename GraphicsSurfaces::graphics_math_type;
    explicit basic_brush(const rgba_color& c);

    template <class InputIterator>
    basic_brush(const basic_point_2d<graphics_math_type>& begin,
               const basic_point_2d<graphics_math_type>& end,
               InputIterator first, InputIterator last);

    basic_brush(const basic_point_2d<graphics_math_type>& begin,
               const basic_point_2d<graphics_math_type>& end,
               ::std::initializer_list<gradient_stop> il);

    template <class InputIterator>
    basic_brush(const basic_circle<graphics_math_type>& start,
               const basic_circle<graphics_math_type>& end,
               InputIterator first, InputIterator last);

    basic_brush(const basic_circle<graphics_math_type>& start,
               const basic_circle<graphics_math_type>& end,
               ::std::initializer_list<gradient_stop> il);

    basic_brush(basic_image_surface<GraphicsSurfaces>&& img);

    brush_type type() const noexcept;
};
}
```

12.7.3 Sampling from a basic_brush object

[io2d.brush.sampling]

- ¹ A `basic_brush` object is sampled from either as a source brush (13.15.2.2) or a mask brush (13.15.2.2).
- ² If it is being sampled from as a source brush, its *wrap mode*, *filter*, and *brush matrix* are defined by a `basic_brush_props` object (13.15.2.4 and 13.15.2.6).
- ³ If it is being sampled from as a mask brush, its *wrap mode*, *filter*, and *mask matrix* are defined by a `basic_mask_props` object (13.15.2.5 and 13.15.2.6).

- ⁴ When sampling from a `basic_brush` object `b`, the `brush_type` returned by calling `b.type()` determines how the results of sampling are determined:

1. If the result of `b.type()` is `brush_type::solid_color` then `b` is a *solid color brush*.
2. If the result of `b.type()` is `brush_type::surface` then `b` is a *surface brush*.
3. If the result of `b.type()` is `brush_type::linear` then `b` is a *linear gradient brush*.
4. If the result of `b.type()` is `brush_type::radial` then `b` is a *radial gradient brush*.

12.7.3.1 Sampling from a solid color brush [io2d.brush.sampling.color]

- ¹ When `b` is a solid color brush, then when sampling from `b`, the visual data returned is always the visual data used to construct `b`, regardless of the point which is to be sampled and regardless of the return values of wrap mode, filter, and brush matrix or mask matrix.

12.7.3.2 Sampling from a linear gradient brush [io2d.brush.sampling.linear]

- ¹ When `b` is a linear gradient brush, when sampling point `pt`, where `pt` is the return value of calling the `transform_pt` member function of brush matrix or mask matrix using the requested point, from `b`, the visual data returned are as specified by 12.2.2 and 12.2.4.

12.7.3.3 Sampling from a radial gradient brush [io2d.brush.sampling.radial]

- ¹ When `b` is a radial gradient brush, when sampling point `pt`, where `pt` is the return value of calling the `transform_pt` member function of brush matrix or mask matrix using the requested point, from `b`, the visual data are as specified by 12.2.3 and 12.2.4.

12.7.3.4 Sampling from a surface brush [io2d.brush.sampling.surface]

- ¹ When `b` is a surface brush, when sampling point `pt` from `b`, where `pt` is the return value of calling the `transform_pt` member function of the brush matrix or mask matrix using the requested point, the visual data returned are from the point `pt` in the graphics data of the brush, as modified by the values of wrap mode (12.3) and filter (12.4).

12.7.4 `basic_brush` constructors and assignment operators [io2d.brush.cons]

```
explicit basic_brush(const rgba_color& c);<TODO><Implementation mismatch>
```

- ¹ *Effects:* Constructs an object of type `basic_brush`.
- ² The brush's brush type shall be set to the value `brush_type::solid_color`.
- ³ The graphics data of the brush are created from the value of `c`. The visual data format of the graphics data are as-if it is that specified by `format::argb32`.
- ⁴ *Remarks:* Sampling from this produces the results specified in 12.7.3.1.

```
template <class InputIterator>
basic_brush(const basic_point_2d<graphics_math_type>& begin,
            const basic_point_2d<graphics_math_type>& end,
            InputIterator first, InputIterator last);
```

- ⁵ *Effects:* Constructs a linear gradient `basic_brush` object with a begin point of `begin`, an end point of `end`, and a sequential series of `gradient_stop` values beginning at `first` and ending at `last - 1`.
- ⁶ The brush's brush type is `brush_type::linear`.
- ⁷ *Remarks:* Sampling from this brush produces the results specified in 12.7.3.2.

```
basic_brush(const basic_point_2d<graphics_math_type>& begin,
            const basic_point_2d<graphics_math_type>& end,
            ::std::initializer_list<gradient_stop> il);
```


8 *Effects:* Constructs a linear gradient `basic_brush` object with a begin point of `begin`, an end point of `end`, and the sequential series of `gradient_stop` values in `il`.

9 The brush's brush type is `brush_type::linear`.

10 *Remarks:* Sampling from this brush produces the results specified in [12.7.3.2](#).

```
template <class InputIterator>
basic_brush(const basic_circle<graphics_math_type>& start,
            const basic_circle<graphics_math_type>& end,
            InputIterator first, InputIterator last);
```

11 *Effects:* Constructs a radial gradient `basic_brush` object with a start circle of `start`, an end circle of `end`, and a sequential series of `gradient_stop` values beginning at first and ending at last - 1.

12 The brush's brush type is `brush_type::radial`.

13 *Remarks:* Sampling from this brush produces the results specified in [12.7.3.3](#).

```
basic_brush(const basic_circle<graphics_math_type>& start,
            const basic_circle<graphics_math_type>& end,
            ::std::initializer_list<gradient_stop> il);
```

14 *Effects:* Constructs a radial gradient `basic_brush` object with a start circle of `start`, an end circle of `end`, and the sequential series of `gradient_stop` values in `il`.

15 The brush's brush type is `brush_type::radial`.

16 *Remarks:* Sampling from this brush produces the results specified in [12.7.3.3](#).

```
basic_brush(basic_image_surface<GraphicsSurfaces>&& img);
```

17 *Effects:* Constructs an object of type `basic_brush`.

18 The brush's brush type is `brush_type::surface`.

19 The graphics data of the brush is as-if it is the raster graphics data of `img`.

20 *Remarks:* Sampling from this brush produces the results specified in [12.7.3.4](#).

12.7.5 `basic_brush` observers

[io2d.brush.observers]

```
brush_type type() const noexcept;
```

1 *Returns:* The brush's brush type.

13 Surfaces

[io2d.surfaces]

- ¹ Surfaces are composed of visual data, stored in a graphics data graphics resource. [*Note:* <TODO>All well-defined **surface**-derived types are currently raster graphics data graphics resources with defined bounds. To allow for easier additions of future surface-derived types which are not composed of raster graphics data or do not have fixed bounds, such as a vector graphics-based surface, the less constrained term graphics data graphics resource is used. — *end note*]
- ² The surface's visual data is manipulated by rendering and composing operations (13.15.2).
- ³ <TODO>The various **surface**-derived classes each provide specific, unique functionality that enables a broad variety of 2D graphics operations to be accomplished efficiently.

13.1 Enum class antialias

[io2d.antialias]

13.1.1 antialias summary

[io2d.antialias.summary]

- ¹ The antialias enum class specifies the type of anti-aliasing that the rendering system uses for rendering and composing paths. See Table 8 for the meaning of each **antialias** enumerator.

13.1.2 antialias synopsis

[io2d.antialias.synopsis]

```
namespace std::experimental::io2d::v1 {
    enum class antialias {
        none,
        fast,
        good,
        best
    };
}
```

13.1.3 antialias enumerators

[io2d.antialias.enumerators]

Table 8 — **antialias** enumerator meanings

Enumerator	Meaning
fast	No anti-aliasing is performed. Some form of anti-aliasing shall be used when this option is selected, but the form used is implementation-defined. [<i>Note:</i> By specifying this value, the user is hinting that faster anti-aliasing is preferable to better anti-aliasing. — <i>end note</i>]
good	Some form of anti-aliasing shall be used when this option is selected, but the form used is implementation-defined. [<i>Note:</i> By specifying this value, the user is hinting that sacrificing some performance to obtain better anti-aliasing is acceptable but that performance is still a concern. — <i>end note</i>]

Table 8 — **antialias** enumerator meanings (continued)

Enumerator	Meaning
best	Some form of anti-aliasing shall be used when this option is selected, but the form used is implementation-defined. [<i>Note</i> : By specifying this value, the user is hinting that anti-aliasing is more important than performance. — <i>end note</i>]

13.2 Enum class fill_rule

[io2d.fillrule]

13.2.1 fill_rule summary

[io2d.fillrule.summary]

- ¹ The **fill_rule** enum class determines how the filling operation (13.15.5) is performed on a path.
- ² For each point, draw a ray from that point to infinity which does not pass through the start point or end point of any non-degenerate segment in the path, is not tangent to any non-degenerate segment in the path, and is not coincident with any non-degenerate segment in the path.
- ³ See Table 9 for the meaning of each **fill_rule** enumerator.

13.2.2 fill_rule synopsis

[io2d.fillrule.synopsis]

```
namespace std::experimental::io2d::v1 {
    enum class fill_rule {
        winding,
        even_odd
    };
}
```

13.2.3 fill_rule enumerators

[io2d.fillrule.enumerators]

Table 9 — **fill_rule** enumerator meanings

Enumerator	Meaning
winding	If the <i>fill rule</i> (13.11.1) is fill_rule::winding , then using the ray described above and beginning with a count of zero, add one to the count each time a non-degenerate segment crosses the ray going left-to-right from its begin point to its end point, and subtract one each time a non-degenerate segment crosses the ray going from right-to-left from its begin point to its end point. If the resulting count is zero after all non-degenerate segments that cross the ray have been evaluated, the point shall not be filled; otherwise the point shall be filled.
even_odd	If the fill rule is fill_rule::even_odd , then using the ray described above and beginning with a count of zero, add one to the count each time a non-degenerate segment crosses the ray. If the resulting count is an odd number after all non-degenerate segments that cross the ray have been evaluated, the point shall be filled; otherwise the point shall not be filled. [<i>Note</i> : Mathematically, zero is an even number, not an odd number. — <i>end note</i>]

13.3 Enum class `line_cap` [io2d.linecap]

13.3.1 `line_cap` summary [io2d.linecap.summary]

- ¹ The `line_cap` enum class specifies how the ends of lines should be rendered when a `interpreted_path` object is stroked. See Table 10 for the meaning of each `line_cap` enumerator.

13.3.2 `line_cap` synopsis [io2d.linecap.synopsis]

```
namespace std::experimental::io2d::v1 {
    enum class line_cap {
        none,
        round,
        square
    };
}
```

13.3.3 `line_cap` enumerators [io2d.linecap.enumerators]

Table 10 — `line_cap` enumerator meanings

Enumerator	Meaning
<code>none</code>	The line has no cap. It terminates exactly at the end point.
<code>round</code>	The line has a circular cap, with the end point serving as the center of the circle and the line width serving as its diameter.
<code>square</code>	The line has a square cap, with the end point serving as the center of the square and the line width serving as the length of each side.

13.4 Enum class `line_join` [io2d.linejoin]

13.4.1 `line_join` summary [io2d.linejoin.summary]

- ¹ The `line_join` enum class specifies how the junction of two line segments should be rendered when a `interpreted_path` is stroked. See Table 11 for the meaning of each enumerator.

13.4.2 `line_join` synopsis [io2d.linejoin.synopsis]

```
namespace std::experimental::io2d::v1 {
    enum class line_join {
        miter,
        round,
        bevel
    };
}
```

13.4.3 `line_join` enumerators [io2d.linejoin.enumerators]

Table 11 — `line_join` enumerator meanings

Enumerator	Meaning
<code>miter</code>	Joins will be mitered or beveled, depending on the miter limit (see: 13.13.1).
<code>round</code>	Joins will be rounded, with the center of the circle being the join point.

Table 11 — `line_join` enumerator meanings (continued)

Enumerator	Meaning
<code>bevel</code>	Joins will be beveled, with the join cut off at half the line width from the join point. Implementations may vary the cut off distance by an amount that is less than one pixel at each join for aesthetic or technical reasons.

13.5 Enum class `compositing_op`

[io2d.compositingop]

13.5.1 `compositing_op` Summary

[io2d.compositingop.summary]

- ¹ The `compositing_op` enum class specifies composition algorithms. See Table 12, Table 13 and Table 14 for the meaning of each `compositing_op` enumerator.

13.5.2 `compositing_op` Synopsis

[io2d.compositingop.synopsis]

```

namespace std::experimental::io2d::v1 {
    enum class compositing_op {
        // basic
        over,
        clear,
        source,
        in,
        out,
        atop,
        dest,
        dest_over,
        dest_in,
        dest_out,
        dest_atop,
        xor_op,
        add,
        saturate,
        // blend
        multiply,
        screen,
        overlay,
        darken,
        lighten,
        color_dodge,
        color_burn,
        hard_light,
        soft_light,
        difference,
        exclusion,
        // hsl
        hsl_hue,
        hsl_saturation,
        hsl_color,
        hsl_luminosity
    };
}

```

13.5.3 compositing_op Enumerators**[io2d.compositingop.enumerators]**

- ¹ The tables below specifies the mathematical formula for each enumerator's composition algorithm. The formulas differentiate between three color channels (red, green, and blue) and an alpha channel (transparency). For all channels, valid channel values are in the range [0.0, 1.0].
- ² Where a visual data format for a visual data element has no alpha channel, the visual data format shall be treated as though it had an alpha channel with a value of 1.0 for purposes of evaluating the formulas.
- ³ Where a visual data format for a visual data element has no color channels, the visual data format shall be treated as though it had a value of 0.0 for all color channels for purposes of evaluating the formulas.
- ⁴ The following symbols and specifiers are used:
 - The *R* symbol means the result color value
 - The *S* symbol means the source color value
 - The *D* symbol means the destination color value
 - The *c* specifier means the color channels of the value it follows
 - The *a* specifier means the alpha channel of the value it follows
- ⁵ The color symbols *R*, *S*, and *D* may appear with or without any specifiers.
- ⁶ If a color symbol appears alone, it designates the entire color as a tuple in the unsigned normalized form (red, green, blue, alpha).
- ⁷ The specifiers *c* and *a* may appear alone or together after any of the three color symbols.
- ⁸ The presence of the *c* specifier alone means the three color channels of the color as a tuple in the unsigned normalized form (red, green, blue).
- ⁹ The presence of the *a* specifier alone means the alpha channel of the color in unsigned normalized form.
- ¹⁰ The presence of the specifiers together in the form *ca* means the value of the color as a tuple in the unsigned normalized form (red, green, blue, alpha), where the value of each color channel is the product of each color channel and the alpha channel and the value of the alpha channel is the original value of the alpha channel. [*Example*: When it appears in a formula, *Sca* means $((Sc \times Sa), Sa)$, such that, given a source color $Sc = (1.0, 0.5, 0.0)$ and an source alpha $Sa = (0.5)$, the value of *Sca* when specified in one of the formulas would be $Sca = (1.0 \times 0.5, 0.5 \times 0.5, 0.0 \times 0.5, 0.5) = (0.5, 0.25, 0.0, 0.5)$. The same is true for *Dca* and *Rca*. — end example]
- ¹¹ No space is left between a value and its channel specifiers. Channel specifiers will be preceded by exactly one value symbol.
- ¹² When performing an operation that involves evaluating the color channels, each color channel should be evaluated individually to produce its own value.
- ¹³ The basic enumerators specify a value for *bound*. This value may be 'Yes', 'No', or 'N/A'.
- ¹⁴ If the bound value is 'Yes', then the source is treated as though it is also a mask. As such, only areas of the surface where the source would affect the surface are altered. The remaining areas of the surface have the same color value as before the compositing operation.
- ¹⁵ If the bound value is 'No', then every area of the surface that is not affected by the source will become transparent black. In effect, it is as though the source was treated as being the same size as the destination surface with every part of the source that does not already have a color value assigned to it being treated as though it were transparent black. Application of the formula with this precondition results in those areas evaluating to transparent black such that evaluation can be bypassed due to the predetermined outcome.
- ¹⁶ If the bound value is 'N/A', the operation would have the same effect regardless of whether it was treated as 'Yes' or 'No' such that those bound values are not applicable to the operation. A 'N/A' formula when applied to an area where the source does not provide a value will evaluate to the original value of the destination even if the source is treated as having a value there of transparent black. As such the result is the same as-if

the source were treated as being a mask, i.e. 'Yes' and 'No' treatment each produce the same result in areas where the source does not have a value.

- ¹⁷ If a clip is set and the bound value is 'Yes' or 'N/A', then only those areas of the surface that the are within the clip will be affected by the compositing operation.
- ¹⁸ If a clip is set and the bound value is 'No', then only those areas of the surface that the are within the clip will be affected by the compositing operation. Even if no part of the source is within the clip, the operation will still set every area within the clip to transparent black. Areas outside the clip are not modified.

Table 12 — `compositing_op` basic enumerator meanings

Enumerator	Bound	Color	Alpha
<code>clear</code>	Yes	$Rc = 0$	$Ra = 0$
<code>source</code>	Yes	$Rc = Sc$	$Ra = Sa$
<code>over</code>	N/A	$Rc = \frac{(Sca + Dca \times (1 - Sa))}{Ra}$	$Ra = Sa + Da \times (1 - Sa)$
<code>in</code>	No	$Rc = Sc$	$Ra = Sa \times Da$
<code>out</code>	No	$Rc = Sc$	$Ra = Sa \times (1 - Da)$
<code>atop</code>	N/A	$Rc = Sca + Dc \times (1 - Sa)$	$Ra = Da$
<code>dest</code>	N/A	$Rc = Dc$	$Ra = Da$
<code>dest_over</code>	N/A	$Rc = \frac{(Sca \times (1 - Da) + Dca)}{Ra}$	$Ra = (1 - Da) \times Sa + Da$
<code>dest_in</code>	No	$Rc = Dc$	$Ra = Sa \times Da$
<code>dest_out</code>	N/A	$Rc = Dc$	$Ra = (1 - Sa) \times Da$
<code>dest_atop</code>	No	$Rc = Sc \times (1 - Da) + Dca$	$Ra = Sa$
<code>xor_op</code>	N/A	$Rc = \frac{(Sca \times (1 - Da) + Dca \times (1 - Sa))}{Ra}$	$Ra = Sa + Da - 2 \times Sa \times Da$
<code>add</code>	N/A	$Rc = \frac{(Sca + Dca)}{Ra}$	$Ra = \min(1, Sa + Da)$
<code>saturate</code>	N/A	$Rc = \frac{(\min(Sa, 1 - Da) \times Sc + Dca)}{Ra}$	$Ra = \min(1, Sa + Da)$

- ¹⁹ The blend enumerators and hsl enumerators share a common formula for the result color's color channel, with only one part of it changing depending on the enumerator. The result color's color channel value formula is as follows: $Rc = \frac{1}{Ra} \times ((1 - Da) \times Sca + (1 - Sa) \times Dca + Sa \times Da \times f(Sc, Dc))$. The function $f(Sc, Dc)$ is the component of the formula that is enumerator dependent.
- ²⁰ For the blend enumerators, the color channels shall be treated as separable, meaning that the color formula shall be evaluated separately for each color channel: red, green, and blue.
- ²¹ The color formula divides 1 by the result color's alpha channel value. As a result, if the result color's alpha channel is zero then a division by zero would normally occur. Implementations shall not throw an exception nor otherwise produce any observable error condition if the result color's alpha channel is zero. Instead, implementations shall bypass the division by zero and produce the result color (0, 0, 0, 0), i.e. *transparent*.

black, if the result color alpha channel formula evaluates to zero. [*Note*: The simplest way to comply with this requirement is to bypass evaluation of the color channel formula in the event that the result alpha is zero. However, in order to allow implementations the greatest latitude possible, only the result is specified. — *end note*]

- ²² For the enumerators in Table 13 and Table 14 the result color's alpha channel value formula is as follows: $Ra = Sa + Da \times (1 - Sa)$. [*Note*: Since it is the same formula for all enumerators in those tables, the formula is not included in those tables. — *end note*]
- ²³ All of the blend enumerators and hsl enumerators have a bound value of 'N/A'.

Table 13 — `compositing_op` blend enumerator meanings

Enumerator	Color
<code>multiply</code>	$f(Sc, Dc) = Sc \times Dc$
<code>screen</code>	$f(Sc, Dc) = Sc + Dc - Sc \times Dc$
<code>overlay</code>	$if(Dc \leq 0.5f) \{$ $f(Sc, Dc) = 2 \times Sc \times Dc$ $\}$ $else \{$ $f(Sc, Dc) =$ $1 - 2 \times (1 - Sc) \times$ $(1 - Dc)$ $\}$ [<i>Note</i>: The difference between this enumerator and <code>hard_light</code> is that this tests the destination color (<i>Dc</i>) whereas <code>hard_light</code> tests the source color (<i>Sc</i>). — <i>end note</i>]
<code>darken</code>	$f(Sc, Dc) = \min(Sc, Dc)$
<code>lighten</code>	$f(Sc, Dc) = \max(Sc, Dc)$
<code>color_dodge</code>	$if(Dc < 1) \{$ $f(Sc, Dc) = \min(1, \frac{Dc}{(1 - Sc)})$ $\}$ $else \{$ $f(Sc, Dc) = 1 \}$
<code>color_burn</code>	$if(Dc > 0) \{$ $f(Sc, Dc) = 1 - \min(1, \frac{1 - Dc}{Sc})$ $\}$ $else \{$ $f(Sc, Dc) = 0$ $\}$

Table 13 — `compositing_op` blend enumerator meanings (continued)

Enumerator	Color
hard_light	$\text{if } (Sc \leq 0.5f) \{$ $f(Sc, Dc) = 2 \times Sc \times Dc$ $\}$ $\text{else } \{$ $f(Sc, Dc) =$ $1 - 2 \times (1 - Sc) \times$ $(1 - Dc)$ $\}$ [<i>Note:</i> The difference between this enumerator and overlay is that this tests the source color (<i>Sc</i>) whereas overlay tests the destination color (<i>Dc</i>). — <i>end note</i>]
soft_light	$\text{if } (Sc \leq 0.5) \{$ $f(Sc, Dc) =$ $Dc - (1 - 2 \times Sc) \times Dc \times$ $(1 - Dc)$ $\}$ $\text{else } \{$ $f(Sc, Dc) =$ $Dc + (2 \times Sc - 1) \times$ $(g(Dc) - Sc)$ $\}$ $g(Dc)$ is defined as follows: $\text{if } (Dc \leq 0.25) \{$ $g(Dc) =$ $((16 \times Dc - 12) \times Dc +$ $4) \times Dc$ $\}$ $\text{else } \{$ $g(Dc) = \sqrt{Dc}$ $\}$
difference	$f(Sc, Dc) = \text{abs}(Dc - Sc)$
exclusion	$f(Sc, Dc) = Sc + Dc - 2 \times Sc \times Dc$

²⁴ For the `hsl` enumerators, the color channels shall be treated as nonseparable, meaning that the color formula shall be evaluated once, with the colors being passed in as tuples in the form (red, green, blue).

²⁵ The following additional functions are used to define the `hsl` enumerator formulas:

²⁶ $\text{min}(x, y, z) = \text{min}(x, \text{min}(y, z))$

²⁷ $\text{max}(x, y, z) = \text{max}(x, \text{max}(y, z))$

²⁸ $\text{sat}(C) = \text{max}(Cr, Cg, Cb) - \text{min}(Cr, Cg, Cb)$

²⁹ $\text{lum}(C) = Cr \times 0.3 + Cg \times 0.59 + Cb \times 0.11$

³⁰ $\text{clip_color}(C) = \{$
 $L = \text{lum}(C)$

```


$$N = \min(Cr, Cg, Cb)$$


$$X = \max(Cr, Cg, Cb)$$

if ( $N < 0.0$ ) {
   $Cr = L + \frac{((Cr - L) \times L)}{(L - N)}$ 
   $Cg = L + \frac{((Cg - L) \times L)}{(L - N)}$ 
   $Cb = L + \frac{((Cb - L) \times L)}{(L - N)}$ 
}
if ( $X > 1.0$ ) {
   $Cr = L + \frac{((Cr - L) \times (1 - L))}{(X - L)}$ 
   $Cg = L + \frac{((Cg - L) \times (1 - L))}{(X - L)}$ 
   $Cb = L + \frac{((Cb - L) \times (1 - L))}{(X - L)}$ 
}
return  $C$ 
}
31 set_lum( $C, L$ ) = {
   $D = L - lum(C)$ 
   $Cr = Cr + D$ 
   $Cg = Cg + D$ 
   $Cb = Cb + D$ 
  return clip_color( $C$ )
}
32 set_sat( $C, S$ ) = {
   $R = C$ 
  auto& max = ( $Rr > Rg$ ) ? (( $Rr > Rb$ ) ?  $Rr : Rb$ ) : (( $Rg > Rb$ ) ?  $Rg : Rb$ )
  auto& mid = ( $Rr > Rg$ ) ? (( $Rr > Rb$ ) ? (( $Rg > Rb$ ) ?  $Rg : Rb$ ) :  $Rr$ ) : (( $Rg > Rb$ ) ? (( $Rr > Rb$ ) ?  $Rr : Rb$ ) :  $Rg$ )
  auto& min = ( $Rr > Rg$ ) ? (( $Rg > Rb$ ) ?  $Rb : Rg$ ) : (( $Rr > Rb$ ) ?  $Rb : Rr$ )
  if ( $max > min$ ) {
     $mid = \frac{((mid - min) \times S)}{max - min}$ 
     $max = S$ 
  }
  else {
     $mid = 0.0$ 
     $max = 0.0$ 
  }
   $min = 0.0$ 
  return  $R$ 
}
[ Note: In the formula,  $max$ ,  $mid$ , and  $min$  are reference variables which are bound to the highest value, second highest value, and lowest value color channels of the (red, blue, green) tuple  $R$  such that the subsequent operations modify the values of  $R$  directly. — end note ]

```

Table 14 — `compositing_op` hsl enumerator meanings

Enumerator	Color & Alpha
<code>hsl_hue</code>	$f(Sc, Dc) = set_lum(set_sat(Sc, sat(Dc)), lum(Dc))$
<code>hsl_saturation</code>	$(Sc, Dc) = set_lum(set_sat(Dc, sat(Sc)), lum(Dc))$
<code>hsl_color</code>	$f(Sc, Dc) = set_lum(Sc, lum(Dc))$
<code>hsl_luminosity</code>	$f(Sc, Dc) = set_lum(Dc, lum(Sc))$

13.6 Enum class format

[io2d.format]

13.6.1 format summary

[io2d.format.summary]

- ¹ The `format` enum class indicates a visual data format. See Table 15 for the meaning of each `format` enumerator.
- ² Unless otherwise specified, a visual data format shall be an unsigned integral value of the specified bit size in native-endian format.
- ³ A channel value of 0x0 means that there is no contribution from that channel. As the channel value increases towards the maximum unsigned integral value representable by the number of bits of the channel, the contribution from that channel also increases, with the maximum value representing the maximum contribution from that channel. [Example: Given a 5-bit channel representing the color , a value of 0x0 means that the red channel does not contribute any value towards the final color of the pixel. A value of 0x1F means that the red channel makes its maximum contribution to the final color of the pixel.

A — end example]

13.6.2 format synopsis

[io2d.format.synopsis]

```
namespace std::experimental::io2d::v1 {
    enum class format {
        invalid,
        argb32,
        rgb24,
        a8,
        rgb16_565,
        rgb30
    };
}
```

13.6.3 format enumerators

[io2d.format.enumerators]

Table 15 — `format` enumerator meanings

Enumerator	Meaning
<code>invalid</code>	A previously specified <code>format</code> is unsupported by the implementation.
<code>argb32</code>	A 32-bit RGB color model pixel format. The upper 8 bits are an alpha channel, followed by an 8-bit red color channel, then an 8-bit green color channel, and finally an 8-bit blue color channel. The value in each channel is an unsigned normalized integer. This is a premultiplied format.
<code>rgb24</code>	A 32-bit RGB color model pixel format. The upper 8 bits are unused, followed by an 8-bit red color channel, then an 8-bit green color channel, and finally an 8-bit blue color channel.

Table 15 — `format` enumerator meanings (continued)

Enumerator	Meaning
<code>a8</code>	An 8-bit transparency data pixel format. All 8 bits are an alpha channel.
<code>rgb16_565</code>	A 16-bit RGB color model pixel format. The upper 5 bits are a red color channel, followed by a 6-bit green color channel, and finally a 5-bit blue color channel.
<code>rgb30</code>	A 32-bit RGB color model pixel format. The upper 2 bits are unused, followed by a 10-bit red color channel, a 10-bit green color channel, and finally a 10-bit blue color channel. The value in each channel is an unsigned normalized integer.

13.7 Enum class scaling

[io2d.scaling]

13.7.1 scaling summary

[io2d.scaling.summary]

- ¹ The scaling enum class specifies the type of scaling a `display_surface` will use when the size of its *display buffer* (??) differs from the size of its *back buffer* (??).
- ² See Table 16 for the meaning of each `scaling` enumerator.

13.7.2 scaling synopsis

[io2d.scaling.synopsis]

```
namespace std::experimental::io2d::v1 {
    enum class scaling {
        letterbox,
        uniform,
        fill_uniform,
        fill_exact,
        none
    };
}
```

13.7.3 scaling enumerators

[io2d.scaling.enumerators]

- ¹ [Note: In the following table, examples will be given to help explain the meaning of each enumerator. The examples will all use a `display_surface` called `ds`.

The back buffer (??) of `ds` is 640x480 (i.e. it has a width of 640 pixels and a height of 480 pixels), giving it an aspect ratio of 1.3.

The display buffer (??) of `ds` is 1280x720, giving it an aspect ratio of 1.7.

When a rectangle is defined in an example, the coordinate $(x1, y1)$ denotes the top left corner of the rectangle, inclusive, and the coordinate $(x2, y2)$ denotes the bottom right corner of the rectangle, exclusive. As such, a rectangle with $(x1, y1) = (10, 10)$, $(x2, y2) = (20, 20)$ is 10 pixels wide and 10 pixels tall and includes the pixel $(x, y) = (19, 19)$ but does not include the pixels $(x, y) = (20, 19)$ or $(x, y) = (19, 20)$. — end note]

Table 16 — **scaling** enumerator meanings

Enumerator	Meaning
letterbox	Fill the display buffer with the letterbox brush (??) of the display_surface. Uniformly scale the back buffer so that one dimension of it is the same length as the same dimension of the display buffer and the second dimension of it is not longer than the second dimension of the display buffer and transfer the scaled back buffer to the display buffer using sampling such that it is centered in the display buffer. [<i>Example:</i> The display buffer of ds will be filled with the brush object returned by ds.letterbox_brush(); . The back buffer of ds will be scaled so that it is 960x720, thereby retaining its original aspect ratio. The scaled back buffer will be transfered to the display buffer using sampling such that it is in the rectangle $(x1, y1) = (\frac{1280}{2} - \frac{960}{2}, 0) = (160, 0),$ $(x2, y2) = (960 + (\frac{1280}{2} - \frac{960}{2}), 720) = (1120, 720).$ This fulfills all of the conditions. At least one dimension of the scaled back buffer is the same length as the same dimension of the display buffer (both have a height of 720 pixels). The second dimension of the scaled back buffer is not longer than the second dimension of the display buffer (the back buffer's scaled width is 960 pixels, which is not longer than the display buffer's width of 1280 pixels. Lastly, the scaled back buffer is centered in the display buffer (on the <i>x</i> axis there are 160 pixels between each vertical side of the scaled back buffer and the nearest vertical edge of the display buffer and on the <i>y</i> axis there are 0 pixels between each horizontal side of the scaled back buffer and the nearest horizontal edge of the display buffer). — <i>end example</i>]

Table 16 — **scaling** enumerator meanings (continued)

Enumerator	Meaning
uniform	Uniformly scale the back buffer so that one dimension of it is the same length as the same dimension of the display buffer and the second dimension of it is not longer than the second dimension of the display buffer and transfer the scaled back buffer to the display buffer using sampling such that it is centered in the display buffer. [<i>Example:</i> The back buffer of ds will be scaled so that it is 960x720, thereby retaining its original aspect ratio. The scaled back buffer will be transfered to the display buffer using sampling such that it is in the rectangle $(x1, y1) = (\frac{1280}{2} - \frac{960}{2}, 0) = (160, 0),$ $(x2, y2) = (960 + (\frac{1280}{2} - \frac{960}{2}), 720) = (1120, 720).$ This fulfills all of the conditions. At least one dimension of the scaled back buffer is the same length as the same dimension of the display buffer (both have a height of 720 pixels). The second dimension of the scaled back buffer is not longer than the second dimension of the display buffer (the back buffer's scaled width is 960 pixels, which is not longer than the display buffer's width of 1280 pixels. Lastly, the scaled back buffer is centered in the display buffer (on the x axis there are 160 pixels between each vertical side of the scaled back buffer and the nearest vertical edge of the display buffer and on the y axis there are 0 pixels between each horizontal side of the scaled back buffer and the nearest horizontal edge of the display buffer). — <i>end example</i>] [<i>Note:</i> The difference between uniform and letterbox is that uniform does not modify the contents of the display buffer that fall outside of the rectangle into which the scaled back buffer is drawn while letterbox fills those areas with the display_surface object's letterbox brush (see: ??). — <i>end note</i>]

Table 16 — **scaling** enumerator meanings (continued)

Enumerator	Meaning
fill_uniform	Uniformly scale the back buffer so that one dimension of it is the same length as the same dimension of the display buffer and the second dimension of it is not shorter than the second dimension of the display buffer and transfer the scaled back buffer to the display buffer using sampling such that it is centered in the display buffer. [<i>Example:</i> The back buffer of ds will be drawn in the rectangle $(x1, y1) = (0, -120)$, $(x2, y2) = (1280, 840)$. This fulfills all of the conditions. At least one dimension of the scaled back buffer is the same length as the same dimension of the display buffer (both have a width of 1280 pixels). The second dimension of the scaled back buffer is not shorter than the second dimension of the display buffer (the back buffer's scaled height is 840 pixels, which is not shorter than the display buffer's height of 720 pixels). Lastly, the scaled back buffer is centered in the display buffer (on the x axis there are 0 pixels between each vertical side of the rectangle and the nearest vertical edge of the display buffer and on the y axis there are 120 pixels between each horizontal side of the rectangle and the nearest horizontal edge of the display buffer). — <i>end example</i>]
fill_exact	Scale the back buffer so that each dimension of it is the same length as the same dimension of the display buffer and transfer the scaled back buffer to the display buffer using sampling such that its origin is at the origin of the display buffer. [<i>Example:</i> The back buffer will be drawn in the rectangle $(x1, y1) = (0, 0)$, $(x2, y2) = (1280, 720)$. This fulfills all of the conditions. Each dimension of the scaled back buffer is the same length as the same dimension of the display buffer (both have a width of 1280 pixels and a height of 720 pixels) and the origin of the scaled back buffer is at the origin of the display buffer. — <i>end example</i>]
none	Do not perform any scaling. Transfer the back buffer to the display buffer using sampling such that its origin is at the origin of the display buffer. [<i>Example:</i> The back buffer of ds will be drawn in the rectangle $(x1, y1) = (0, 0)$, $(x2, y2) = (640, 480)$ such that no scaling occurs and the origin of the back buffer is at the origin of the display buffer. — <i>end example</i>]

13.8 Enum class refresh_rate**[io2d.refreshrate]****13.8.1 refresh_rate summary****[io2d.refreshrate.summary]**

- ¹ The **refresh_rate** enum class describes when the *draw callback* (Table ??) of a **display_surface** object shall be called. See Table 17 for the meaning of each enumerator.

13.8.2 refresh_rate synopsis**[io2d.refreshrate.synopsis]**

```

namespace std::experimental::io2d::v1 {
    enum class refresh_rate {
        as_needed,
        as_fast_as_possible,
        fixed
    };
}

```

13.8.3 refresh_rate enumerators

[io2d.refreshrate.enumerators]

Table 17 — refresh_rate value meanings

Enumerator	Meaning
as_needed	The draw callback shall be called when the implementation needs to do so. [<i>Note</i> : The intention of this enumerator is that implementations will call the draw callback as little as possible in order to minimize power usage. Users can call <code>display_surface::redraw_required</code> to make the implementation run the draw callback whenever the user requires. — <i>end note</i>]
as_fast_as_possible	The draw callback shall be called as frequently as possible, subject to any limits of the execution environment and the underlying rendering and presentation technologies.
fixed	The draw callback shall be called as frequently as needed to maintain the <i>desired frame rate</i> (Table ??) as closely as possible. If more time has passed between two successive calls to the draw callback than is required, it shall be called <i>excess time</i> and it shall count towards the <i>required time</i> , which is the time that is required to pass after a call to the draw callback before the next successive call to the draw callback shall be made. If the excess time is greater than the required time, implementations shall call the draw callback and then repeatedly subtract the required time from the excess time until the excess time is less than the required time. If the implementation needs to call the draw callback for some other reason, it shall use that call as the new starting point for maintaining the desired frame rate. [<i>Example</i> : Given a desired frame rate of 20.0f, then as per the above, the implementation would call the draw callback at 50 millisecond intervals or as close thereto as possible. If for some reason the excess time is 51 milliseconds, the implementation would call the draw callback, subtract 50 milliseconds from the excess time, and then would wait 49 milliseconds before calling the draw callback again. If only 15 milliseconds have passed since the draw callback was last called and the implementation needs to call the draw callback again, then the implementation shall call the draw callback immediately and proceed to wait 50 milliseconds before calling the draw callback again. — <i>end example</i>]

13.9 Enum class `image_file_format` [io2d.imagefileformat]

13.9.1 `image_file_format` summary [io2d.imagefileformat.summary]

- ¹ The `image_file_format` enum class specifies the data format that an `image_surface` object is constructed from or saved to. This allows data in a format that is required to be supported to be read or written regardless of its extension.
- ² It also has a value that allows implementations to support additional file formats if it recognizes them.

13.9.2 `image_file_format` synopsis [io2d.imagefileformat.synopsis]

```
namespace std::experimental::io2d::v1 {
    enum class image_file_format {
        unknown,
        png,
        jpeg,
        tiff
    };
}
```

13.9.3 `image_file_format` enumerators [io2d.imagefileformat.enumerators]

Table 18 — `imagefileformat` enumerator meanings

Enumerator	Meaning
<code>unknown</code>	The format is unknown because it is not an image file format that is required to be supported. It may be known and supported by the implementation.
<code>png</code>	The PNG format.
<code>jpeg</code>	The JPEG format.
<code>tiff</code>	The TIFF format.

13.10 Class `basic_render_props` [io2d.renderprops]

13.10.1 `basic_render_props` summary [io2d.renderprops.summary]

- ¹ The `basic_render_props` class provides general state information that is applicable to all rendering and compositing operations (13.15.2).
- ² It has an *antialias* of type `antialias` with a default value of `antialias::good`, a *surface matrix* of type `basic_matrix_2d` with a default constructed value, and a *compositing operator* of type `compositing_op` with a default value of `compositing_op::over`.

13.10.2 `basic_render_props` synopsis [io2d.renderprops.synopsis]

```
namespace std::experimental::io2d::v1 {
    template <class GraphicsSurfaces>
    class basic_render_props {
    public:
        using graphics_math_type = typename GraphicsSurfaces::graphics_math_type;

        // 13.10.3, constructors:
        basic_render_props() noexcept;
        explicit basic_render_props(antialias a,
            const basic_matrix_2d<graphics_math_type>& m = basic_matrix_2d<graphics_math_type>{},
            compositing_op co = compositing_op::over) noexcept;
```

```

// 13.10.4, modifiers:
void antialiasing(antialias a) noexcept;
void compositing(compositing_op co) noexcept;
void surface_matrix(const basic_matrix_2d<graphics_math_type>& m) noexcept;

// 13.10.5, observers:
antialias antialiasing() const noexcept;
compositing_op compositing() const noexcept;
basic_matrix_2d<graphics_math_type> surface_matrix() const noexcept;
};
}

```

13.10.3 basic_render_props constructors [io2d.renderprops.cons]

```
basic_render_props() noexcept;
```

¹ *Effects:* Equivalent to: `basic_render_props(antialias::good)`.

```
explicit basic_render_props(antialias a,
    const basic_matrix_2d<graphics_math_type>& m = basic_matrix_2d<graphics_math_type>{},
    compositing_op co = compositing_op::over) noexcept;
```

² *Requires:* `m.is_invertible() == true`.

³ *Effects:* The antialias is a. The surface matrix is m. The compositing operator is co.

13.10.4 basic_render_props modifiers [io2d.renderprops.modifiers]

```
void antialiasing(antialias a) noexcept;
```

¹ *Effects:* The antialias is a.

```
void compositing(compositing_op co) noexcept;
```

² *Effects:* The compositing operator is co.

```
void surface_matrix(const basic_matrix_2d<graphics_math_type>& m) noexcept;
```

³ *Requires:* `m.is_invertible() == true`.

⁴ *Effects:* The surface matrix is m.

13.10.5 basic_render_props observers [io2d.renderprops.observers]

```
antialias antialiasing() const noexcept;
```

¹ *Returns:* The antialias.

```
compositing_op compositing() const noexcept;
```

² *Returns:* The compositing operator.

```
basic_matrix_2d<graphics_math_type> surface_matrix() const noexcept;
```

³ *Returns:* The surface matrix.

13.11 Class basic_brush_props [io2d.brushprops]

13.11.1 basic_brush_props summary [io2d.brushprops.summary]

¹ The `basic_brush_props` class provides general state information that is applicable to all rendering and composing operations (13.15.2).

- ² It has a *wrap mode* of type `wrap_mode`, a *filter* of type `filter`, a *fill rule* of type `fill_rule`, and a *brush matrix* of type `basic_matrix_2d`.

13.11.2 `basic_brush_props` synopsis

[io2d.brushprops.synopsis]

```
namespace std::experimental::io2d::v1 {
    template <class GraphicsSurfaces>
    class basic_brush_props {
    public:
        using graphics_math_type = typename GraphicsSurfaces::graphics_math_type;

        // 13.11.3, constructor:
        basic_brush_props(io2d::wrap_mode w = io2d::wrap_mode::none,
            io2d::filter fi = io2d::filter::good,
            io2d::fill_rule fr = io2d::fill_rule::winding,
            const basic_matrix_2d<graphics_math_type>& m = basic_matrix_2d<graphics_math_type>{})
            noexcept;

        // 13.11.4, modifiers:
        void wrap_mode(io2d::wrap_mode w) noexcept;
        void filter(io2d::filter fi) noexcept;
        void fill_rule(io2d::fill_rule fr) noexcept;
        void brush_matrix(const basic_matrix_2d<graphics_math_type>& m) noexcept;

        // 13.11.5, observers:
        io2d::wrap_mode wrap_mode() const noexcept;
        io2d::filter filter() const noexcept;
        io2d::fill_rule fill_rule() const noexcept;
        basic_matrix_2d<graphics_math_type> brush_matrix() const noexcept;
    };
}
```

13.11.3 `basic_brush_props` constructor

[io2d.brushprops.cons]

```
basic_brush_props(io2d::wrap_mode w = io2d::wrap_mode::none,
    io2d::filter fi = io2d::filter::good,
    io2d::fill_rule fr = io2d::fill_rule::winding,
    const basic_matrix_2d<graphics_math_type>& m = basic_matrix_2d<graphics_math_type>{})
    noexcept;
```

- ¹ *Requires:* `m.is_invertible() == true`.
- ² *Effects:* Constructs an object of type `basic_brush_props`.
- ³ The wrap mode is `w`. The filter is `fi`. The fill rule is `fr`. The brush matrix is `m`.

13.11.4 `basic_brush_props` modifiers

[io2d.brushprops.modifiers]

```
void wrap_mode(io2d::wrap_mode w) noexcept;
```

- ¹ *Effects:* The wrap mode is `w`.

```
void filter(io2d::filter fi) noexcept;
```

- ² *Effects:* The filter is `fi`.

```
void fill_rule(io2d::fill_rule fr) noexcept;
```

- ³ *Effects:* The fill rule is `fr`.

```
void brush_matrix(const basic_matrix_2d<graphics_math_type>& m) noexcept;
```

4 *Requires:* `m.is_invertible() == true`.

5 *Effects:* The brush matrix is `m`.

13.11.5 `basic_brush_props` observers [io2d.brushprops.observers]

`io2d::wrap_mode wrap_mode() const noexcept;`

1 *Returns:* The wrap mode.

`io2d::filter filter() const noexcept;`

2 *Returns:* The filter.

`io2d::fill_rule fill_rule() const noexcept;`

3 *Returns:* The fill rule.

`basic_matrix_2d<graphics_math_type> brush_matrix() const noexcept;`

4 *Returns:* The brush matrix.

13.12 Class `basic_clip_props` [io2d.clipprops]

13.12.1 `basic_clip_props` summary [io2d.clipprops.summary]

1 The `basic_clip_props` class provides general state information that is applicable to all rendering and composing operations ([13.15.2](#)).

2 It has a *clip area* of type `interpreted_path` and a *fill rule* of type `fill_rule`.

13.12.2 `basic_clip_props` synopsis [io2d.clipprops.synopsis]

```
namespace std::experimental::io2d::v1 {
    template <class GraphicsSurfaces>
    class basic_clip_props {
    public:
        using graphics_math_type = typename GraphicsSurfaces::graphics_math_type;

        // 13.12.3, constructors:
        basic_clip_props() noexcept;
        template <class Allocator>
        explicit basic_clip_props(const basic_path_builder<GraphicsSurfaces, Allocator>& pb,
            io2d::fill_rule fr = io2d::fill_rule::winding);
        explicit basic_clip_props(const basic_interpreted_path<GraphicsSurfaces>& ip,
            io2d::fill_rule fr = io2d::fill_rule::winding) noexcept;
        explicit basic_clip_props(const basic_bounding_box<graphics_math_type>& r,
            io2d::fill_rule fr = io2d::fill_rule::winding);

        // 13.12.4, modifiers:
        template <class Allocator>
        void clip(const basic_path_builder<GraphicsSurfaces, Allocator>& pb);
        void clip(const basic_interpreted_path<GraphicsSurfaces>& ip) noexcept;
        void fill_rule(io2d::fill_rule fr) noexcept;

        // 13.12.5, observers:
        basic_interpreted_path<GraphicsSurfaces> clip() const noexcept;
        io2d::fill_rule fill_rule() const noexcept;
    };
}
```

13.12.3 basic_clip_props constructors**[io2d.clipprops.cons]**

```
basic_clip_props() noexcept;
```

¹ *Effects:* Equivalent to: `basic_clip_props(basic_path_builder<>{ })`.

```
template <class Allocator>
explicit basic_clip_props(const basic_path_builder<GraphicsSurfaces, Allocator>& pb,
    io2d::fill_rule fr = io2d::fill_rule::winding);
explicit basic_clip_props(const basic_interpreted_path<GraphicsSurfaces>& ip,
    io2d::fill_rule fr = io2d::fill_rule::winding) noexcept;
explicit basic_clip_props(const basic_bounding_box<graphics_math_type>& r,
    io2d::fill_rule fr = io2d::fill_rule::winding)
```

² *Effects:* Constructs an object of type `basic_clip_props`.

³ The clip area is:

- (3.1) — `interpreted_path{pb}`; or
- (3.2) — `ip`; or
- (3.3) — `interpreted_path{r}`; <TODO>

⁴ The fill rule is `fr`.

13.12.4 basic_clip_props modifiers**[io2d.clipprops.modifiers]**

```
template <class Allocator>
void clip(const basic_path_builder<GraphicsSurfaces, Allocator>& pb);
void clip(const basic_interpreted_path<GraphicsSurfaces>& ip) noexcept;
```

¹ *Effects:* The clip area is:

- (1.1) — `interpreted_path{pb}`; or
- (1.2) — `ip`.

```
void fill_rule(experimental::io2d::fill_rule fr) noexcept;
```

² *Effects:* The fill rule is `fr`.

13.12.5 basic_clip_props observers**[io2d.clipprops.observers]**

```
basic_interpreted_path<GraphicsSurfaces> clip() const noexcept;
```

¹ *Returns:* The clip area.

```
io2d::fill_rule fill_rule() const noexcept;
```

² *Returns:* The fill rule.

13.13 Class basic_stroke_props**[io2d.strokeprops]****13.13.1 basic_stroke_props summary****[io2d.strokeprops.summary]**

¹ The `basic_stroke_props` class provides state information that is applicable to the stroking operation (see: [13.15.2](#) and [13.15.6](#)).

² It has a *line width* of type `float`, a *line cap* of type `line_cap`, a *line join* of type `line_join`, and a *miter limit* of type `float`.

13.13.2 basic_stroke_props synopsis**[io2d.strokeprops.synopsis]**

```
namespace std::experimental::io2d::v1 {
    template <class GraphicsSurfaces>
```

```

class basic_stroke_props {
public:
    using graphics_math_type = typename GraphicsSurfaces::graphics_math_type;

    // 13.13.3, constructors:
    basic_stroke_props() noexcept;
    explicit basic_stroke_props(float w, io2d::line_cap lc = io2d::line_cap::none,
        io2d::line_join lj = io2d::line_join::miter, float ml = 10.0f) noexcept;

    // 13.13.4, modifiers:
    void line_width(float w) noexcept;
    void line_cap(io2d::line_cap lc) noexcept;
    void line_join(io2d::line_join lj) noexcept;
    void miter_limit(float ml) noexcept;

    // 13.13.5, observers:
    float line_width() const noexcept;
    io2d::line_cap line_cap() const noexcept;
    io2d::line_join line_join() const noexcept;
    float miter_limit() const noexcept;
    float max_miter_limit() const noexcept;
};
}

```

13.13.3 basic_stroke_props constructors

[io2d.strokeprops.cons]

basic_stroke_props() noexcept;

1 *Effects:* Equivalent to: basic_stroke_props(2.0f).

explicit basic_stroke_props(float w, io2d::line_cap lc = io2d::line_cap::none,
 io2d::line_join lj = io2d::line_join::miter,
 float ml = 10.0f) noexcept;

2 *Requires:* w > 0.0f. ml >= 10.0f. ml <= max_miter_limit().

3 *Effects:* The line width is w. The line cap is lc. The line join is lj. The miter limit is ml.

13.13.4 basic_stroke_props modifiers

[io2d.strokeprops.modifiers]

void line_width(float w) noexcept;

1 *Requires:* w >= 0.0f.

2 *Effects:* The line width is w.

void line_cap(io2d::line_cap lc) noexcept;

3 *Effects:* The line cap is lc.

void line_join(io2d::line_join lj) noexcept;

4 *Effects:* The line join is lj.

void miter_limit(float ml) noexcept;

5 *Requires:* ml >= 1.0f and ml <= max_miter_limit.

6 *Effects:* The miter limit is ml.

13.13.5 basic_stroke_props observers

[io2d.strokeprops.observers]

```
float line_width() const noexcept;
```

1 *Returns:* The line width.

```
io2d::line_cap line_cap() const noexcept;
```

2 *Returns:* The line cap.

```
io2d::line_join line_join() const noexcept;
```

3 *Returns:* The line join.

```
float miter_limit() const noexcept;
```

4 *Returns:* The miter limit.

```
float max_miter_limit() const noexcept;
```

5 *Requires:* This value shall be finite and greater than 10.0f.

6 *Returns:* The implementation-defined maximum value of miter limit.

13.14 Class `basic_mask_props` [io2d.maskprops]

13.14.1 `basic_mask_props` summary [io2d.maskprops.summary]

1 The `basic_mask_props` class provides state information that is applicable to the mask rendering and composing operation (13.15.2).

2 It has a *wrap mode* of type `wrap_mode`, a *filter* of type `filter`, and a *mask matrix* of type `matrix_2d`.

13.14.2 `basic_mask_props` synopsis [io2d.maskprops.synopsis]

```
namespace std::experimental::io2d::v1 {
    template <class GraphicsSurfaces>
    class basic_mask_props {
    public:
        using graphics_math_type = typename GraphicsSurfaces::graphics_math_type;

        // 13.14.3, constructor:
        basic_mask_props(io2d::wrap_mode w = io2d::wrap_mode::repeat,
            io2d::filter fi = io2d::filter::good,
            const basic_matrix_2d<graphics_math_type>& m = basic_matrix_2d<graphics_math_type>{})
            noexcept;

        // 13.14.4, modifiers:
        void wrap_mode(io2d::wrap_mode w) noexcept;
        void filter(io2d::filter fi) noexcept;
        void mask_matrix(const basic_matrix_2d<graphics_math_type>& m) noexcept;

        // 13.14.5, observers:
        io2d::wrap_mode wrap_mode() const noexcept;
        io2d::filter filter() const noexcept;
        basic_matrix_2d<graphics_math_type> mask_matrix() const noexcept;
    };
}
```

13.14.3 `basic_mask_props` constructor [io2d.maskprops.cons]

```
basic_mask_props(io2d::wrap_mode w = io2d::wrap_mode::repeat,
    io2d::filter fi = io2d::filter::good,
    const basic_matrix_2d<graphics_math_type>& m = basic_matrix_2d<graphics_math_type>{}) noexcept;
```

Requires: `m.is_invertible() == true`.

1 *Effects:* The wrap mode is `w`. The filter is `fi`. The mask matrix is `m`.

13.14.4 `basic_mask_props` modifiers [io2d.maskprops.modifiers]

`void wrap_mode(io2d::wrap_mode w) noexcept;`

1 *Effects:* The wrap mode is `w`.

`void filter(io2d::filter fi) noexcept;`

2 *Effects:* The filter is `fi`.

`void mask_matrix(const basic_matrix_2d<graphics_math_type>& m) noexcept;`

3 *Requires:* `m.is_invertible() == true`.

4 *Effects:* The mask matrix is `m`.

13.14.5 `basic_mask_props` observers [io2d.maskprops.observers]

`io2d::wrap_mode wrap_mode() const noexcept;`

1 *Returns:* The wrap mode.

`io2d::filter filter() const noexcept;`

2 *Returns:* The filter.

`basic_matrix_2d<graphics_math_type> mask_matrix() const noexcept;`

3 *Returns:* The mask matrix.

13.15 Overview of surface classes [io2d.surface]

13.15.1 Surface class templates description [io2d.surface.intro]

1 There are three surface class templates:

(1.1) — `basic_image_surface`

(1.2) — `basic_output_surface`

(1.3) — `basic_unmanaged_output_surface`

2 The surface classes provides an interface for managing a graphics data graphics resource.

3 A surface object is a move-only object.

4 The surface classes modify their graphics resource through rendering and composing operations. They shall provide well-defined semantics for the graphics data graphics resource.

5 The definitions of the rendering and composing operations in [13.15.2](#) shall only be applicable when the graphics data graphics resource on which the surface class members operate is a raster graphics data graphics resource. In all other cases, any attempt to invoke the rendering and composing operations shall result in undefined behavior.

13.15.2 Rendering and composing [io2d.surface.rendering]

13.15.2.1 Operations [io2d.surface.rendering.ops]

1 The surface classes provide four fundamental rendering and composing operations:

Table 19 — surface rendering and composing operations

Operation	Function(s)
Painting	<code>paint</code>
Filling	<code>fill</code>
Stroking	<code>stroke</code>
Masking	<code>mask</code>

² All composing operations shall happen in a linear color space, regardless of the color space of the graphics data that is involved.

³ [*Note*: While a color space such as sRGB helps produce expected, consistent results when graphics data are viewed by people, composing operations only produce expected results when the channel data in the graphics data involved are uniformly (i.e. linearly) spaced. — *end note*]

13.15.2.2 Rendering and composing brushes [io2d.surface.rendering.brushes]

¹ All rendering and composing operations use a *source brush* of type `basic_brush`.

² The masking operation uses a *mask brush* of type `basic_brush`.

13.15.2.3 Rendering and composing source path [io2d.surface.rendering.sourcepath]

¹ In addition to brushes (13.15.2.2), all rendering and composing operation except for painting and masking use a *source path*. The source path is either a `basic_path_builder<Allocator>` object or a `basic_interpreted_path` object. If it is a `basic_path_builder<Allocator>` object, it is interpreted (11.3.16) before it is used as the source path.

13.15.2.4 Common state data [io2d.surface.rendering.commonstate]

¹ All rendering and composing operations use the following state data:

Table 20 — surface rendering and composing common state data

Name	Type
Brush properties	<code>brush_props</code>
Surface properties	<code>render_props</code>
Clip properties	<code>clip_props</code>

13.15.2.5 Specific state data [io2d.surface.rendering.specificstate]

¹ In addition to the common state data (13.15.2.4), certain rendering and composing operations use state data that is specific to each of them:

Table 21 — surface rendering and composing specific state data

Operation	Name	Type
Stroking	Stroke properties	<code>stroke_props</code>
Stroking	Dashes	<code>dashes</code>
Masking	Mask properties	<code>mask_props</code>

13.15.2.6 State data default values [io2d.surface.rendering.statedefaults]

¹ For all rendering and composing operations, the state data objects named above are provided using `optional<T>` class template arguments.

- ² If there is no contained value for a state data object, it is interpreted as-if the `optional<T>` argument contained a default constructed object of the relevant state data object.

13.15.3 Standard coordinate spaces

[io2d.surface.coordinatespaces]

- ¹ There are four standard coordinate spaces relevant to the rendering and composing operations (13.15.2):
- (1.1) — the brush coordinate space;
 - (1.2) — the mask coordinate space;
 - (1.3) — the user coordinate space; and
 - (1.4) — the surface coordinate space.
- ² The *brush coordinate space* is the standard coordinate space of the source brush (13.15.2.2). Its transformation matrix is the brush properties' brush matrix (13.11.1).
- ³ The *mask coordinate space* is the standard coordinate space of the mask brush (13.15.2.2). Its transformation matrix is the mask properties' mask matrix (13.14.1).
- ⁴ The *user coordinate space* is the standard coordinate space of `basic_interpreted_path` objects. Its transformation matrix is a default-constructed `basic_matrix_2d`.
- ⁵ The *surface coordinate space* is the standard coordinate space of the surface object's underlying graphics data graphics resource. Its transformation matrix is the surface properties' surface matrix (13.10.1).
- ⁶ Given a point `pt`, a brush coordinate space transformation matrix `bcsm`, a mask coordinate space transformation matrix `mcsm`, a user coordinate space transformation matrix `ucsm`, and a surface coordinate space transformation matrix `scsm`, the following table describes how to transform it from each of these standard coordinate spaces to the other standard coordinate spaces:

Table 22 — Point transformations

From	To	Transform
brush coordinate space	mask coordinate space	<code>mcsm.transform_pt(pt(bcsm.invert().transform_pt(pt)))</code> .
brush coordinate space	user coordinate space	<code>bcsm.invert().transform_pt(pt)</code> .
brush coordinate space	surface coordinate space	<code>scsm.transform_pt(pt(bcsm.invert().transform_pt(pt)))</code> .
user coordinate space	brush coordinate space	<code>bcsm.transform_pt(pt)</code> .
user coordinate space	mask coordinate space	<code>mcsm.transform_pt(pt)</code> .
user coordinate space	surface coordinate space	<code>scsm.transform_pt(pt)</code> .
surface coordinate space	brush coordinate space	<code>bcsm.transform_pt(scsm.invert().transform_pt(pt))</code> .
surface coordinate space	mask coordinate space	<code>mcsm.transform_pt(scsm.invert().transform_pt(pt))</code> .
surface coordinate space	user coordinate space	<code>scsm.invert().transform_pt(pt)</code> .

13.15.4 surface painting

[io2d.surface.painting]

- ¹ When a painting operation is initiated on a surface, the implementation shall produce results as-if the following steps were performed:
1. For each integral point *sp* of the underlying graphics data graphics resource, determine if *sp* is within

the clip area (`io2d.clipprops.summary`); if so, proceed with the remaining steps.

2. Transform *sp* from the surface coordinate space (13.15.3) to the brush coordinate space (Table 22), resulting in point *bp*.
3. Sample from point *bp* of the source brush (13.15.2.2), combine the resulting visual data with the visual data at point *sp* in the underlying graphics data graphics resource in the manner specified by the surface's current *compositing operator* (13.10.1), and modify the visual data of the underlying graphics data graphics resource at point *sp* to reflect the result produced by application of the compositing operator.

13.15.5 surface filling

[io2d.surface.filling]

- ¹ When a filling operation is initiated on a surface, the implementation shall produce results as-if the following steps were performed:
 1. For each integral point *sp* of the underlying graphics data graphics resource, determine if *sp* is within the *clip area* (13.12.1); if so, proceed with the remaining steps.
 2. Transform *sp* from the surface coordinate space (13.15.3) to the user coordinate space (Table 22), resulting in point *up*.
 3. Using the source path (13.15.2.3) and the fill rule (13.11.1), determine whether *up* shall be filled; if so, proceed with the remaining steps.
 4. Transform *up* from the user coordinate space to the brush coordinate space (13.15.3 and Table 22), resulting in point *bp*.
 5. Sample from point *bp* of the source brush (13.15.2.2), combine the resulting visual data with the visual data at point *sp* in the underlying graphics data graphics resource in the manner specified by the surface's current compositing operator (13.10.1), and modify the visual data of the underlying graphics data graphics resource at point *sp* to reflect the result produced by application of the compositing operator.

13.15.6 surface stroking

[io2d.surface.stroking]

- ¹ When a stroking operation is initiated on a surface, it is carried out for each figure in the source path (13.15.2).
- ² The following rules shall apply when a stroking operation is carried out on a figure:
 1. No part of the underlying graphics data graphics resource that is outside of the clip area shall be modified.
 2. If the figure is a closed figure, then the point where the end point of its final segment meets the start point of the initial segment shall be rendered as specified by the *line join* value (see: 13.13.1 and 13.15.2.5); otherwise the start point of the initial segment and end point of the final segment shall each be rendered as specified by the line cap value. The remaining meetings between successive end points and start points shall be rendered as specified by the line join value.
 3. If the dash pattern (Table 21) has its default value or if its `vector<float>` member is empty, the segments shall be rendered as a continuous path.
 4. If the dash pattern's `vector<float>` member contains only one value, that value shall be used to define a repeating pattern in which the path is shown then hidden. The ends of each shown portion of the path shall be rendered as specified by the line cap value.
 5. If the dash pattern's `vector<float>` member contains two or more values, the values shall be used to define a pattern in which the figure is alternatively rendered then not rendered for the length specified by the value. The ends of each rendered portion of the figure shall be rendered as specified by the line cap value. If the dash pattern's `float` member, which specifies an offset value, is not `0.0f`, the meaning

of its value is implementation-defined. If a rendered portion of the figure overlaps a not rendered portion of the figure, the rendered portion shall be rendered.

- ³ When a stroking operation is carried out on a figure, the width of each rendered portion shall be the *line width* (see: 13.13.1 and 13.15.2.5). Ideally this means that the diameter of the stroke at each rendered point should be equal to the line width. However, because there are an infinite number of points along each rendered portion, implementations may choose an unspecified method of determining minimum distances between points along each rendered portion and the diameter of the stroke between those points shall be the same. [Note: This concept is sometimes referred to as a tolerance. It allows for a balance between precision and performance, especially in situations where the end result is in a non-exact format such as raster graphics data. — end note]
- ⁴ After all figures in the path have been rendered but before the rendered result is composed to the underlying graphics data graphics resource, the rendered result shall be transformed from the user coordinate space (13.15.3) to the surface coordinate space (13.15.3).

13.15.7 surface masking

[io2d.surface.masking]

- ¹ A *mask brush* is composed of a graphics data graphics resource, a `wrap_mode` value, a `filter` value, and a `basic_matrix_2d` object.
- ² When a masking operation is initiated on a surface, the implementation shall produce results as-if the following steps were performed:
 1. For each integral point *sp* of the underlying graphics data graphics resource, determine if *sp* is within the clip area (13.12.1); if so, proceed with the remaining steps.
 2. Transform *sp* from the surface coordinate space (13.15.3) to the mask coordinate space (Table 22), resulting in point *mp*.
 3. Sample the alpha channel from point *mp* of the mask brush and store the result in *mac*; if the visual data format of the mask brush does not have an alpha channel, the value of *mac* shall always be 1.0.
 4. Transform *sp* from the surface coordinate space to the brush coordinate space, resulting in point *bp*.
 5. Sample from point *bp* of the source brush (13.15.2.2), combine the resulting visual data with the visual data at point *sp* in the underlying graphics data graphics resource in the manner specified by the surface's current compositing operator (13.10.1), multiply each channel of the result produced by application of the compositing operator by *map* if the visual data format of the underlying graphics data graphics resource is a premultiplied format and if not then just multiply the alpha channel of the result by *map*, and modify the visual data of the underlying graphics data graphics resource at point *sp* to reflect the multiplied result.

13.15.8 output surface miscellaneous behavior

[io2d.outputsurface.misc]

- ¹ What constitutes an output device is implementation-defined, with the sole constraint being that an output device must allow the user to see the dynamically-updated contents of the display buffer. [Example: An output device might be a window in a windowing system environment or the usable screen area of a smart phone or tablet. — end example]
- ² Implementations do not need to support the simultaneous existence of multiple `display_surface` objects.
- ³ All functions inherited from `surface` that affect its underlying graphics data graphics resource shall operate on the back buffer.

13.15.9 output surface state

[io2d.outputsurface.state]

- ¹ Table 23 specifies the name, type, function, and default value for each item of a display surface's observable state.

Table 23 — Output surface observable state

Name	Type	Function	Default value
<i>Letterbox brush</i>	brush	This is the brush that shall be used as specified by scaling::letterbox (Table 16)	brush{ { rgba_color::black } }
<i>Letterbox brush props</i>	brush_props	This is the brush properties for the letterbox brush	brush_props{ }
<i>Scaling type</i>	scaling	When the user scaling callback is equal to its default value, this is the type of scaling that shall be used when transferring the back buffer to the display buffer	scaling::letterbox
<i>Draw width</i>	int	The width in pixels of the back buffer. The minimum value is 1. The maximum value is unspecified. Because users can only request a preferred value for the draw width when setting and altering it, the maximum value may be a run-time determined value. If the preferred draw width exceeds the maximum value, then if a preferred draw height has also been supplied then implementations should provide a back buffer with the largest dimensions possible that maintain as nearly as possible the aspect ratio between the preferred draw width and the preferred draw height otherwise implementations should provide a back buffer with the largest dimensions possible that maintain as nearly as possible the aspect ratio between the preferred draw width and the current draw height	<i>N/A</i> [<i>Note:</i> It is impossible to create an output surface object without providing a preferred draw width value; as such a default value cannot exist. — <i>end note</i>]

Table 23 — Output surface observable state (continued)

Name	Type	Function	Default value
<i>Draw height</i>	int	The height in pixels of the back buffer. The minimum value is 1. The maximum value is unspecified. Because users can only request a preferred value for the draw height when setting and altering it, the maximum value may be a run-time determined value. If the preferred draw height exceeds the maximum value, then if a preferred draw width has also been supplied then implementations should provide a back buffer with the largest dimensions possible that maintain as nearly as possible the aspect ratio between the preferred draw width and the preferred draw height otherwise implementations should provide a back buffer with the largest dimensions possible that maintain as nearly as possible the aspect ratio between the current draw width and the preferred draw height	<i>N/A</i> [<i>Note</i> : It is impossible to create an output surface object without providing a preferred draw height value; as such a default value cannot exist. — <i>end note</i>]
<i>Draw format</i>	format	The pixel format of the back buffer. When an output surface object is created, a preferred pixel format value is provided. If the implementation does not support the preferred pixel format value as the value of draw format, the resulting value of draw format is implementation-defined	<i>N/A</i> [<i>Note</i> : It is impossible to create an output surface object without providing a preferred draw format value; as such a default value cannot exist. — <i>end note</i>]

Table 23 — Output surface observable state (continued)

Name	Type	Function	Default value
<i>Display width</i>	int	The width in pixels of the display buffer. The minimum value is unspecified. The maximum value is unspecified. Because users can only request a preferred value for the display width when setting and altering it, both the minimum value and the maximum value may be run-time determined values. If the preferred display width is not within the range between the minimum value and the maximum value, inclusive, then if a preferred display height has also been supplied then implementations should provide a display buffer with the largest dimensions possible that maintain as nearly as possible the aspect ratio between the preferred display width and the preferred display height otherwise implementations should provide a display buffer with the largest dimensions possible that maintain as nearly as possible the aspect ratio between the preferred display width and the current display height	<i>N/A</i> [<i>Note</i> : It is impossible to create an output surface object without providing a preferred display width value since in the absence of an explicit display width argument the mandatory preferred draw width argument is used as the preferred display width; as such a default value cannot exist. — <i>end note</i>]

Table 23 — Output surface observable state (continued)

Name	Type	Function	Default value
<i>Display height</i>	<code>int</code>	The height in pixels of the display buffer. The minimum value is unspecified. The maximum value is unspecified. Because users can only request a preferred value for the display height when setting and altering it, both the minimum value and the maximum value may be run-time determined values. If the preferred display height is not within the range between the minimum value and the maximum value, inclusive, then if a preferred display width has also been supplied then implementations should provide a display buffer with the largest dimensions possible that maintain as nearly as possible the aspect ratio between the preferred display width and the preferred display height otherwise implementations should provide a display buffer with the largest dimensions possible that maintain as nearly as possible the aspect ratio between the current display width and the preferred display height	<i>N/A</i> [<i>Note</i> : It is impossible to create an output surface object without providing a preferred display height value since in the absence of an explicit display height argument the mandatory preferred draw height argument is used as the preferred display height; as such a default value cannot exist. — <i>end note</i>]
<i>Auto clear</i>	<code>bool</code>	If <code>true</code> the implementation shall call <code>clear</code> , which shall clear the back buffer, immediately before it executes the draw callback	<code>false</code>
<i>Refresh rate</i>	<code>refresh_rate</code>	The <code>refresh_rate</code> value that determines when the draw callback shall be called while <code>output_surface::show</code> is being executed	<code>refresh_rate::as_fast_as_possible</code>

Table 23 — Output surface observable state (continued)

Name	Type	Function	Default value
<i>Desired frame rate</i>	float	This value is the number of times the draw callback shall be called per second while output surface::show is being executed when the value of refresh rate is refresh_rate::fixed, subject to the additional requirements documented in the meaning of refresh_rate::fixed (Table 17)	

13.16 Class basic_image_surface [io2d.imagesurface]

13.16.1 basic_image_surface summary [io2d.imagesurface.summary]

- ¹ The class `basic_image_surface` provides an interface to a raster graphics data graphics resource.
- ² It has a *pixel format* of type `format`, a *width* of type `int`, and a *height* of type `int`.
- ³ [Note: Because of the functionality it provides and what it can be used for, it is expected that developers familiar with other graphics technologies will think of the `basic_image_surface` class as being a form of *render target*. This is intentional, though this Technical Specification does not formally define or use that term to avoid any minor ambiguities and differences in its meaning between the various graphics technologies that do use the term render target. — end note]

13.16.2 basic_image_surface synopsis [io2d.imagesurface.synopsis]

```
namespace std::experimental::io2d::v1 {
    template <class GraphicsSurfaces>
    class basic_image_surface {
    public:
        using graphics_math_type = typename GraphicsSurfaces::graphics_math_type;

        // 13.16.3, construct/copy/move/destroy:
        basic_image_surface(io2d::format fmt, int width, int height);
        basic_image_surface(filesystem::path f, io2d::image_file_format iff, io2d::format fmt);
        basic_image_surface(filesystem::path f, io2d::image_file_format iff, io2d::format fmt,
            error_code& ec) noexcept;
        basic_image_surface(basic_image_surface&&) noexcept;
        basic_image_surface& operator=(basic_image_surface&&) noexcept;

        // 13.16.4, members:
        void save(filesystem::path p, image_file_format i);
        void save(filesystem::path p, image_file_format i, error_code& ec) noexcept;

        // 13.16.5, static members:
        static basic_display_point<graphics_math_type> max_dimensions() noexcept;

        // 13.16.6, observers:
        io2d::format format() const noexcept;
        basic_display_point<graphics_math_type> dimensions() const noexcept;
```

```

// 13.16.7, modifiers:
void clear();
void flush();
void flush(error_code& ec) noexcept;
void mark_dirty();
void mark_dirty(error_code& ec) noexcept;
void mark_dirty(const basic_bounding_box<graphics_math_type>& extents);
void mark_dirty(const basic_bounding_box<graphics_math_type>& extents,
    error_code& ec) noexcept;
void paint(const basic_brush<GraphicsSurfaces>& b,
    const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
    const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
    const optional<basic_clip_props<GraphicsSurfaces>>& cl = nullopt);
template <class Allocator>
void stroke(const basic_brush<GraphicsSurfaces>& b,
    const basic_path_builder<GraphicsSurfaces, Allocator>& pb,
    const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
    const optional<basic_stroke_props<GraphicsSurfaces>>& sp = nullopt,
    const optional<basic_dashes<GraphicsSurfaces>>& d = nullopt,
    const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
    const optional<basic_clip_props<GraphicsSurfaces>>& cl = nullopt);
void stroke(const basic_brush<GraphicsSurfaces>& b,
    const basic_interpreted_path<GraphicsSurfaces>& ip,
    const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
    const optional<basic_stroke_props<GraphicsSurfaces>>& sp = nullopt,
    const optional<basic_dashes<GraphicsSurfaces>>& d = nullopt,
    const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
    const optional<basic_clip_props<GraphicsSurfaces>>& cl = nullopt);
template <class Allocator>
void fill(const basic_brush<GraphicsSurfaces>& b,
    const basic_path_builder<GraphicsSurfaces, Allocator>& pb,
    const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
    const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
    const optional<basic_clip_props<GraphicsSurfaces>>& cl = nullopt);
void fill(const basic_brush<GraphicsSurfaces>& b,
    const basic_interpreted_path<GraphicsSurfaces>& ip,
    const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
    const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
    const optional<basic_clip_props<GraphicsSurfaces>>& cl = nullopt);
void mask(const basic_brush<GraphicsSurfaces>& b,
    const basic_brush<GraphicsSurfaces>& mb,
    const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
    const optional<basic_mask_props<GraphicsSurfaces>>& mp = nullopt,
    const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
    const optional<basic_clip_props<GraphicsSurfaces>>& cl = nullopt);
};

template <class GraphicsSurfaces>
basic_image_surface<GraphicsSurfaces> copy_image_surface(
    basic_image_surface<GraphicsSurfaces>& sfc) noexcept;
}

```

13.16.3 basic_image_surface constructors and assignment operators [io2d.imagesurface.cons]

```
basic_image_surface(io2d::format fmt, int w, int h);
```

1 *Requires:* `w` is greater than 0 and not greater than `basic_image_surface::max_width()`.

2 `h` is greater than 0 and not greater than `basic_image_surface::max_height()`.

3 `fmt` is not `io2d::format::invalid`.

4 *Effects:* Constructs an object of type `basic_image_surface`.

5 The pixel format is `fmt`, the width is `w`, and the height is `h`.

```
basic_image_surface(filesystem::path f, io2d::image_file_format i, io2d::format fmt);
basic_image_surface(filesystem::path f, io2d::image_file_format i, io2d::format fmt,
    error_code& ec) noexcept;
```

6 *Requires:* `f` is a file and its contents are data in either JPEG format, TIFF format or PNG format.

7 `fmt` is not `io2d::format::invalid`.

8 *Effects:* Constructs an object of type `basic_image_surface`.

9 The data of the underlying raster graphics data graphics resource is the raster graphics data that results from processing `f` into uncompressed raster graphics in the manner specified by the standard that specifies how to transform the contents of data contained in `f` into raster graphics data and then transforming that raster graphics data into the format specified by `fmt`.

10 The data of `f` is processed into uncompressed raster graphics data as specified by the value of `i`.

11 If `i` is `image_file_format::unknown`, implementations may attempt to process the data of `f` into uncompressed raster graphics data. The manner in which it does so is unspecified. If no uncompressed raster graphics data is produced, the error specified below occurs.

12 [*Note:* The intent of `image_file_format::unknown` is to allow implementations to support image file formats that are not required to be supported. — *end note*]

13 If the width of the uncompressed raster graphics data would be less than 1 or greater than `basic_image_surface::max_width()` or if the height of the uncompressed raster graphics data would be less than 1 or greater than `basic_image_surface::max_height()`, the error specified below occurs.

14 The resulting uncompressed raster graphics data is then transformed into the data format specified by `fmt`. If the format specified by `fmt` only contains an alpha channel, the values of the color channels, if any, of the underlying raster graphics data graphics resource are unspecified. If the format specified by `fmt` only contains color channels and the resulting uncompressed raster graphics data is in a premultiplied format, then the value of each color channel for each pixel is be divided by the value of the alpha channel for that pixel. The visual data is then set as the visual data of the underlying raster graphics data graphics resource.

15 The width is the width of the uncompressed raster graphics data. The height is the height of the uncompressed raster graphics data.

16 *Throws:* As specified in Error reporting (5).

17 *Error conditions:* Any error that could result from trying to access `f`, open `f` for reading, or reading data from `f`.

18 `errc::not_supported` if `image_file_format::unknown` is passed as an argument and the implementation is unable to determine the file format or does not support saving in the image file format it determined.

19 `errc::invalid_argument` if `fmt` is `io2d::format::invalid`.

20 `errc::argument_out_of_domain` if the width would be less than 1, the width would be greater than `basic_image_surface::max_width()`, the height would be less than 1, or the height would be greater than `basic_image_surface::max_height()`.

13.16.4 `basic_image_surface` members [io2d.imagesurface.members]

```
void save(filesystem::path p, image_file_format i);
void save(filesystem::path p, image_file_format i, error_code& ec) noexcept;
```

- 1 *Requires:* `p` shall be a valid path to a file. The file need not exist provided that the other components of the path are valid.
- 2 If the file exists, it shall be writable. If the file does not exist, it shall be possible to create the file at the specified path and then the created file shall be writable.
- 3 *Effects:* Any pending rendering and composing operations (13.15.2) are performed.
- 4 The visual data of the underlying raster graphics data graphics resource is written to `p` in the data format specified by `i`.
- 5 If `i` is `image_file_format::unknown`, it is implementation-defined whether the surface is saved in the image file format, if any, that the implementation associates with `p.extension()` provided that `p.has_extension() == true`. If `p.has_extension() == false`, the implementation does not associate an image file format with `p.extension()`, or the implementation does not support saving in that image file format, the error specified below occurs.
- 6 *Throws:* As specified in Error reporting (5).
- 7 *Error conditions:* Any error that could result from trying to create `f`, access `f`, or write data to `f`.
- 8 `errc::not_supported` if `image_file_format::unknown` is passed as an argument and the implementation is unable to determine the file format or does not support saving in the image file format it determined.

13.16.5 `basic_image_surface` static members [io2d.imagesurface.staticmembers]

```
static basic_display_point<graphics_math_type> max_dimensions() noexcept;
```

- 1 *Returns:* <TODO>The maximum height and width for a `basic_image_surface` object.

13.16.6 `basic_image_surface` observers [io2d.imagesurface.observers]

```
io2d::format format() const noexcept;
```

- 1 *Returns:* The pixel format.

```
basic_display_point<graphics_math_type> dimensions() const noexcept;
```

- 2 *Returns:* <TODO>The height and width.

13.16.7 `basic_image_surface` modifiers [io2d.imagesurface.mofifiers]

```
void clear();
```

- 1 *Effects:* <TODO>

```
void flush();
```

```
void flush(error_code& ec) noexcept;
```

- 2 *Effects:* If the implementation does not provide a native handle to the surface's underlying graphics data graphics resource, this function does nothing.
- 3 If the implementation does provide a native handle to the surface's underlying graphics data graphics resource, then the implementation performs every action necessary to ensure that all operations on the surface that produce observable effects occur.
- 4 The implementation performs any other actions necessary to ensure that the surface will be usable again after a call to `basic_image_surface::mark_dirty`.

Once a call to `basic_image_surface::flush` is made, `basic_image_surface::mark_dirty` shall be called before any other member function of the surface is called or the surface is used as an argument to any other function.

Throws: As specified in Error reporting (5).

Remarks: This function exists to allow the user to take control of the underlying surface using an implementation-provided native handle without introducing a race condition. The implementation's responsibility is to ensure that the user can safely use the underlying surface.

Error conditions: The potential errors are implementation-defined.

Implementations should avoid producing errors here.

If the implementation does not provide a native handle to the `basic_image_surface` object's underlying graphics data graphics resource, this function shall not produce any errors.

[*Note:* There are several purposes for `basic_image_surface::flush` and `basic_image_surface::mark_dirty`.

One is to allow implementation wide latitude in how they implement the rendering and composing operations (13.15.2), such as batching calls and then sending them to the underlying rendering and presentation technologies at appropriate times.

Another is to give implementations the chance during the call to `basic_image_surface::flush` to save any internal state that might be modified by the user and then restore it during the call to `basic_image_surface::mark_dirty`.

Other uses of this pair of calls are also possible. — *end note*]

```
void mark_dirty();
void mark_dirty(error_code& ec) noexcept;
void mark_dirty(const basic_bounding_box<graphics_math_type>& extents);
void mark_dirty(const basic_bounding_box<graphics_math_type>& extents, error_code& ec) noexcept;
```

Effects: If the implementation does not provide a native handle to the `basic_image_surface` object's underlying graphics data graphics resource, this function shall do nothing.

If the implementation does provide a native handle to the `basic_image_surface` object's underlying graphics data graphics resource, then:

— If called without a `basic_bounding_box` argument, informs the implementation that external changes using a native handle were potentially made to the entire underlying graphics data graphics resource.

— If called with a `basic_bounding_box` argument, informs the implementation that external changes using a native handle were potentially made to the underlying graphics data graphics resource within the bounds specified by the *bounding rectangle* `basic_bounding_box{ round(extents.x()), round(extents.y()), round(extents.width()), round(extents.height()) }`. No part of the bounding rectangle shall be outside of the bounds of the underlying graphics data graphics resource; no diagnostic is required.

Throws: As specified in Error reporting (5).

Remarks: After external changes are made to this `basic_image_surface` object's underlying graphics data graphics resource using a native pointer, this function shall be called before using this `basic_image_surface` object; no diagnostic is required.

Error conditions: The errors, if any, produced by this function are implementation-defined.

If the implementation does not provide a native handle to the `basic_image_surface` object's underlying graphics data graphics resource, this function shall not produce any errors.

```
void paint(const basic_brush<GraphicsSurfaces>& b,
  const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
  const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
  const optional<basic_clip_props<GraphicsSurfaces>>& cl = nullopt);
```

- 21 *Effects:* Performs the painting rendering and composing operation as specified by [13.15.4](#).
 22 The meanings of the parameters are specified by [13.15.2](#).
 23 *Throws:* As specified in Error reporting (5).
 24 *Error conditions:* The errors, if any, produced by this function are implementation-defined.

```
template <class Allocator>
void stroke(const basic_brush<GraphicsSurfaces>& b,
  const basic_path_builder<GraphicsSurfaces, Allocator>& pb,
  const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
  const optional<basic_stroke_props<GraphicsSurfaces>>& sp = nullopt,
  const optional<basic_dashes<GraphicsSurfaces>>& d = nullopt,
  const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
  const optional<basic_clip_props<GraphicsSurfaces>>& cl = nullopt);
void stroke(const basic_brush<GraphicsSurfaces>& b,
  const basic_interpreted_path<GraphicsSurfaces>& ip,
  const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
  const optional<basic_stroke_props<GraphicsSurfaces>>& sp = nullopt,
  const optional<basic_dashes<GraphicsSurfaces>>& d = nullopt,
  const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
  const optional<basic_clip_props<GraphicsSurfaces>>& cl = nullopt);
```

- 25 *Effects:* Performs the stroking rendering and composing operation as specified by [13.15.6](#).
 26 The meanings of the parameters are specified by [13.15.2](#).
 27 *Throws:* As specified in Error reporting (5).
 28 *Error conditions:* The errors, if any, produced by this function are implementation-defined.

```
template <class Allocator>
void fill(const basic_brush<GraphicsSurfaces>& b,
  const basic_path_builder<GraphicsSurfaces, Allocator>& pb,
  const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
  const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
  const optional<basic_clip_props<GraphicsSurfaces>>& cl = nullopt);
void fill(const basic_brush<GraphicsSurfaces>& b,
  const basic_interpreted_path<GraphicsSurfaces>& ip,
  const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
  const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
  const optional<basic_clip_props<GraphicsSurfaces>>& cl = nullopt);
```

- 29 *Effects:* Performs the filling rendering and composing operation as specified by [13.15.5](#).
 30 The meanings of the parameters are specified by [13.15.2](#).
 31 *Throws:* As specified in Error reporting (5).
 32 *Error conditions:* The errors, if any, produced by this function are implementation-defined.

```
void mask(const basic_brush<GraphicsSurfaces>& b,
  const basic_brush<GraphicsSurfaces>& mb,
  const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
  const optional<basic_mask_props<GraphicsSurfaces>>& mp = nullopt,
  const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
```

```
const optional<basic_clip_props<GraphicsSurfaces>>& c1 = nullopt);
```

33 *Effects:* Performs the masking rendering and composing operation as specified by 13.15.7.

34 The meanings of the parameters are specified by 13.15.2.

35 *Throws:* As specified in Error reporting (5).

36 *Error conditions:*

The errors, if any, produced by this function are implementation-defined.

13.17 Class `basic_output_surface` [io2d.outputsurface]

13.17.1 `basic_output_surface` summary [io2d.outputsurface.summary]

1 <TODO>

13.17.2 `basic_output_surface` synopsis [io2d.outputsurface.synopsis]

```
namespace std::experimental::io2d::v1 {
    template <class GraphicsSurfaces>
    class basic_output_surface {
    public:
        using graphics_math_type = typename GraphicsSurfaces::graphics_math_type;

        // 13.17.3, constructors:
        basic_output_surface(int preferredWidth, int preferredHeight, io2d::format preferredFormat,
            io2d::scaling scl = io2d::scaling::letterbox,
            io2d::refresh_style rr = io2d::refresh_style::as_fast_as_possible, float fps = 30.0f);
        basic_output_surface(int preferredWidth, int preferredHeight, io2d::format preferredFormat,
            error_code& ec, io2d::scaling scl = io2d::scaling::letterbox,
            io2d::refresh_style rr = io2d::refresh_style::as_fast_as_possible, float fps = 30.0f)
            noexcept;
        basic_output_surface(int preferredWidth, int preferredHeight, io2d::format preferredFormat,
            int preferredDisplayWidth, int preferredDisplayHeight,
            io2d::scaling scl = io2d::scaling::letterbox,
            io2d::refresh_style rr = io2d::refresh_style::as_fast_as_possible, float fps = 30.0f);
        basic_output_surface(int preferredWidth, int preferredHeight, io2d::format preferredFormat,
            int preferredDisplayWidth, int preferredDisplayHeight, error_code& ec,
            io2d::scaling scl = io2d::scaling::letterbox,
            io2d::refresh_style rr = io2d::refresh_style::as_fast_as_possible, float fps = 30.0f)
            noexcept;

        // 13.17.4, observers:
        io2d::format format() const noexcept;
        basic_display_point<graphics_math_type> dimensions() const noexcept;
        basic_display_point<graphics_math_type> max_dimensions() const noexcept;
        basic_display_point<graphics_math_type> display_dimensions() const noexcept;
        basic_display_point<graphics_math_type> max_display_dimensions() const noexcept;
        io2d::scaling scaling() const noexcept;
        optional<basic_brush<GraphicsSurfaces>> letterbox_brush() const noexcept;
        optional<basic_brush_props<GraphicsSurfaces>> letterbox_brush_props() const noexcept;
        bool auto_clear() const noexcept;

        // 13.17.5, modifiers:
        int begin_show();
        void end_show();
        void clear();
        void flush();
```

```

void flush(error_code& ec) noexcept;
void mark_dirty();
void mark_dirty(error_code& ec) noexcept;
void mark_dirty(const basic_bounding_box<graphics_math_type>& extents);
void mark_dirty(const basic_bounding_box<graphics_math_type>& extents, error_code& ec)
    noexcept;
void paint(const basic_brush<GraphicsSurfaces>& b,
    const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
    const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
    const optional<basic_clip_props<GraphicsSurfaces>>& cl = nullopt);
template <class Allocator>
void stroke(const basic_brush<GraphicsSurfaces>& b,
    const basic_path_builder<GraphicsSurfaces, Allocator>& pb,
    const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
    const optional<basic_stroke_props<GraphicsSurfaces>>& sp = nullopt,
    const optional<basic_dashes<GraphicsSurfaces>>& d = nullopt,
    const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
    const optional<basic_clip_props<GraphicsSurfaces>>& cl = nullopt);
void stroke(const basic_brush<GraphicsSurfaces>& b,
    const basic_interpreted_path<GraphicsSurfaces>& ip,
    const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
    const optional<basic_stroke_props<GraphicsSurfaces>>& sp = nullopt,
    const optional<basic_dashes<GraphicsSurfaces>>& d = nullopt,
    const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
    const optional<basic_clip_props<GraphicsSurfaces>>& cl = nullopt);
template <class Allocator>
void fill(const basic_brush<GraphicsSurfaces>& b,
    const basic_path_builder<GraphicsSurfaces, Allocator>& pb,
    const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
    const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
    const optional<basic_clip_props<GraphicsSurfaces>>& cl = nullopt);
void fill(const basic_brush<GraphicsSurfaces>& b,
    const basic_interpreted_path<GraphicsSurfaces>& ip,
    const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
    const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
    const optional<basic_clip_props<GraphicsSurfaces>>& cl = nullopt);
void mask(const basic_brush<GraphicsSurfaces>& b,
    const basic_brush<GraphicsSurfaces>& mb,
    const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
    const optional<basic_mask_props<GraphicsSurfaces>>& mp = nullopt,
    const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
    const optional<basic_clip_props<GraphicsSurfaces>>& cl = nullopt);
void draw_callback(const function<void(basic_output_surface& sfc)>& fn);
void size_change_callback(const function<void(basic_output_surface& sfc)>& fn);
void dimensions(basic_display_point<graphics_math_type> dp);
void dimensions(basic_display_point<graphics_math_type> dp, error_code& ec) noexcept;
void display_dimensions(basic_display_point<graphics_math_type> dp);
void display_dimensions(basic_display_point<graphics_math_type> dp, error_code& ec) noexcept;
void scaling(io2d::scaling scl) noexcept;
void user_scaling_callback(const
    function<basic_bounding_box<graphics_math_type>(const basic_output_surface&, bool&)>& fn);
void letterbox_brush(const optional<basic_brush<GraphicsSurfaces>>& b,
    const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt) noexcept;
void letterbox_brush_props(const optional<basic_brush_props<GraphicsSurfaces>>& bp) noexcept;
void auto_clear(bool val) noexcept;

```



```

        void redraw_required(bool val = true) noexcept;
    };
}

```

13.17.3 basic_output_surface constructors

[io2d.outputsurface.cons]

```

basic_output_surface(int preferredWidth, int preferredHeight, io2d::format preferredFormat,
    io2d::scaling scl = io2d::scaling::letterbox,
    io2d::refresh_style rr = io2d::refresh_style::as_fast_as_possible, float fps = 30.0f);
basic_output_surface(int preferredWidth, int preferredHeight, io2d::format preferredFormat,
    error_code& ec, io2d::scaling scl = io2d::scaling::letterbox,
    io2d::refresh_style rr = io2d::refresh_style::as_fast_as_possible, float fps = 30.0f)
    noexcept;

```

1 <TODO>

```

basic_output_surface(int preferredWidth, int preferredHeight, io2d::format preferredFormat,
    int preferredDisplayWidth, int preferredDisplayHeight,
    io2d::scaling scl = io2d::scaling::letterbox,
    io2d::refresh_style rr = io2d::refresh_style::as_fast_as_possible, float fps = 30.0f);
basic_output_surface(int preferredWidth, int preferredHeight,
    io2d::format preferredFormat, int preferredDisplayWidth, int preferredDisplayHeight,
    error_code& ec, io2d::scaling scl = io2d::scaling::letterbox,
    io2d::refresh_style rr = io2d::refresh_style::as_fast_as_possible, float fps = 30.0f)
    noexcept;

```

2 <TODO>

13.17.4 basic_output_surface observers

[io2d.outputsurface.observers]

```

io2d::format format() const noexcept;

```

1 *Returns:* The pixel format.

```

basic_display_point<graphics_math_type> dimensions() const noexcept;

```

2 *Returns:* <TODO>The height and width.

```

basic_display_point<graphics_math_type> max_dimensions() const noexcept;

```

3 *Returns:* <TODO>The maximum available height and width of a basic_output_surcace for the device.

```

basic_display_point<graphics_math_type> display_dimensions() const noexcept;

```

4 *Returns:* <TODO>

```

basic_display_point<graphics_math_type> max_display_dimensions() const noexcept;

```

5 *Returns:* <TODO>

```

io2d::scaling scaling() const noexcept;

```

6 *Returns:* The scaling type.

```

optional<basic_brush<GraphicsSurfaces>> letterbox_brush() const noexcept;

```

7 *Returns:* An optional<basic_brush<GraphicsSurfaces>> object constructed using the user-provided letterbox brush or, if the letterbox brush is set to its default value, an empty optional<basic_brush<GraphicsSurfaces>> object.

```

optional<basic_brush_props<GraphicsSurfaces>> letterbox_brush_props() const noexcept;

```

8 *Returns:* An `optional<basic_brush_props<GraphicsSurfaces>>` object constructed using the user-provided letterbox brush props or, if the letterbox brush props is set to its default value, an empty `optional<basic_brush_props<GraphicsSurfaces>>` object.

```
bool auto_clear() const noexcept;
```

9 *Returns:* The value of auto clear.

13.17.5 basic_output_surface modifiers [io2d.outputsurface.modifiers]

```
int begin_show();
```

1 *Effects:* Performs the following actions in a continuous loop:

1. Handle any implementation and host environment matters. If there are no pending implementation or host environment matters to handle, proceed immediately to the next action.
2. Run the size change callback if doing so is required by its specification and it does not have a value equivalent to its default value.
3. If the refresh rate requires that the draw callback be called then:
 - a) Evaluate auto clear and perform the actions required by its specification, if any.
 - b) Run the draw callback.
 - c) Ensure that all operations from the draw callback that can effect the back buffer have completed.
 - d) Transfer the contents of the back buffer to the display buffer using sampling with an unspecified filter. If the user scaling callback does not have a value equivalent to its default value, use it to determine the position where the contents of the back buffer shall be transferred to and whether or not the letterbox brush should be used. Otherwise use the value of scaling type to determine the position and whether the letterbox brush should be used.

2 If `basic_output_surface::end_show` is called from the draw callback, the implementation shall finish executing the draw callback and shall immediately cease to perform any actions in the continuous loop other than handling any implementation and host environment matters needed to exit the loop properly.

3 No later than when this function returns, the output device shall cease to display the contents of the display buffer.

4 What the output device shall display when it is not displaying the contents of the display buffer is unspecified.

5 *Returns:* The possible values and meanings of the possible values returned are implementation-defined.

6 *Throws:* As specified in Error reporting (5).

7 *Remarks:* Since this function calls the draw callback and can call the size change callback and the user scaling callback, in addition to the errors documented below, any errors that the callback functions produce can also occur.

8 *Error conditions:* `errc::operation_would_block` if the value of draw callback is equivalent to its default value or if it becomes equivalent to its default value before this function returns.

9 Other errors, if any, produced by this function are implementation-defined.

```
void end_show();
```

10 *Effects:* If this function is called outside of the draw callback while it is being executed in the `basic_output_surface::begin_show` function's continuous loop, it does nothing.

- 11 Otherwise, the implementation initiates the process of exiting the `basic_output_surface::begin_show` function's continuous loop.
- 12 If possible, any procedures that the host environment requires in order to cause the `basic_output_surface::show` function's continuous loop to stop executing without error should be followed.
- 13 The `basic_output_surface::begin_show` function's loop continues execution until it returns.
- ```
void clear();
```
- 14 *Effects:* <TODO>
- ```
void flush();
void flush(error_code& ec) noexcept;
```
- 15 *Effects:* If the implementation does not provide a native handle to the surface's underlying graphics data graphics resource, this function does nothing.
- 16 If the implementation does provide a native handle to the surface's underlying graphics data graphics resource, then the implementation performs every action necessary to ensure that all operations on the surface that produce observable effects occur.
- 17 The implementation performs any other actions necessary to ensure that the surface will be usable again after a call to `basic_output_surface::mark_dirty`.
- 18 Once a call to `basic_output_surface::flush` is made, `basic_output_surface::mark_dirty` shall be called before any other member function of the surface is called or the surface is used as an argument to any other function.
- 19 *Throws:* As specified in Error reporting (5).
- 20 *Remarks:* This function exists to allow the user to take control of the underlying surface using an implementation-provided native handle without introducing a race condition. The implementation's responsibility is to ensure that the user can safely use the underlying surface.
- 21 *Error conditions:* The potential errors are implementation-defined.
- 22 Implementations should avoid producing errors here.
- 23 If the implementation does not provide a native handle to the `basic_output_surface` object's underlying graphics data graphics resource, this function shall not produce any errors.
- 24 [*Note:* There are several purposes for `basic_output_surface::flush` and `basic_output_surface::mark_dirty`.
- 25 One is to allow implementation wide latitude in how they implement the rendering and composing operations (13.15.2), such as batching calls and then sending them to the underlying rendering and presentation technologies at appropriate times.
- 26 Another is to give implementations the chance during the call to `basic_output_surface::flush` to save any internal state that might be modified by the user and then restore it during the call to `basic_output_surface::mark_dirty`.
- 27 Other uses of this pair of calls are also possible. — *end note*]
- ```
void mark_dirty();
void mark_dirty(error_code& ec) noexcept;
void mark_dirty(const basic_bounding_box<graphics_math_type>& extents);
void mark_dirty(const basic_bounding_box<graphics_math_type>& extents, error_code& ec) noexcept;
```
- 28 *Effects:* If the implementation does not provide a native handle to the `basic_output_surface` object's underlying graphics data graphics resource, this function shall do nothing.

29 If the implementation does provide a native handle to the `basic_output_surface` object's underlying graphics data graphics resource, then:

- (29.1) — If called without a `basic_bounding_box` argument, informs the implementation that external changes using a native handle were potentially made to the entire underlying graphics data graphics resource.
- (29.2) — If called with a `basic_bounding_box` argument, informs the implementation that external changes using a native handle were potentially made to the underlying graphics data graphics resource within the bounds specified by the *bounding rectangle* `basic_bounding_box{ round(extents.x()), round(extents.y()), round(extents.width()), round(extents.height()) }`. No part of the bounding rectangle shall be outside of the bounds of the underlying graphics data graphics resource; no diagnostic is required.

30 *Throws:* As specified in Error reporting (5).

31 *Remarks:* After external changes are made to this `basic_output_surface` object's underlying graphics data graphics resource using a native pointer, this function shall be called before using this `basic_output_surface` object; no diagnostic is required.

32 *Error conditions:* The errors, if any, produced by this function are implementation-defined.

33 If the implementation does not provide a native handle to the `basic_output_surface` object's underlying graphics data graphics resource, this function shall not produce any errors.

```
void paint(const basic_brush<GraphicsSurfaces>& b,
const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
const optional<basic_clip_props<GraphicsSurfaces>>& cl = nullopt);
```

34 *Effects:* Performs the painting rendering and composing operation as specified by 13.15.4.

35 The meanings of the parameters are specified by 13.15.2.

36 *Throws:* As specified in Error reporting (5).

37 *Error conditions:* The errors, if any, produced by this function are implementation-defined.

```
template <class Allocator>
void stroke(const basic_brush<GraphicsSurfaces>& b,
const basic_path_builder<GraphicsSurfaces, Allocator>& pb,
const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
const optional<basic_stroke_props<GraphicsSurfaces>>& sp = nullopt,
const optional<basic_dashes<GraphicsSurfaces>>& d = nullopt,
const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
const optional<basic_clip_props<GraphicsSurfaces>>& cl = nullopt);
void stroke(const basic_brush<GraphicsSurfaces>& b,
const basic_interpreted_path<GraphicsSurfaces>& ip,
const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
const optional<basic_stroke_props<GraphicsSurfaces>>& sp = nullopt,
const optional<basic_dashes<GraphicsSurfaces>>& d = nullopt,
const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
const optional<basic_clip_props<GraphicsSurfaces>>& cl = nullopt);
```

38 *Effects:* Performs the stroking rendering and composing operation as specified by 13.15.6.

39 The meanings of the parameters are specified by 13.15.2.

40 *Throws:* As specified in Error reporting (5).

41 *Error conditions:* The errors, if any, produced by this function are implementation-defined.

```

template <class Allocator>
void fill(const basic_brush<GraphicsSurfaces>& b,
 const basic_path_builder<GraphicsSurfaces, Allocator>& pb,
 const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
 const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
 const optional<basic_clip_props<GraphicsSurfaces>>& cl = nullopt);
void fill(const basic_brush<GraphicsSurfaces>& b,
 const basic_interpreted_path<GraphicsSurfaces>& ip,
 const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
 const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
 const optional<basic_clip_props<GraphicsSurfaces>>& cl = nullopt);

```

42       *Effects:* Performs the filling rendering and composing operation as specified by [13.15.5](#).

43       The meanings of the parameters are specified by [13.15.2](#).

44       *Throws:* As specified in Error reporting (5).

45       *Error conditions:* The errors, if any, produced by this function are implementation-defined.

```

void mask(const basic_brush<GraphicsSurfaces>& b,
 const basic_brush<GraphicsSurfaces>& mb,
 const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
 const optional<basic_mask_props<GraphicsSurfaces>>& mp = nullopt,
 const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
 const optional<basic_clip_props<GraphicsSurfaces>>& cl = nullopt);

```

46       *Effects:* Performs the masking rendering and composing operation as specified by [13.15.7](#).

47       The meanings of the parameters are specified by [13.15.2](#).

48       *Throws:* As specified in Error reporting (5).

49       *Error conditions:*

The errors, if any, produced by this function are implementation-defined.

```

void draw_callback(const function<void(basic_output_surface& sfc)>& fn);

```

50       *Effects:* <TODO>

```

void size_change_callback(const function<void(basic_output_surface& sfc)>& fn);

```

51       *Effects:* <TODO>

```

void dimensions(basic_display_point<graphics_math_type> dp);
void dimensions(basic_display_point<graphics_math_type> dp, error_code& ec) noexcept;

```

52       *Effects:* <TODO>

```

void display_dimensions(basic_display_point<graphics_math_type> dp);
void display_dimensions(basic_display_point<graphics_math_type> dp, error_code& ec) noexcept;

```

53       *Effects:* <TODO>

```

void scaling(io2d::scaling scl) noexcept;

```

54       *Effects:* <TODO>

```

void user_scaling_callback(const
 function<basic_bounding_box<graphics_math_type>(const basic_output_surface&, bool&)>& fn);

```

55       *Effects:* <TODO>

```

void letterbox_brush(const optional<basic_brush<GraphicsSurfaces>>& b,

```

```
 const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt) noexcept;
void letterbox_brush_props(const optional<basic_brush_props<GraphicsSurfaces>>& bp) noexcept;
```

56     *Effects:* <TODO>

```
void auto_clear(bool val) noexcept;
```

57     *Effects:* <TODO>

```
void redraw_required(bool val = true) noexcept;
```

58     *Effects:* <TODO>

## 13.18 Class basic\_unmanaged\_output\_surface [io2d.unmanagedoutputsurface]

### 13.18.1 basic\_unmanaged\_output\_surface summary [io2d.unmanagedoutputsurface.summary]

1 <TODO>

### 13.18.2 basic\_unmanaged\_output\_surface synopsis [io2d.unmanagedoutputsurface.synopsis]

```
namespace std::experimental::io2d::v1 {
 template <class GraphicsSurfaces>
 class basic_unmanaged_output_surface {
 public:
 using graphics_math_type = typename GraphicsSurfaces::graphics_math_type;
 using data_type = typename GraphicsSurfaces::surfaces::unmanaged_output_surface_data_type;

 // 13.18.3, constructor:
 basic_unmanaged_output_surface(data_type&& data) noexcept;

 // 13.18.4, observers:
 bool has_draw_callback() const noexcept;
 bool has_size_change_callback() const noexcept;
 bool has_user_scaling_callback() const noexcept;
 io2d::format format() const noexcept;
 basic_display_point<graphics_math_type> dimensions() const noexcept;
 basic_display_point<graphics_math_type> max_dimensions() const noexcept;
 basic_display_point<graphics_math_type> display_dimensions() const noexcept;
 basic_display_point<graphics_math_type> max_display_dimensions() const noexcept;
 io2d::scaling scaling() const noexcept;
 optional<basic_brush<GraphicsSurfaces>> letterbox_brush() const noexcept;
 optional<basic_brush_props<GraphicsSurfaces>> letterbox_brush_props() const noexcept;
 bool auto_clear() const noexcept;

 // 13.18.5, modifiers:
 void invoke_draw_callback();
 void invoke_size_change_callback();
 basic_bounding_box<graphics_math_type> invoke_user_scaling_callback(bool& useLetterbox);
 void draw_to_output();
 void clear();
 void flush();
 void flush(error_code& ec) noexcept;
 void mark_dirty();
 void mark_dirty(error_code& ec) noexcept;
 void mark_dirty(const basic_bounding_box<graphics_math_type>& extents);
```

```

void mark_dirty(const basic_bounding_box<graphics_math_type>& extents, error_code& ec)
 noexcept;
void paint(const basic_brush<GraphicsSurfaces>& b,
 const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
 const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
 const optional<basic_clip_props<GraphicsSurfaces>>& cl = nullopt);
template <class Allocator>
void stroke(const basic_brush<GraphicsSurfaces>& b,
 const basic_path_builder<GraphicsSurfaces, Allocator>& pb,
 const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
 const optional<basic_stroke_props<GraphicsSurfaces>>& sp = nullopt,
 const optional<basic_dashes<GraphicsSurfaces>>& d = nullopt,
 const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
 const optional<basic_clip_props<GraphicsSurfaces>>& cl = nullopt);
void stroke(const basic_brush<GraphicsSurfaces>& b,
 const basic_interpreted_path<GraphicsSurfaces>& ip,
 const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
 const optional<basic_stroke_props<GraphicsSurfaces>>& sp = nullopt,
 const optional<basic_dashes<GraphicsSurfaces>>& d = nullopt,
 const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
 const optional<basic_clip_props<GraphicsSurfaces>>& cl = nullopt);
template <class Allocator>
void fill(const basic_brush<GraphicsSurfaces>& b,
 const basic_path_builder<GraphicsSurfaces, Allocator>& pb,
 const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
 const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
 const optional<basic_clip_props<GraphicsSurfaces>>& cl = nullopt);
void fill(const basic_brush<GraphicsSurfaces>& b,
 const basic_interpreted_path<GraphicsSurfaces>& ip,
 const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
 const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
 const optional<basic_clip_props<GraphicsSurfaces>>& cl = nullopt);
void mask(const basic_brush<GraphicsSurfaces>& b,
 const basic_brush<GraphicsSurfaces>& mb,
 const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
 const optional<basic_mask_props<GraphicsSurfaces>>& mp = nullopt,
 const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
 const optional<basic_clip_props<GraphicsSurfaces>>& cl = nullopt);
void draw_callback(const function<void(basic_unmanaged_output_surface& sfc)>& fn);
void size_change_callback(const function<void(basic_unmanaged_output_surface& sfc)>& fn);
void dimensions(basic_display_point<graphics_math_type> dp);
void dimensions(basic_display_point<graphics_math_type> dp, error_code& ec) noexcept;
void display_dimensions(basic_display_point<graphics_math_type> dp);
void display_dimensions(basic_display_point<graphics_math_type> dp, error_code& ec) noexcept;
void scaling(io2d::scaling scl) noexcept;
void user_scaling_callback(const
 function<basic_bounding_box<graphics_math_type>(const
 basic_unmanaged_output_surface&, bool&)>& fn);
void letterbox_brush(const optional<basic_brush<GraphicsSurfaces>>& b,
 const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt) noexcept;
void letterbox_brush_props(const optional<basic_brush_props<GraphicsSurfaces>>& bp) noexcept;
void auto_clear(bool val) noexcept;
void redraw_required(bool val = true) noexcept;
};
}

```

### 13.18.3 basic\_unmanaged\_output\_surface constructor [io2d.unmanagedoutputsurface.cons]

basic\_unmanaged\_output\_surface(data\_type&& data) noexcept;

1 <TODO>

### 13.18.4 basic\_unmanaged\_output\_surface observers [io2d.unmanagedoutputsurface.observers]

bool has\_draw\_callback() const noexcept;

1 *Returns:* <TODO>

bool has\_size\_change\_callback() const noexcept;

2 *Returns:* <TODO>

bool has\_user\_scaling\_callback() const noexcept;

3 *Returns:* <TODO>

io2d::format format() const noexcept;

4 *Returns:* <TODO>

basic\_display\_point<graphics\_math\_type> dimensions() const noexcept;

5 *Returns:* <TODO>

basic\_display\_point<graphics\_math\_type> max\_dimensions() const noexcept;

6 *Returns:* <TODO>

basic\_display\_point<graphics\_math\_type> display\_dimensions() const noexcept;

7 *Returns:* <TODO>

basic\_display\_point<graphics\_math\_type> max\_display\_dimensions() const noexcept;

8 *Returns:* <TODO>

io2d::scaling scaling() const noexcept;

9 *Returns:* <TODO>

optional<basic\_brush<GraphicsSurfaces>> letterbox\_brush() const noexcept;

10 *Returns:* <TODO>

optional<basic\_brush\_props<GraphicsSurfaces>> letterbox\_brush\_props() const noexcept;

11 *Returns:* <TODO>

bool auto\_clear() const noexcept;

12 *Returns:* <TODO>

### 13.18.5 basic\_unmanaged\_output\_surface modifiers [io2d.unmanagedoutputsurface.mofifiers]

void clear();

1 *Effects:* <TODO>



```
void flush();
void flush(error_code& ec) noexcept;
```

- 2 *Effects:* If the implementation does not provide a native handle to the surface's underlying graphics data graphics resource, this function does nothing.
- 3 If the implementation does provide a native handle to the surface's underlying graphics data graphics resource, then the implementation performs every action necessary to ensure that all operations on the surface that produce observable effects occur.
- 4 The implementation performs any other actions necessary to ensure that the surface will be usable again after a call to `basic_unmanaged_output_surface::mark_dirty`.
- 5 Once a call to `basic_unmanaged_output_surface::flush` is made, `basic_unmanaged_output_surface::mark_dirty` shall be called before any other member function of the surface is called or the surface is used as an argument to any other function.
- 6 *Throws:* As specified in Error reporting (5).
- 7 *Remarks:* This function exists to allow the user to take control of the underlying surface using an implementation-provided native handle without introducing a race condition. The implementation's responsibility is to ensure that the user can safely use the underlying surface.
- 8 *Error conditions:* The potential errors are implementation-defined.
- 9 Implementations should avoid producing errors here.
- 10 If the implementation does not provide a native handle to the `basic_unmanaged_output_surface` object's underlying graphics data graphics resource, this function shall not produce any errors.
- 11 [ *Note:* There are several purposes for `basic_unmanaged_output_surface::flush` and `basic_unmanaged_output_surface::mark_dirty`.
- 12 One is to allow implementation wide latitude in how they implement the rendering and composing operations (13.15.2), such as batching calls and then sending them to the underlying rendering and presentation technologies at appropriate times.
- 13 Another is to give implementations the chance during the call to `basic_unmanaged_output_surface::flush` to save any internal state that might be modified by the user and then restore it during the call to `basic_unmanaged_output_surface::mark_dirty`.
- 14 Other uses of this pair of calls are also possible. — *end note* ]

```
void mark_dirty();
void mark_dirty(error_code& ec) noexcept;
void mark_dirty(const basic_bounding_box<graphics_math_type>& extents);
void mark_dirty(const basic_bounding_box<graphics_math_type>& extents, error_code& ec) noexcept;
```

- 15 *Effects:* If the implementation does not provide a native handle to the `basic_unmanaged_output_surface` object's underlying graphics data graphics resource, this function shall do nothing.
- 16 If the implementation does provide a native handle to the `basic_unmanaged_output_surface` object's underlying graphics data graphics resource, then:
- (16.1) — If called without a `basic_bounding_box` argument, informs the implementation that external changes using a native handle were potentially made to the entire underlying graphics data graphics resource.
- (16.2) — If called with a `basic_bounding_box` argument, informs the implementation that external changes using a native handle were potentially made to the underlying graphics data graphics resource within the bounds specified by the *bounding rectangle* `basic_bounding_box{ round(extents.x()), round(extents.y()), round(extents.width()), round(extents.height()) }`. No part of

the bounding rectangle shall be outside of the bounds of the underlying graphics data graphics resource; no diagnostic is required.

17 *Throws:* As specified in Error reporting (5).

18 *Remarks:* After external changes are made to this `basic_unmanaged_output_surface` object's underlying graphics data graphics resource using a native pointer, this function shall be called before using this `basic_unmanaged_output_surface` object; no diagnostic is required.

19 *Error conditions:* The errors, if any, produced by this function are implementation-defined.

20 If the implementation does not provide a native handle to the `basic_unmanaged_output_surface` object's underlying graphics data graphics resource, this function shall not produce any errors.

```
void paint(const basic_brush<GraphicsSurfaces>& b,
 const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
 const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
 const optional<basic_clip_props<GraphicsSurfaces>>& cl = nullopt);
```

21 *Effects:* Performs the painting rendering and composing operation as specified by 13.15.4.

22 The meanings of the parameters are specified by 13.15.2.

23 *Throws:* As specified in Error reporting (5).

24 *Error conditions:* The errors, if any, produced by this function are implementation-defined.

```
template <class Allocator>
void stroke(const basic_brush<GraphicsSurfaces>& b,
 const basic_path_builder<GraphicsSurfaces, Allocator>& pb,
 const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
 const optional<basic_stroke_props<GraphicsSurfaces>>& sp = nullopt,
 const optional<basic_dashes<GraphicsSurfaces>>& d = nullopt,
 const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
 const optional<basic_clip_props<GraphicsSurfaces>>& cl = nullopt);
void stroke(const basic_brush<GraphicsSurfaces>& b,
 const basic_interpreted_path<GraphicsSurfaces>& ip,
 const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
 const optional<basic_stroke_props<GraphicsSurfaces>>& sp = nullopt,
 const optional<basic_dashes<GraphicsSurfaces>>& d = nullopt,
 const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
 const optional<basic_clip_props<GraphicsSurfaces>>& cl = nullopt);
```

25 *Effects:* Performs the stroking rendering and composing operation as specified by 13.15.6.

26 The meanings of the parameters are specified by 13.15.2.

27 *Throws:* As specified in Error reporting (5).

28 *Error conditions:* The errors, if any, produced by this function are implementation-defined.

```
template <class Allocator>
void fill(const basic_brush<GraphicsSurfaces>& b,
 const basic_path_builder<GraphicsSurfaces, Allocator>& pb,
 const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
 const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
 const optional<basic_clip_props<GraphicsSurfaces>>& cl = nullopt);
void fill(const basic_brush<GraphicsSurfaces>& b,
 const basic_interpreted_path<GraphicsSurfaces>& ip,
 const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
 const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
 const optional<basic_clip_props<GraphicsSurfaces>>& cl = nullopt);
```

- 29 *Effects:* Performs the filling rendering and composing operation as specified by 13.15.5.  
 30 The meanings of the parameters are specified by 13.15.2.  
 31 *Throws:* As specified in Error reporting (5).  
 32 *Error conditions:* The errors, if any, produced by this function are implementation-defined.

```
void mask(const basic_brush<GraphicsSurfaces>& b,
 const basic_brush<GraphicsSurfaces>& mb,
 const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt,
 const optional<basic_mask_props<GraphicsSurfaces>>& mp = nullopt,
 const optional<basic_render_props<GraphicsSurfaces>>& rp = nullopt,
 const optional<basic_clip_props<GraphicsSurfaces>>& cl = nullopt);
```

- 33 *Effects:* Performs the masking rendering and composing operation as specified by 13.15.7.  
 34 The meanings of the parameters are specified by 13.15.2.  
 35 *Throws:* As specified in Error reporting (5).  
 36 *Error conditions:*  
 The errors, if any, produced by this function are implementation-defined.

```
void draw_callback(const function<void(basic_unmanaged_output_surface& sfc)>& fn);
```

- 37 *Effects:* <TODO>

```
void size_change_callback(const function<void(basic_unmanaged_output_surface& sfc)>& fn);
```

- 38 *Effects:* <TODO>

```
void dimensions(basic_display_point<graphics_math_type> dp);
void dimensions(basic_display_point<graphics_math_type> dp, error_code& ec) noexcept;
```

- 39 *Effects:* <TODO>

```
void display_dimensions(basic_display_point<graphics_math_type> dp);
void display_dimensions(basic_display_point<graphics_math_type> dp, error_code& ec) noexcept;
```

- 40 *Effects:* <TODO>

```
void scaling(io2d::scaling scl) noexcept;
```

- 41 *Effects:* <TODO>

```
void user_scaling_callback(const
 function<basic_bounding_box<graphics_math_type>(const
 basic_unmanaged_output_surface&, bool&)>& fn);
```

- 42 *Effects:* <TODO>

```
void letterbox_brush(const optional<basic_brush<GraphicsSurfaces>>& b,
 const optional<basic_brush_props<GraphicsSurfaces>>& bp = nullopt) noexcept;
void letterbox_brush_props(const optional<basic_brush_props<GraphicsSurfaces>>& bp) noexcept;
```

- 43 *Effects:* <TODO>

```
void auto_clear(bool val) noexcept;
```

- 44 *Effects:* <TODO>

```
void redraw_required(bool val = true) noexcept;
```

- 45 *Effects:* <TODO>

## 14 Input

[io2d.input]

- <sup>1</sup> [*Note:* Input, such as keyboard, mouse, and touch, to user-visible surfaces will be added at a later date. This section is a placeholder. It is expected that input will be added via deriving from a user-visible surface. One possibility is that an `io_surface` class deriving from `display_surface`. This would allow developers to choose not to incur any additional costs of input support where the surface does not require user input. — *end note*]

## 15 Standalone functions [io2d.standalone]

### 15.1 Standalone functions synopsis [io2d.standalone.synopsis]

```
namespace std::experimental::io2d::v1 {
 int format_stride_for_width(format format, int width) noexcept;
 float angle_for_point(point_2d ctr, point_2d pt) noexcept;
 point_2d point_for_angle(float ang, float rad = 1.0f) noexcept;
 point_2d point_for_angle(float ang, point_2d rad) noexcept;
 point_2d arc_start(point_2d ctr, float sang, point_2d rad,
 const matrix_2d& m = matrix_2d{}) noexcept;
 point_2d arc_center(point_2d cpt, float sang, point_2d rad,
 const matrix_2d& m = matrix_2d{}) noexcept;
 point_2d arc_end(point_2d cpt, float eang, point_2d rad,
 const matrix_2d& m = matrix_2d{}) noexcept;
}
```

### 15.2 format\_stride\_for\_width [io2d.standalone.formatstrideforwidth]

```
int format_stride_for_width(format fmt, int width) noexcept;
```

- 1 *Returns:* The size in bytes of a row of pixels with a visual data format of `fmt` that is `width` pixels wide. This value may be larger than the value obtained by multiplying the number of bytes specified by the `format` enumerator specified by `fmt` by the number of pixels specified by `width`.
- 2 If `fmt == format::invalid`, this function shall return 0.

### 15.3 angle\_for\_point [io2d.standalone.angleforpoint]

```
float angle_for_point(point_2d ctr, point_2d pt) noexcept;
```

- 1 *Returns:* The angle, in radians, of `pt` as a point on a circle with a center at `ctr`. If the angle is less than `pi<float> / 180000.0f`, returns `0.0f`.

### 15.4 point\_for\_angle [io2d.standalone.pointforangle]

```
point_2d point_for_angle(float ang, float rad = 1.0f) noexcept;
point_2d point_for_angle(float ang, point_2d rad) noexcept;
```

- 1 *Requires:* If it is a `float`, `rad` is greater than `0.0f`. If it is a `point_2d`, `rad.x` or `rad.y` is greater than `0.0f` and neither is less than `0.0f`.
- 2 *Returns:* The result of rotating the point `point_2d{ 1.0f, 0.0f }`, around an origin of `point_2d{ 0.0f, 0.0f }` by `ang` radians, with a positive value of `ang` meaning counterclockwise rotation and a negative value meaning clockwise rotation, with the result being multiplied by `rad`.

### 15.5 arc\_start [io2d.standalone.arcstart]

```
point_2d arc_start(point_2d ctr, float sang, point_2d rad,
 const matrix_2d& m = matrix_2d{}) noexcept;
```

- 1 *Requires:* `rad.x` and `rad.y` are both greater than `0.0f`.
- 2 *Returns:* As-if:

```
 auto lmtx = m;
 lmtx.m20 = 0.0f; lmtx.m21 = 0.0f;
```

```

 auto pt = point_for_angle(sang, rad);
 return ctr + pt * lmtx;

```

- 3     [ *Note*: Among other things, this function is useful for determining the point at which a new figure should begin if the first item in the figure is an arc and the user wishes to clearly define its center. — *end note* ]

## 15.6 arc\_center

[io2d.standalone.arccenter]

```

point_2d arc_center(point_2d cpt, float sang, point_2d rad,
 const matrix_2d& m = matrix_2d{}) noexcept;

```

- 1     *Requires*: rad.x and rad.y are both greater than 0.0f.

- 2     *Returns*: As-if:

```

 auto lmtx = m;
 lmtx.m20 = 0.0f; lmtx.m21 = 0.0f;
 auto centerOffset = point_for_angle(two_pi<float> - sang, rad);
 centerOffset.y = -centerOffset.y;
 return cpt - centerOffset * lmtx;

```

## 15.7 arc\_end

[io2d.standalone.arcend]

```

point_2d arc_end(point_2d cpt, float eang, point_2d rad,
 const matrix_2d& m = matrix_2d{}) noexcept;

```

- 1     *Requires*: rad.x and rad.y are both greater than 0.0f.

- 2     *Returns*: As-if:

```

 auto lmtx = m;
 auto tfrm = matrix_2d::init_rotate(eang);
 lmtx.m20 = 0.0f; lmtx.m21 = 0.0f;
 auto pt = (rad * tfrm);
 pt.y = -pt.y;
 return cpt + pt * lmtx;

```

## Annex A (informative)

### Bibliography

[bibliography]

<sup>1</sup> The following is a list of informative resources intended to assist in the understanding or use of this Technical Specification.

- (1.1) — Porter, Thomas and Duff, Tom, 1984, Compositing digital images. ACM SIGGRAPH Computer Graphics. 1984. Vol. 18, no. 3, p. 253-259. DOI 10.1145/964965.808606. Association for Computing Machinery (ACM)
- (1.2) — Foley, James D. et al., *Computer graphics: principles and practice*. 2nd ed. Reading, Massachusetts : Addison-Wesley, 1996.